

Name: Ms. Prarthi Hrishit Kothari

Class: M. Sc. Bioinformatics (Part II)

Roll Number: 115

Course: M. Sc. Bioinformatics

Department: Department of Bioinformatics

Paper: Mandatory Paper II

Paper Name and Code: Perl and R Programming
Language for Bioinformatics
(GNKPSBIMJ2P504)

Academic Year: 2024-25



SGCP's

Guru Nanak Khalsa College
of Arts, Science & Commerce (Autonomous)

DEPARTMENT OF BIOINFORMATICS

CERTIFICATE

This is to certify that Ms. Prarthi Hrishit Kothari (Roll No: 115) of M. Sc. Bioinformatics (Part II) has satisfactorily completed the practical for Mandatory Paper II: Perl and R Programming Language for Bioinformatics (GNKPSBIMJ2P504) for Semester IV course prescribed by the University of Mumbai during the academic year 2024-2025.

Teacher-in-Charge
(Prof. Sermarani Nadar)

Head of Department
(Dr. Gursimran Kaur Uppal)

External Examiner

INDEX

Practical No.	Title	Page No.	Date	Signature
Perl1	PERL: Simple Programs on Variable Types used in Perl	1	16/01/2025	
Perl2	PERL: Conditional Statements and Loops	8	16/01/2025	
Perl3	PERL: Operations used on Scalar, Array and Hash Variables	20	16/01/2025	
Perl4	PERL: Subroutine	36	27/01/2025	
Perl5	PERL: References, Dereferences and Scope of Variables	41	04/02/2025	
Perl6	PERL: Regular Expressions	49	27/01/2025	
Perl7	PERL: Metacharacters, Quantifiers, Substring and File Handling	56	17/02/2025	
Perl8	PERL: Perl Formatting	70	18/02/2025	
Perl9	PERL: OOPs in Perl and DBI in Perl	81	25/02/2025	
R1	R: Introduction to R	96	17/02/2025	
R2	R: Data Visualization	105	06/03/2025	
R3	R: Bioconductor	113	10/03/2025	

PRACTICAL 1: PERL

SIMPLE PROGRAMS ON VARIABLE TYPES USED IN PERL

AIM:

To create simple programs based on the concept of variable types used in Perl.

INTRODUCTION:

Perl Programming Language

In 1987, Larry Wall created Perl, a high-level, interpreted programming language. It initially was designed for text manipulation, but it has since developed into a language with great versatility that is utilized in many different fields, including network programming, system administration, and web development. A common expansion of the term "Perl" is "Practical Extraction and Reporting Language".

Perl's syntax is similar to that of C, making it relatively easy for programmers familiar with C or C++ to learn. It supports both procedural and object-oriented programming paradigms, allowing developers to choose the approach that best fits their needs. Perl's dynamic typing and automatic memory management make it flexible and user-friendly.

Pros and Cons of Perl Programming Language

Pros	Cons
Versatility: Perl is useful for a variety of tasks, including system administration, data analysis, and web development. Powerful Text Processing: It is perfect for data extraction and text manipulation since it performs exceptionally well in jobs involving regular expressions. Rich Library: Perl's capability is further expanded by the extensive collection of modules made accessible through CPAN (Comprehensive Perl Archive Network). Community Support: A sizable and vibrant community helps with ongoing enhancements and provides assistance to new users.	Readability: For people who are unfamiliar with Perl syntax, it might be challenging to read and maintain Perl code. Performance: Perl is generally efficient, although for some operations it might not be as good as compiled languages like C or C++. Decline in Popularity: Perl's appeal has diminished as other recent languages, like as Python, have acquired acceptance, resulting in a smaller pool of potential developers.

Basics of Perl Programming Language

Perl is dynamically typed, meaning variables do not require explicit declaration of their types. Instead, the type is inferred based on the context in which they are used. The primary elements in Perl include:

1. **Variables:** Identified by sigils (\$) for scalars, @ for arrays, % for hashes).
2. **Control Structures:** Similar to C, including loops (for, while) and conditional statements (if, unless).
3. **Subroutines:** Functions that can be defined and called within the program to promote code reuse.

Data Types/Variable Types

Perl features three primary data types:

1. **Scalars:** Represent single values such as numbers or strings. Scalar variables are prefixed with a dollar sign (\$). For example:

Code	Output
my \$number1 = 42; my \$string1 = "Hello, World!"; print "Number is \$number1 \nString is \$string1";	Number is 42 String is Hello, World!

2. **Arrays:** Ordered lists of scalars prefixed with an at sign (@). Arrays can hold multiple scalar values. For example:

Code	Output
my @fruits = ("apple", "banana", "cherry"); for (\$i = 0; \$i <= 2; \$i++){ print \$fruits[i]; }	apple banana cherry

3. **Hashes (Associative Arrays):** Collections of key-value pairs prefixed with a percent sign (%). Hashes allow for efficient data retrieval based on keys. For example:

Code	Output
my %ages = ("Alice" => 30, "Bob" => 25); print "Alice's age is \$ages{"Alice"}";	Alice's age is 30

Operators used in Perl Programming Language

Operator	Description	Syntax	Example
Arithmetic Operators: Perform basic mathematical operations.			
+	Addition	\$a + \$b	my \$sum = 5 + 10;
-	Subtraction	\$a - \$b	my \$diff = 10 - 5;
*	Multiplication	\$a * \$b	my \$prod = 5 * 2;
/	Division	\$a / \$b	my \$quot = 10 / 2;
%	Modulus (remainder)	\$a % \$b	my \$rem = 10 % 3;

String Operators: Used for string manipulation.			
.	Concatenation operator, used to join two strings together.	<code>\$string1 . \$string2</code>	<code>my \$greeting = "Hello" . " World";</code>
x	Repetition operator, used to repeat a string a specified number of times.	<code>\$string x \$n</code>	<code>my \$repeat = "Ha" x 3;</code> # Output: HaHaHa
eq	String equality operator, checks if two strings are equal.	<code>\$string1 eq \$string2</code>	<code>if ("abc" eq "abc") {</code> <code>print "Equal"; }</code>
ne	String inequality operator, checks if two strings are not equal.	<code>\$string1 ne \$string2</code>	<code>if ("abc" ne "def") {</code> <code>print "Not Equal"; }</code>
lt	Less than operator, checks if the left string is less than the right string in lexicographical order.	<code>\$string1 lt \$string2</code>	<code>if ("apple" lt "banana")</code> <code>{ print "Apple is less</code> <code>than Banana"; }</code>
gt	Greater than operator, checks if the left string is greater than the right string in lexicographical order.	<code>\$string1 gt \$string2</code>	<code>if ("banana" gt "apple")</code> <code>{ print "Banana is</code> <code>greater than Apple"; }</code>
le	Less than or equal to operator, checks if the left string is less than or equal to the right string.	<code>\$string1 le \$string2</code>	<code>if ("apple" le "apple") {</code> <code>print "Apple is less than</code> <code>or equal to Apple"; }</code>
ge	Greater than or equal to operator, checks if the left string is greater than or equal to the right string.	<code>\$string1 ge \$string2</code>	<code>if ("banana" ge "apple")</code> <code>{ print "Banana is</code> <code>greater than or equal to</code> <code>Apple"; }</code>
cmp	Comparison operator, returns -1, 0, or 1 depending on whether the left string is less than, equal to, or greater than the right string.	<code>\$string1 cmp \$string2</code>	<code>my \$result = "abc" cmp</code> <code>"abc"; # Output: 0</code> <code>(equal)</code>
Comparison Operators: Compare values.			
==	Numeric equality	<code>\$a == \$b</code>	<code>if (\$x == 5) { #code }</code>
!=	Numeric inequality	<code>\$a != \$b</code>	<code>if (\$x != 5) { #code }</code>
<	Less than	<code>\$a < \$b</code>	<code>if (\$x < 10) { #code }</code>
>	Greater than	<code>\$a > \$b</code>	<code>if (\$x > 10) { #code }</code>
<=	Less than or equal to	<code>\$a <= \$b</code>	<code>if (\$x <= 10) { #code }</code>
>=	Greater than or equal to	<code>\$a >= \$b</code>	<code>if (\$x >= 10) { #code }</code>

PROGRAMS:

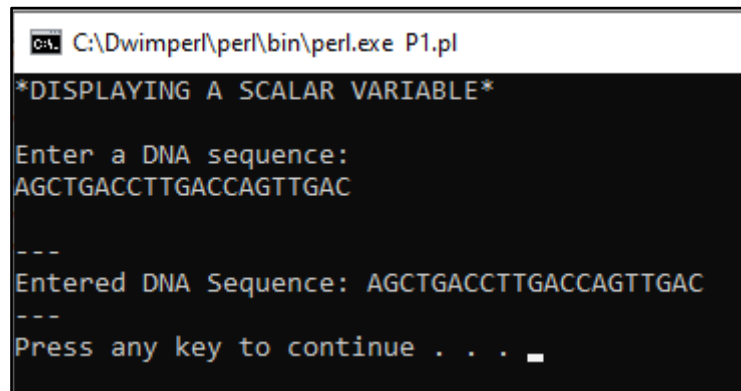
Question 1:

Write a Perl script to store DNA sequence in scalar variable entered by user and display an output.

CODE:

```
print("*DISPLAYING A SCALAR VARIABLE*");  
$dna;  
print("\n\nEnter a DNA sequence:\n");  
$dna = <stdin>;  
print("\n---\n");  
print("Entered DNA Sequence: $dna");  
print("---\n");
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe P1.pl  
*DISPLAYING A SCALAR VARIABLE*  
  
Enter a DNA sequence:  
AGCTGACCTTGACCAGTTGAC  
  
---  
Entered DNA Sequence: AGCTGACCTTGACCAGTTGAC  
---  
Press any key to continue . . . _
```

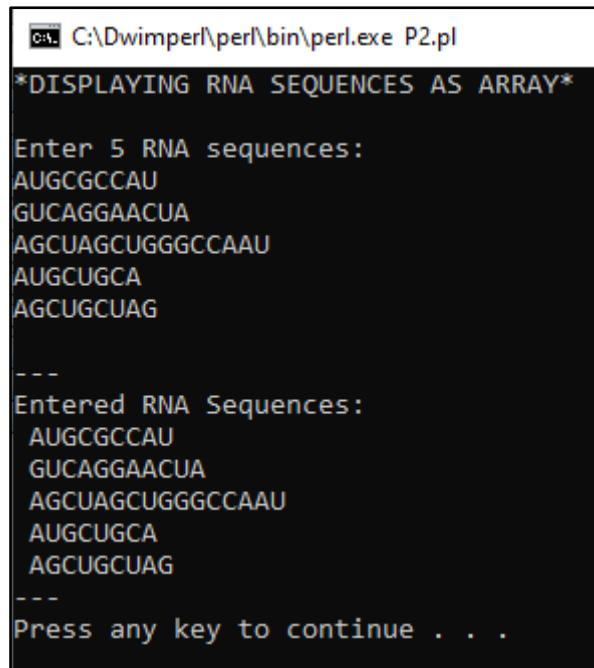
Figure 1: Displaying the user-entered DNA sequence as a scalar variable

Question 2:

Write a Perl script to ask user to enter RNA sequence using an array without loops (5 sequences).

CODE:

```
print("*DISPLAYING RNA SEQUENCES AS ARRAY*");
@rna;
print("\n\nEnter 5 RNA sequences:\n");
$rna[0] = <stdin>;
$rna[1] = <stdin>;
$rna[2] = <stdin>;
$rna[3] = <stdin>;
$rna[4] = <stdin>;
print("\n---\n");
print("Entered RNA Sequences: \n @rna");
print("---\n");
```

OUTPUT:A screenshot of a Windows command prompt window titled "C:\Dwimper\perl\bin\perl.exe P2.pl". The window shows the output of a Perl script. It starts with a header line "*DISPLAYING RNA SEQUENCES AS ARRAY*", followed by a prompt "Enter 5 RNA sequences:". The user has entered five RNA sequences: "AUGCGCCAU", "GUCAGGAACUA", "AGCUAGCUGGGCCAAU", "AUGCUGCA", and "AGCUGCUAG". After a separator line "---", the script displays "Entered RNA Sequences:" followed by the same five sequences. Another separator line "---" follows, and the prompt "Press any key to continue . . ." is shown at the bottom.

```
C:\Dwimper\perl\bin\perl.exe P2.pl
*DISPLAYING RNA SEQUENCES AS ARRAY*
Enter 5 RNA sequences:
AUGCGCCAU
GUCAGGAACUA
AGCUAGCUGGGCCAAU
AUGCUGCA
AGCUGCUAG
---
Entered RNA Sequences:
AUGCGCCAU
GUCAGGAACUA
AGCUAGCUGGGCCAAU
AUGCUGCA
AGCUGCUAG
---
Press any key to continue . . .
```

Figure 2: Displaying the 5 user-entered RNA sequences stored as an array

Question 3:

Write a Perl script to store codon using hash variables.

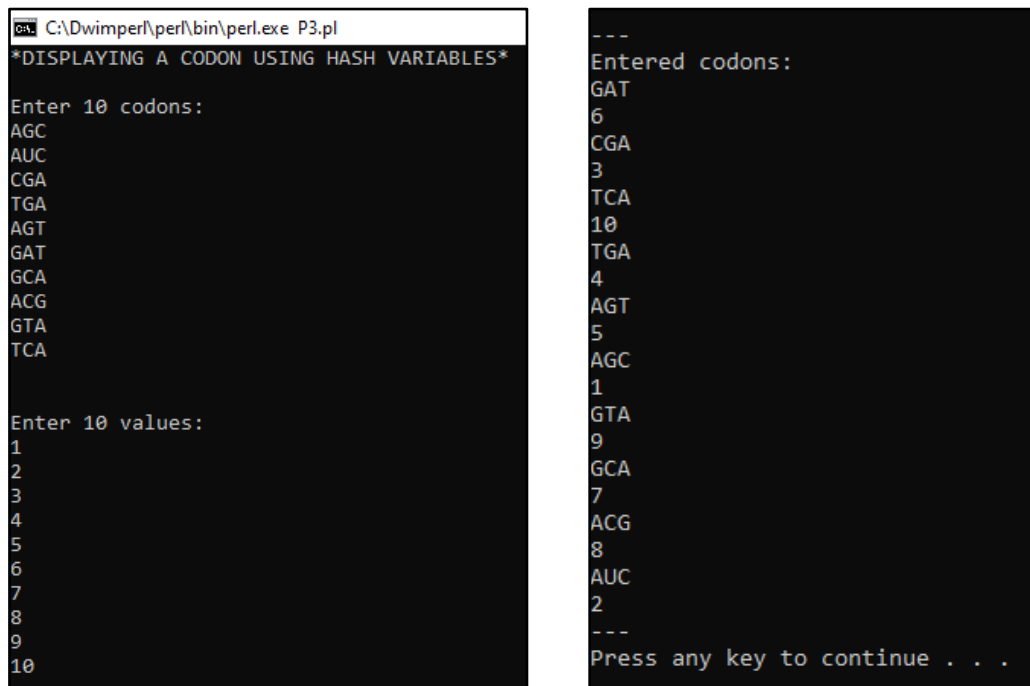
CODE:

```
print("*DISPLAYING A CODON USING HASH VARIABLES*");
@codon;
@value;
print("\n\nEnter 10 codons: \n");
$codon[0] = <stdin>;
$codon[1] = <stdin>;
$codon[2] = <stdin>;
$codon[3] = <stdin>;
$codon[4] = <stdin>;
$codon[5] = <stdin>;
$codon[6] = <stdin>;
$codon[7] = <stdin>;
$codon[8] = <stdin>;
$codon[9] = <stdin>;

print("\n\nEnter 10 values: \n");
$value[0] = <stdin>;
$value[1] = <stdin>;
$value[2] = <stdin>;
$value[3] = <stdin>;
$value[4] = <stdin>;
$value[5] = <stdin>;
$value[6] = <stdin>;
$value[7] = <stdin>;
$value[8] = <stdin>;
$value[9] = <stdin>;

%codons_hash = ($codon[0]=>$value[0], $codon[1]=>$value[1], $codon[2]=>$value[2],
$codon[3]=>$value[3],          $codon[4]=>$value[4],          $codon[5]=>$value[5],
$codon[6]=>$value[6],          $codon[7]=>$value[7],          $codon[8]=>$value[8],
$codon[9]=>$value[9]);
print("\n---\n");
print("Entered codons: \n");
print(%codons_hash);
print("---\n");
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe P3.pl
*DISPLAYING A CODON USING HASH VARIABLES*

Enter 10 codons:
AGC
AUC
CGA
TGA
AGT
GAT
GCA
ACG
GTA
TCA

Enter 10 values:
1
2
3
4
5
6
7
8
9
10

---
Entered codons:
GAT
6
CGA
3
TCA
10
TGA
4
AGT
5
AGC
1
GTA
9
GCA
7
ACG
8
AUC
2
---
Press any key to continue . . .
```

Figure 3: Displaying the 10 codons stored as hash variables

RESULTS:

Programs based on the concept of variable types used in Perl programming language were coded and executed to display the user-entered DNA sequence as a scalar variable, to display the 5 user-entered RNA sequences stored as an array and to display the 10 codons stored as hash variables.

CONCLUSION:

Programs based on the concept of variable types used in Perl programming language were coded and executed.

REFERENCES:

1. GeeksforGeeks. (2022, June 22). Perl | Data Types. GeeksforGeeks. <https://www.geeksforgeeks.org/perl-data-types/>
2. GeeksforGeeks. (2023, April 13). Introduction to Perl. GeeksforGeeks. <https://www.geeksforgeeks.org/introduction-to-perl/>
3. Perl - Operators. (n.d.). https://www.tutorialspoint.com/perl/perl_operators.htm
4. Perl Operator Types - javatpoint. (n.d.). [www.javatpoint.com. https://www.javatpoint.com/perl-operator-types](https://www.javatpoint.com/perl-operator-types)

PRACTICAL 2: PERL CONDITIONAL STATEMENTS AND LOOPS

AIM:

To create simple programs based on the concept of conditional statements and loops used in Perl.

INTRODUCTION:

Larry Wall created Perl in 1987 as a high-level, interpreted programming language initially for text manipulation. Over time, it evolved into a versatile language used in web development, system administration, and network programming. Perl's syntax resembles C, making it accessible for C or C++ programmers. It supports both procedural and object-oriented programming, featuring dynamic typing and automatic memory management, enhancing its flexibility and user-friendliness. The acronym "Practical Extraction and Reporting Language" is commonly associated with Perl.

Conditional Statements in Perl Programming Language

In Perl, conditional statements serve as fundamental constructs that enable the execution of various code paths according to whether particular conditions evaluate to true or false. These statements allow for dynamic behavior based on input or state and are essential for program decision-making.

Conditional Statement	Description	Syntax	Example
if Statement	The simplest form of a conditional statement, the if statement executes a block of code if the specified condition is true.	<pre>if (condition) { # code to execute if condition is true }</pre>	<pre>my \$x = 5; if (\$x > 3) { print "x is greater than 3\n"; }</pre>
if-else Statement	This structure allows for an alternative code path if the condition evaluates to false.	<pre>if (condition) { # code if condition is true } else { # code if condition is false }</pre>	<pre>my \$y = 2; if (\$y > 3) { print "y is greater than 3\n"; } else { print "y is not greater than 3\n"; }</pre>
if-elsif-else Statement	This construct is used for checking multiple conditions sequentially. If one condition is true, its	<pre>if (condition1) { # code for condition1 } elsif (condition2) {</pre>	<pre>my \$z = 10; if (\$z < 5) { print "z is less than 5\n";</pre>

	corresponding block executes, and the rest are skipped.	# code for condition2 } else { # code if none of the above conditions are true }	} elsif (\$z == 10) { print "z equals 10\n"; } else { print "z is greater than 5 but not equal to 10\n"; }
unless Statement	The unless statement executes a block of code only if the specified condition is false, effectively serving as the opposite of an if statement.	unless (condition) { # code to execute if condition is false }	my \$a = 0; unless (\$a) { print "a is false\n"; # This will execute because \$a is 0 (false) }
Ternary Operator	The ternary operator provides a shorthand way to write simple if-else statements. It evaluates a condition and returns one of two values based on whether the condition is true or false.	condition ? value_if_true : value_if_false;	my \$age = 18; my \$status = (\$age >= 18) ? "Adult" : "Minor"; print "\$status\n"; # Output: Adult

Loops in Perl Programming Language

In Perl, loops are control structures that permit a block of code to be executed repeatedly under particular circumstances. They are necessary for effectively completing repetitive tasks. Perl has a variety of loop types, each catering to distinct syntactical preferences and use cases.

Type of Loop	Description	Syntax	Example
for Loop	The for loop allows for concise iteration, combining initialization, condition checking, and incrementing/decrementing in a single line. This structure is particularly useful when the number of iterations is known beforehand.	for (initializing statement; conditional statement; update statement) { # code to execute }	for (my \$count = 1; \$count < 10; \$count++) { print "My count is: \$count\n"; }
foreach Loop	The foreach loop simplifies iterating over elements in an array or list. It assigns each element to a variable in each iteration, allowing for easy access to the values.	foreach variable (array) { # code to execute }	my @colors = ('red', 'blue', 'yellow'); foreach my \$color (@colors) { print "Color: \$color\n"; }

while Loop	The while loop repeatedly executes a block of code as long as the specified condition evaluates to true. The condition is checked before each iteration of the loop.	while (condition) { # code to execute }	my \$count = 10; while (\$count > 0) { print "Countdown is: \$count\n"; \$count--; }
do...while Loop	The do...while loop executes a block of code at least once before checking the condition. This makes it useful when you want to ensure that the loop body runs at least once regardless of the condition.	do { # code to execute } while (condition);	my \$count = 10; do { print "Countdown is: \$count\n"; \$count--; } while (\$count > 0);
until Loop	The until loop is the opposite of the while loop. It executes a block of code until the specified condition becomes true. Like while, the condition is evaluated before executing the block.	until (condition) { # code to execute }	my \$count = 1; until (\$count > 10) { print "Countup is: \$count\n"; \$count++; }

PROGRAMS:

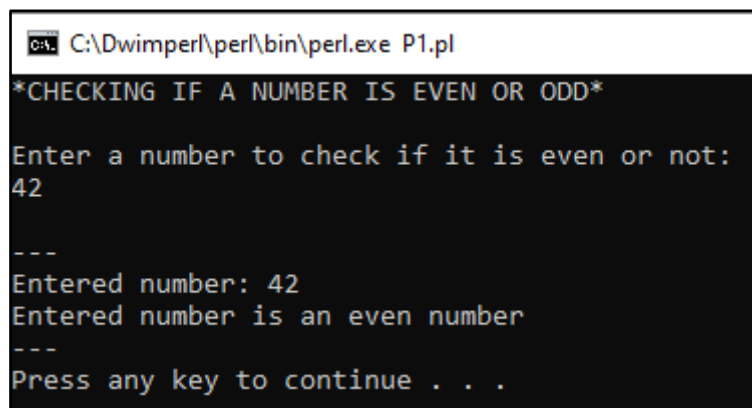
Question 1:

Write a Perl script to ask user to enter a number and check whether entering number is even or odd.

CODE:

```
print("*CHECKING IF A NUMBER IS EVEN OR ODD*");
$num;
print("\n\nEnter a number to check if it is even or not:\n");
$num = <stdin>;
if($num % 2 == 0){
    print("\n---\n");
    print("Entered number: $num");
    print("Entered number is an even number \n");
    print("---\n");
}
else{
    print("\n---\n");
    print("Entered number: $num");
    print("Entered number is an odd number \n");
    print("---\n");
}
```

OUTPUT:



```
C:\Dwimper\perl\bin\perl.exe P1.pl
*CHECKING IF A NUMBER IS EVEN OR ODD*

Enter a number to check if it is even or not:
42

---
Entered number: 42
Entered number is an even number
---
Press any key to continue . . .
```

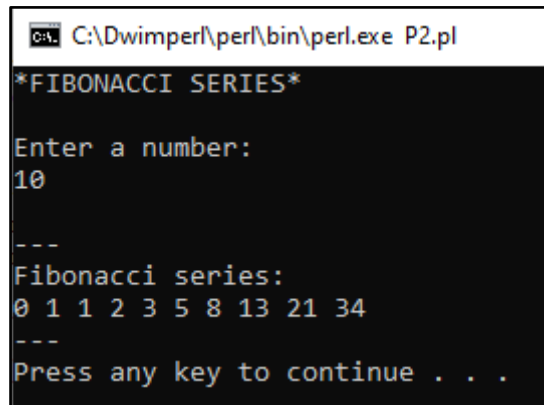
Figure 1: Checking and displaying the number if it is even or odd

Question 2:

Write a Perl script to ask user to enter number to display Fibonacci series.

CODE:

```
print("*FIBONACCI SERIES*");
$num;
print("\n\nEnter a number:\n");
$num = <stdin>;
$num1 = 0;
$num2 = 1;
print("\n---\nFibonacci series: \n");
print("$num1 ");
print("$num2 ");
for ($i = 1; $i <= $num-2; $i++){
    $num3 = $num1 + $num2;
    print("$num3 ");
    $num1 = $num2;
    $num2 = $num3;
}
print("\n---\n");
```

OUTPUT:

```
C:\Dwimperl\perl\bin\perl.exe P2.pl
*FIBONACCI SERIES*
Enter a number:
10
---
Fibonacci series:
0 1 1 2 3 5 8 13 21 34
---
Press any key to continue . . .
```

Figure 2: Displaying the Fibonacci series until the user-entered number

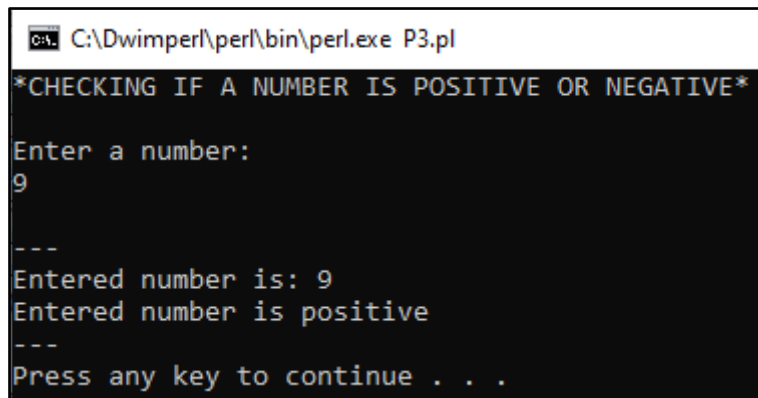
Question 3:

Write a Perl script to ask user to enter a number and check whether entering number is negative or positive.

CODE:

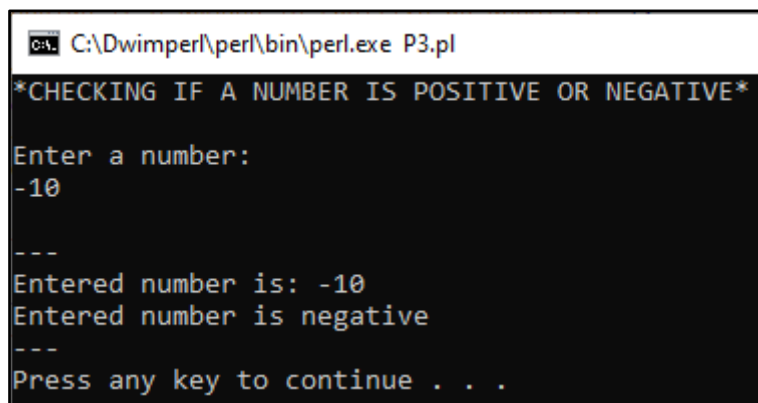
```
print("*CHECKING IF A NUMBER IS POSITIVE OR NEGATIVE*");
$num;
print("\n\nEnter a number:\n");
$num = <stdin>;
print("\n---\n");
print("Entered number is: $num");
if($num<0){
    print("Entered number is negative\n");
}
elsif($num>0){
    print("Entered number is positive\n");
}
else{
    print("Entered number is neither positive nor negative\n");
}
print("---\n");
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe P3.pl
*CHECKING IF A NUMBER IS POSITIVE OR NEGATIVE*
Enter a number:
9
---
Entered number is: 9
Entered number is positive
---
Press any key to continue . . .
```

Figure 3: Displaying the output when the user-entered number is positive



```
C:\Dwimperl\perl\bin\perl.exe P3.pl
*CHECKING IF A NUMBER IS POSITIVE OR NEGATIVE*
Enter a number:
-10
---
Entered number is: -10
Entered number is negative
---
Press any key to continue . . .
```

Figure 4: Displaying the output when the user-entered number is negative


```
C:\Dwimper\perl\bin\perl.exe P3.pl
*CHECKING IF A NUMBER IS POSITIVE OR NEGATIVE*
Enter a number:
0
---
Entered number is: 0
Entered number is neither positive nor negative
---
Press any key to continue . . .
```

Figure 5: Displaying the output when the user-entered number is neither positive nor negative

Question 4:

Write a Perl script to ask user to enter RNA sequence using an array with for and foreach loops.

CODE:

```
print("*RNA SEQUENCE USING ARRAY WITH FOR AND FOREACH LOOP*");
@rna;
print("\nEnter RNA sequence: \n");
for($i=0; $i<=4; $i++){
    $rna[$i] = <stdin>;
}
print("\n---\n");
print("To display the sequence: \n");
foreach $rna_seq1(@rna){
    print("Element: $rna_seq1");
}
print("---\n");
```

OUTPUT:

```
*RNA SEQUENCE USING ARRAY WITH FOR AND FOREACH LOOP*
Enter RNA sequence:
AUCGC
CGUAGA
GUCAGAC
ACGU
GUCGA
---
To display the sequence:
Element: AUCGC
Element: CGUAGA
Element: GUCAGAC
Element: ACGU
Element: GUCGA
---
Press any key to continue . . .
```

Figure 6: Displaying the user-entered RNA sequences using an array with for and foreach loops

Question 5:

Write a Perl script to ask user to enter DNA sequence using a hashes and display keys and values separately.

CODE:

```
print("*DNA SEQUENCE USING HASH*");
%dna;
@name;
@value;
for ($i=0; $i<=4; $i++){
    print("\n\nEnter a key:\n");
    $name[$i] = <stdin>;
    print("\nEnter a DNA sequence (value):\n");
    $value[$i] = <stdin>;
    $dna{$name[$i]} = $value[$i];
}

print("\n---\n");
print("HASH: \n");
print(%dna);

print("\nKEYS: \n");
@name1 = keys %dna;
for ($i=0; $i<=4; $i++){
    print($name1[$i]);
}
print("\nVALUES: \n");
@value1 = values %dna;
for ($i=0; $i<=4; $i++){
    print($value1[$i]);
}

print("\n---\n")
```

OUTPUT:

```
C:\Dwimperl\perl\bin\perl.exe P5.pl
*DNA SEQUENCE USING HASH*

Enter a key:
1st

Enter a DNA sequence (value):
AGCTGAGCA

Enter a key:
2nd

Enter a DNA sequence (value):
GCTAGCTAGAA

Enter a key:
3rd

Enter a DNA sequence (value):
CGATCGATAAGCTA

Enter a key:
4th

Enter a DNA sequence (value):
GCTAGCTGCAT

Enter a key:
5th

Enter a DNA sequence (value):
GCTAGCCTGACTA

---
HASH:
5th
GCTAGCCTGACTA
3rd
CGATCGATAAGCTA
1st
AGCTGAGCA
2nd
GCTAGCTAGAA
4th
GCTAGCTGCAT

KEYS:
5th
3rd
1st
2nd
4th

VALUES:
GCTAGCCTGACTA
CGATCGATAAGCTA
AGCTGAGCA
GCTAGCTAGAA
GCTAGCTGCAT

---
Press any key to continue . . .
```

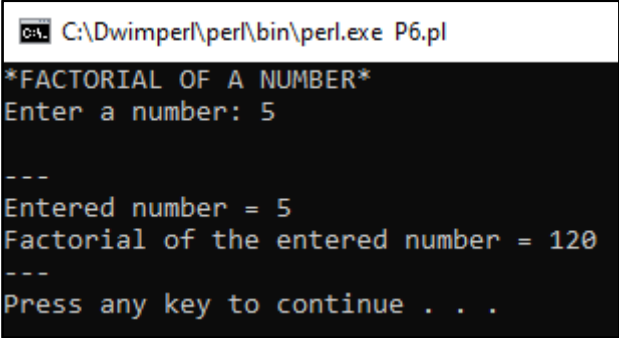
Figure 7: Displaying the keys and values of user-entered DNA sequences separately using hash

Question 6:

Write a Perl script to ask user to enter number and find factorial of an entered number.

CODE:

```
print("*FACTORIAL OF A NUMBER*");  
$num;  
print("\nEnter a number: ");  
$num = <stdin>;  
$fact = 1;  
for ($i=1; $i<=$num; $i++){  
    $fact = $fact * $i;  
}  
print("\n---\n");  
print("Entered number = $num");  
print("Factorial of the entered number = $fact");  
print("\n---\n");
```

OUTPUT:

```
C:\Dwimper\perl\bin\perl.exe P6.pl  
*FACTORIAL OF A NUMBER*  
Enter a number: 5  
---  
Entered number = 5  
Factorial of the entered number = 120  
---  
Press any key to continue . . .
```

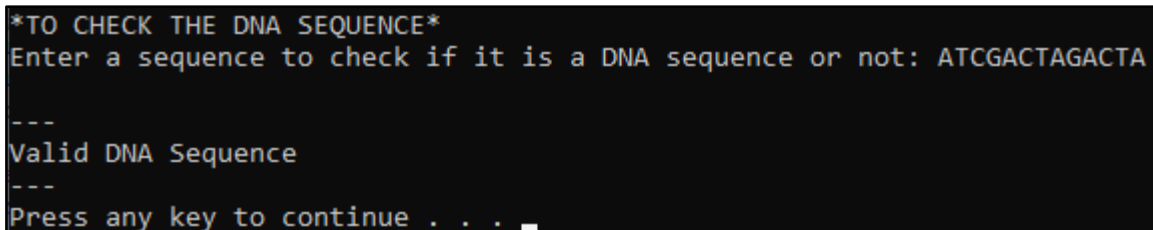
Figure 8: Displaying the factorial of the user-entered number

Question 7:

Write a Perl script to store DNA sequence (by user) and check entered sequence is DNA or not.

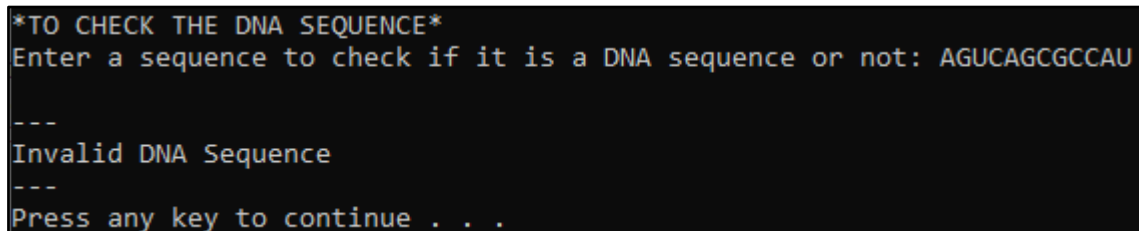
CODE:

```
print("*TO CHECK THE DNA SEQUENCE*");
print("\nEnter a sequence to check if it is a DNA sequence or not: ");
chomp($dna = <stdin>);
$len=length($dna);
for($i=0; $i<$len; $i++){
    $ch=substr($dna,$i,1);
    if($ch eq "a" or $ch eq "t" or $ch eq "g" or $ch eq "c" or $ch eq "A" or $ch eq "T" or
$ch eq "G" or $ch eq "C"){
        $f=1;
    }
    else{
        $f=0;
        break;
    }
}
print("\n---\n");
if($f==1){
    print ("Valid DNA Sequence");
}
else{
    print ("Invalid DNA Sequence");
}
print("\n---\n");
```

OUTPUT:

```
*TO CHECK THE DNA SEQUENCE*
Enter a sequence to check if it is a DNA sequence or not: ATCGACTAGACTA
---
Valid DNA Sequence
---
Press any key to continue . . .
```

Figure 9: Displaying the output when a valid DNA sequence is entered



```
*TO CHECK THE DNA SEQUENCE*
Enter a sequence to check if it is a DNA sequence or not: AGUCAGCGCCAU
---
Invalid DNA Sequence
---
Press any key to continue . . .
```

Figure 10: Displaying the output when an invalid DNA sequence is entered

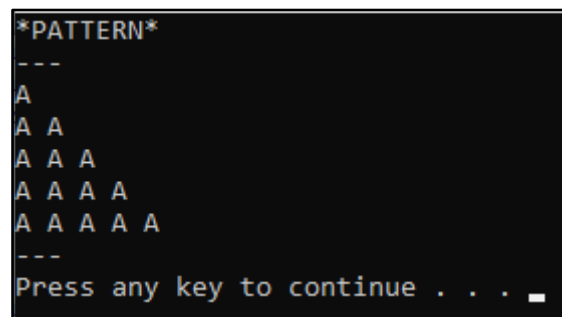
Question 8:

Write a Perl script to display following pattern:

```
A A A A A
A A A A
A A A
A A
A A
A
```

CODE:

```
print("*PATTERN*");
print("\n---\n");
for ($i=1; $i<=5; $i++){
    for($j=1; $j<=$i; $j++){
        print("A ");
    }
    print("\n");
}
print("---\n");
```

OUTPUT:

```
*PATTERN*
---
A
A A
A A A
A A A A
A A A A A
---
Press any key to continue . . . _
```

Figure 10: Displaying the pattern using Perl script

RESULTS:

Simple programs were coded and executed based on the concept of conditional statements and loops used in Perl to check and display the number if it is even or odd, to display the Fibonacci series until the user-entered number, to display the output when the user-entered number is positive, when the user-entered number is negative and when the user-entered number is neither positive nor negative, to display the user-entered RNA sequences using an array with for and foreach loops, to display the keys and values of user-entered DNA sequences separately using hash, to display the factorial of the user-entered number, to display the output when a valid DNA sequence is entered, to display the output when an invalid DNA sequence is entered and to display the given pattern using Perl script.

CONCLUSION:

Simple programs were coded and executed based on the concept of conditional statements and loops used in Perl.

REFERENCES:

1. GeeksforGeeks. (2022, August 24). Perl | Decision Making (if, ifelse, Nested-if, ifelsif ladder, unless, unlesselse, unlesselsif). GeeksforGeeks. <https://www.geeksforgeeks.org/perl-decision-making-if-if-else-nested-if-if-elsif-ladder-unless-unless-else-unless-elsif/>
 2. Perl if statement. (2023, February 9). Perl Tutorial. <https://www.perltutorial.org/perl-if/>
 3. Simplilearn. (2023, February 21). Perl programming for beginners. Simplilearn.com. <https://www.simplilearn.com/perl-programming-for-beginners-article>
 4. Perl - Loops. (n.d.). https://www.tutorialspoint.com/perl/perl_loops.htm
 5. Perl for Loop - javatpoint. (n.d.). www.javatpoint.com. <https://www.javatpoint.com/perl-for-loop>
-

PRACTICAL 3: PERL

OPERATORS USED ON SCALAR, ARRAY AND HASH VARIABLES

AIM:

To create simple programs based on the concept of operators used on scalar, array and hash variables used in Perl.

INTRODUCTION:

Larry Wall created Perl in 1987 as a high-level, interpreted programming language initially for text manipulation. Over time, it evolved into a versatile language used in web development, system administration, and network programming. Perl's syntax resembles C, making it accessible for C or C++ programmers. It supports both procedural and object-oriented programming, featuring dynamic typing and automatic memory management, enhancing its flexibility and user-friendliness. The acronym "Practical Extraction and Reporting Language" is commonly associated with Perl.

Types of Variables and their Operators

1. Scalar Variables

In Perl, a scalar variable is designed to hold a single unit of data, which can be of various types including integers, floating-point numbers, strings, or even references to other data structures. Scalar variables are denoted by a dollar sign (\$) preceding the variable name.

For example:

```
$age = 25;      # Integer
$name = "John"; # String
$salary = 50000.50; # Floating-point
```

When using scalar variables, Perl automatically manages memory allocation based on the type of data being stored. It allows for dynamic typing, meaning you do not need to declare the type explicitly. You can also perform various operations on scalars, such as arithmetic or string concatenation, depending on the context in which they are used.

2. Array Variables

An array variable in Perl is used to store ordered lists of scalars. Array variables are prefixed with an at sign (@). Each element in an array can be accessed using its index, which starts at zero.

For example:

```
@ages = (25, 30, 35); # Array of integers
@names = ("Alice", "Bob", "Charlie"); # Array of strings
```


To access a specific element in an array, you use the dollar sign followed by the array name and the index in square brackets:

```
print $ages[0]; # Outputs: 25
print $names[1]; # Outputs: Bob
```

Arrays in Perl can dynamically grow and shrink as needed, allowing for flexible data manipulation.

3. Hash Variables

A hash variable is a collection of key/value pairs and is denoted by a percent sign (%). Each key is unique within a hash and is used to retrieve its corresponding value. For example:

```
%fruit_colors = (
    "apple" => "red",
    "banana" => "yellow",
    "grape" => "purple"
);
```

You can access values in a hash by using the key within curly braces:

```
print $fruit_colors{"apple"}; # Outputs: red
```

Hashes provide an efficient way to manage associative arrays where you want to look up values based on keys rather than numerical indices.

Operators

Operator	Symbol	Description	Syntax	Example
ARITHMETIC OPERATORS				
Addition	+	Adds two numeric values.	\$result = \$a + \$b;	\$sum = 5 + 3;
Subtraction	-	Subtracts the second numeric value from the first.	\$result = \$a - \$b;	\$difference = 5 - 3;
Multiplication	*	Multiplies two numeric values.	\$result = \$a * \$b;	\$product = 5 * 3;
Division	/	Divides the first numeric value by the second.	\$result = \$a / \$b;	\$quotient = 6 / 3;
Modulus	%	Returns the remainder of division.	\$result = \$a % \$b;	\$remainder = 5 % 2;
Exponent	**	Performs exponential (power) calculation on operators.	\$result = \$a ** \$b;	\$exponent = 5 ** 2;

COMPARISON OPERATORS				
Numeric Equality	<code>==</code>	Checks if two numeric values are equal.	<code>if (\$a == \$b) { ... }</code>	<code>if (5 == 5) { print "Equal"; }</code>
Numeric Not Equal	<code>!=</code>	Checks if two numeric values are not equal.	<code>if (\$a != \$b) { ... }</code>	<code>if (5 != 3) { print "Not Equal"; }</code>
Less Than	<code><</code>	Checks if the first value is less than the second.	<code>if (\$a < \$b) { ... }</code>	<code>if (3 < 5) { print "True"; }</code>
Greater Than	<code>></code>	Checks if the first value is greater than the second.	<code>if (\$a > \$b) { ... }</code>	<code>if (5 > 3) { print "True"; }</code>
Less Than or Equal To	<code><=</code>	Checks if the first value is less than or equal to the second.	<code>if (\$a <= \$b) { ... }</code>	<code>if (3 <= 3) { print "True"; }</code>
Greater Than or Equal To	<code>>=</code>	Checks if the first value is greater than or equal to the second.	<code>if (\$a >= \$b) { ... }</code>	<code>if (5 >= 4) { print "True"; }</code>
Numeric Comparison	<code><=></code>	Compares two numbers and returns: 1 if the left operand is greater than the right 0 if they are equal -1 if the left operand is less than the right.	<code>\$result = \$a <=> \$b;</code>	<code>\$lowVar = 8; \$midVar = 10; print(\$lowVar <=> \$midVar); # Outputs: -1</code>
STRING – COMPARISON OPERATORS				
String Concatenation	<code>.</code>	Joins two strings together.	<code>\$result = \$a . \$b;</code>	<code>\$full_name = "John" . " " . "Doe";</code>
String Equality	<code>eq</code>	Checks if two strings are equal.	<code>if (\$str1 eq \$str2) { ... }</code>	<code>if ("hello" eq "hello") { print "Equal"; }</code>
String Not Equal	<code>ne</code>	Checks if two strings are not equal.	<code>if (\$str1 ne \$str2) { ... }</code>	<code>if ("hello" ne "world") { print "Not Equal"; }</code>
String Less Than	<code>lt</code>	Checks if one string is less than another based on ASCII values.	<code>if (\$str1 lt \$str2) { ... }</code>	<code>if ("apple" lt "banana") { print "True"; }</code>
String Greater Than	<code>gt</code>	Checks if one string is greater than another based on ASCII values.	<code>if (\$str1 gt \$str2) { ... }</code>	<code>if ("banana" gt "apple") { print "True"; }</code>
Less Than or Equal To	<code>le</code>	Checks if the left operand is less than or equal to the right	<code>if (\$a le \$b)</code>	<code>if (5 le 5) { print "True"; }</code>

		operand. Used for numeric comparisons.		
Greater Than or Equal To	ge	Checks if the left operand is greater than or equal to the right operand. Used for numeric comparisons.	if (\$a ge \$b)	if (10 ge 5) { print "True"; }
String Comparison	cmp	Compares two strings and returns: -1 if the left string is less than the right 0 if they are equal 1 if the left string is greater than the right.	\$result = \$str1 cmp \$str2;	\$a = "apple"; \$b = "banana"; \$c = \$a cmp \$b; # Returns -1

PROGRAMS:

Question 1:

Write a Perl script accept two number and a string and perform the following operations:

- a. Perl Arithmetic Operators
- b. Miscellaneous Operators

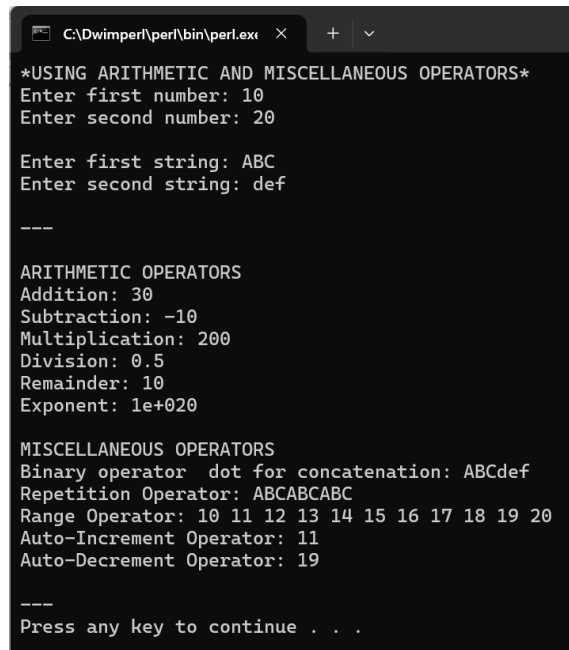
CODE:

```
print("*USING ARITHMETIC AND MISCELLANEOUS OPERATORS*");
print("\nEnter first number: ");
$num1 = <stdin>;
print("Enter second number: ");
$num2 = <stdin>;
print("\nEnter first string: ");
chomp($str1 = <stdin>);
print("Enter second string: ");
chomp($str2 = <stdin>);

print("\n---\n");
print("\nARITHMETIC OPERATORS");
$ans = $num1 + $num2;
print("\nAddition: $ans");
$ans = $num1 - $num2;
print("\nSubtraction: $ans");
$ans = $num1 * $num2;
print("\nMultiplication: $ans");
$ans = $num1 / $num2;
print("\nDivision: $ans");
$ans = $num1 % $num2;
print("\nRemainder: $ans");
$ans = $num1 ** $num2;
print("\nExponent: $ans");

print("\n\nMISCELLANEOUS OPERATORS");
$ans = $str1.$str2;
print("\nBinary operator dot for concatenation: $ans");
$ans = $str1 x 3;
print("\nRepetition Operator: $ans");
@ans = ($num1 .. $num2);
print("\nRange Operator: @ans");
$num1++;
print("\nAuto-Increment Operator: $num1");
$num2--;
print("\nAuto-Decrement Operator: $num2\n");
print("\n---\n");
```

OUTPUT:



```
C:\Dwimper\perl\bin\perl.exe x + v
*USING ARITHMETIC AND MISCELLANEOUS OPERATORS*
Enter first number: 10
Enter second number: 20

Enter first string: ABC
Enter second string: def

---

ARITHMETIC OPERATORS
Addition: 30
Subtraction: -10
Multiplication: 200
Division: 0.5
Remainder: 10
Exponent: 1e+020

MISCELLANEOUS OPERATORS
Binary operator dot for concatenation: ABCdef
Repetition Operator: ABCABCABC
Range Operator: 10 11 12 13 14 15 16 17 18 19 20
Auto-Increment Operator: 11
Auto-Decrement Operator: 19

---
Press any key to continue . . .
```

Figure 1: Displaying the output after performing operations on user-entered numbers and strings using arithmetic operators and miscellaneous operators

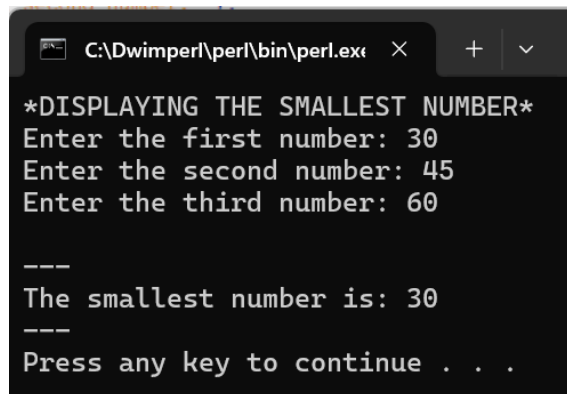
Question 2:

Write a Perl script to accept three number and display smallest number.

CODE:

```
print("*DISPLAYING THE SMALLEST NUMBER*");
print("\nEnter the first number: ");
$num1 = <stdin>;
print("Enter the second number: ");
$num2 = <stdin>;
print("Enter the third number: ");
$num3 = <stdin>;
print("\n---\n");
if($num1<$num2 && $num1<$num3){
    print("The smallest number is: $num1");
}
elsif ($num2<$num1 && $num2<$num3){
    print("The smallest number is: $num2");
}
elsif ($num3<$num1 && $num3<$num2){
    print("The smallest number is: $num3");
}
print("---\n");
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe x + v
*DISPLAYING THE SMALLEST NUMBER*
Enter the first number: 30
Enter the second number: 45
Enter the third number: 60

---
The smallest number is: 30
---
Press any key to continue . . .
```

Figure 2: Displaying the smallest number out of the three user-entered numbers

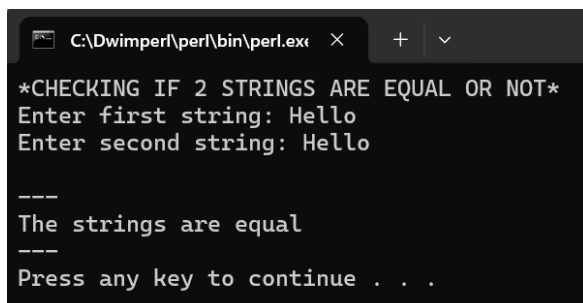
Question 3:

Write a Perl script to enter two string and check whether its equal or not.

CODE:

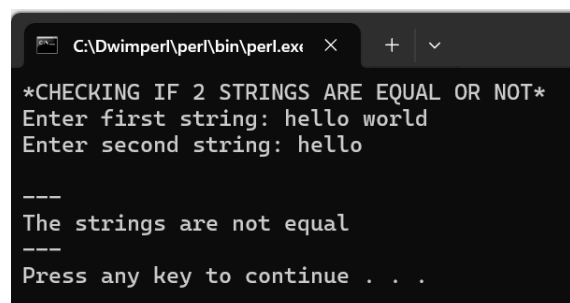
```
print("*CHECKING IF 2 STRINGS ARE EQUAL OR NOT*");
print("\nEnter first string: ");
chomp($str1 = <stdin>);
print("Enter second string: ");
chomp($str2 = <stdin>);
print("\n---\n");
if($str1 eq $str2){
    print("The strings are equal");
}
else{
    print("The strings are not equal");
}
print("\n---\n");
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe x + v
*CHECKING IF 2 STRINGS ARE EQUAL OR NOT*
Enter first string: Hello
Enter second string: Hello

---
The strings are equal
---
Press any key to continue . . .
```



```
C:\Dwimperl\perl\bin\perl.exe x + v
*CHECKING IF 2 STRINGS ARE EQUAL OR NOT*
Enter first string: hello world
Enter second string: hello

---
The strings are not equal
---
Press any key to continue . . .
```

Figure 3: Checking and displaying the output when two user-entered strings are equal and not equal

Question 4:

Write a Perl script store elements in an array and perform the following:

- Find length of an array
- Add one element at end of an array
- Remove one element at beginning of an array
- Add one element at beginning of an array
- Remove one element at end of an array

CODE:

```
print("*ARRAY OPERATIONS*");
print("\nEnter elements of the array separated by spaces: ");
chomp(my $input = <STDIN>);
my @arr = split(' ', $input);

print("\n---\n");
my $length = scalar @arr;
print("Length of the array: $length\n");

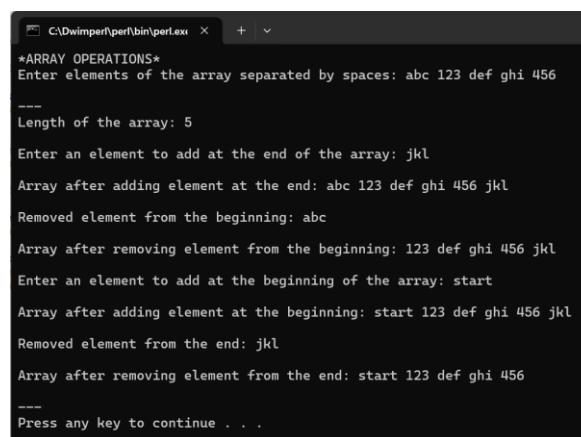
print("\nEnter an element to add at the end of the array: ");
chomp(my $add_end = <STDIN>);
push @arr, $add_end;
print("\nArray after adding element at the end: @arr\n");

my $removed_start = shift @arr;
print("\nRemoved element from the beginning: $removed_start\n");
print("\nArray after removing element from the beginning: @arr\n");

print("\nEnter an element to add at the beginning of the array: ");
chomp(my $add_start = <STDIN>);
unshift @arr, $add_start;
print("\nArray after adding element at the beginning: @arr\n");

my $removed_end = pop @arr;
print("\nRemoved element from the end: $removed_end\n");
print("\nArray after removing element from the end: @arr\n");
print("\n---\n");
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe x + v
*ARRAY OPERATIONS*
Enter elements of the array separated by spaces: abc 123 def ghi 456
---
Length of the array: 5
Enter an element to add at the end of the array: jkl
Array after adding element at the end: abc 123 def ghi 456 jkl
Removed element from the beginning: abc
Array after removing element from the beginning: 123 def ghi 456 jkl
Enter an element to add at the beginning of the array: start
Array after adding element at the beginning: start 123 def ghi 456 jkl
Removed element from the end: jkl
Array after removing element from the end: start 123 def ghi 456
---
Press any key to continue . . .
```

Figure 4: Displaying the output after performing array operations on a user-entered array

Question 5:

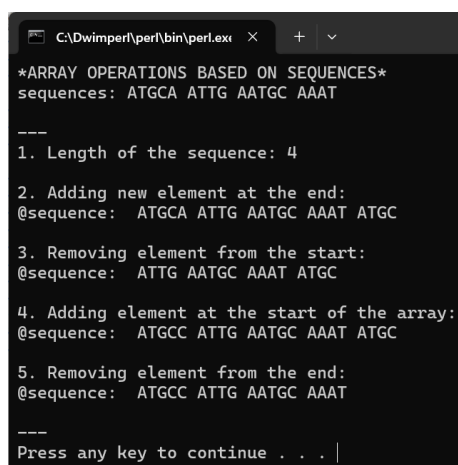
Write a Perl script create elements in an array like ATGCA, ATTG, AATGC, AAAT and perform the following:

- Find length of an array
- Add one element i.e. ATGC at bottom of an array
- Remove one element at beginning of an array
- Add one element i.e. ATGCC at top of an array
- Remove one element at end of an array

CODE:

```
print("*ARRAY OPERATIONS BASED ON SEQUENCES*");
@sequence = ("ATGCA","ATTG","AATGC","AAAT");
print("\nsequences: @sequence\n");
$len = $#sequence+1;
print("\n---\n");
print( "1. Length of the sequence: $len\n");
print( "\n2. Adding new element at the end: \n");
push(@sequence,"ATGC");
print( "\n\n@sequence: @sequence\n");
print( "\n3. Removing element from the start: \n");
shift(@sequence);
print("\n\n@sequence: @sequence\n");
print("\n4. Adding element at the start of the array: \n");
unshift(@sequence,"ATGCC");
print("\n\n@sequence: @sequence\n");
print("\n5. Removing element from the end: \n");
pop(@sequence);
print("\n\n@sequence: @sequence\n");
print("\n\n---\n");
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe
*ARRAY OPERATIONS BASED ON SEQUENCES*
sequences: ATGCA ATTG AATGC AAAT

---
1. Length of the sequence: 4

2. Adding new element at the end:
@sequence:  ATGCA ATTG AATGC AAAT ATGC

3. Removing element from the start:
@sequence:  ATTG AATGC AAAT ATGC

4. Adding element at the start of the array:
@sequence:  ATGCC ATTG AATGC AAAT ATGC

5. Removing element from the end:
@sequence:  ATGCC ATTG AATGC AAAT

---
Press any key to continue . . . |
```

Figure 5: Displaying the output after performing array operations based on an array consisting of DNA sequences

Question 6:

Write a Perl script to create an array and perform following operations such as merge, reverse and sorting.

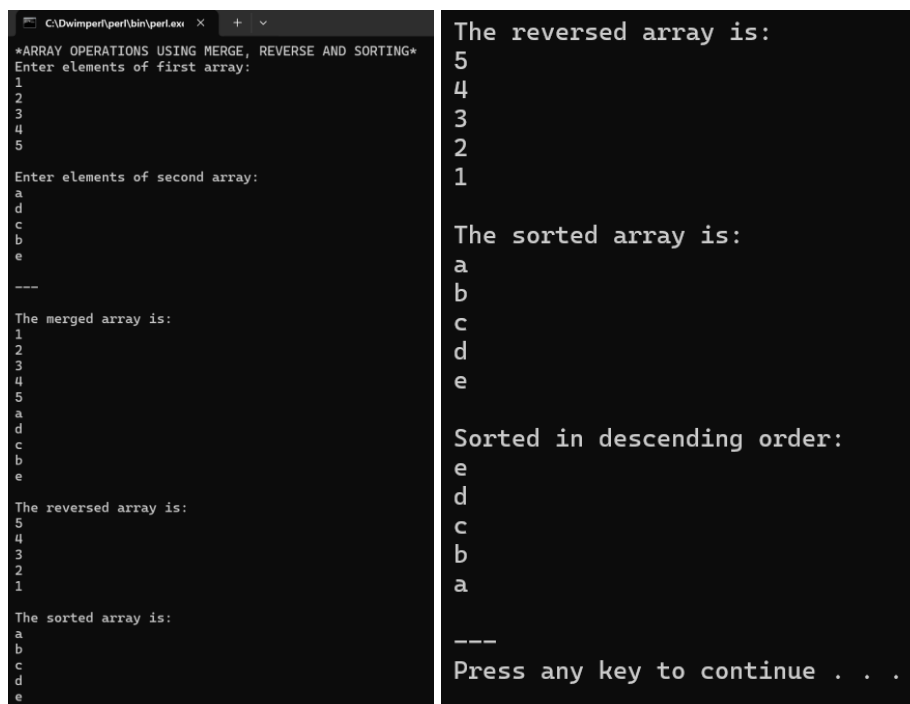
CODE:

```
print("*ARRAY OPERATIONS USING MERGE, REVERSE AND SORTING*");
print("\nEnter elements of first array: \n");
@arr;
@arr1;
for($i=0; $i<5; $i++){
    $arr[$i]=<stdin>;
}

print("\nEnter elements of second array: \n");
for($i=0; $i<5; $i++){
    $arr1[$i]=<stdin>;
}

print("\n---\n");
@merged=(@arr,@arr1);
print("\nThe merged array is: \n", @merged);

@reversed=reverse(@arr);
print("\nThe reversed array is: \n", @reversed);
print("\nThe sorted array is: \n",sort(@arr1));
@y = sort { $b cmp $a } @arr1;
print("\nSorted in descending order: \n",@y);
print("\n---\n");
```

OUTPUT:


The screenshot shows a terminal window with the following output:

```
*ARRAY OPERATIONS USING MERGE, REVERSE AND SORTING*
Enter elements of first array:
1
2
3
4
5

Enter elements of second array:
a
d
c
b
e

---

The merged array is:
1
2
3
4
5
a
d
c
b
e

The reversed array is:
5
4
3
2
1

The sorted array is:
a
b
c
d
e

The reversed array is:
5
4
3
2
1

The sorted array is:
a
b
c
d
e

Sorted in descending order:
e
d
c
b
a

---
Press any key to continue . . .
```

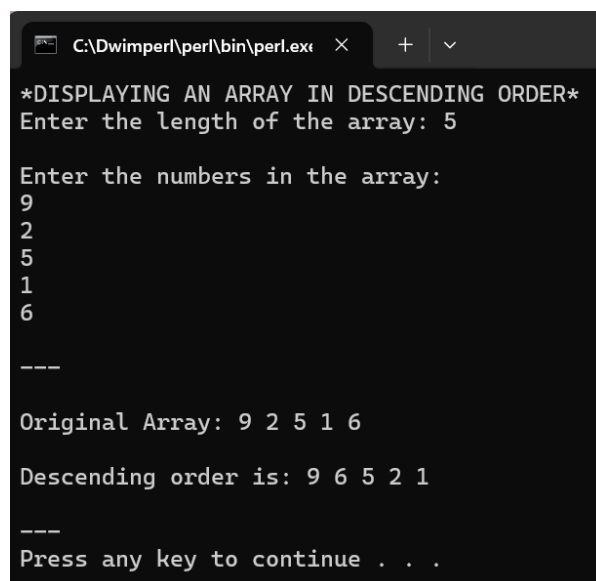
Figure 6: Displaying the output after performing array operations using merge, reverse and sorting

Question 7:

Write a Perl script create an array(numbers) and display in descending order.

CODE:

```
print("*DISPLAYING AN ARRAY IN DESCENDING ORDER*");
print("\nEnter the length of the array: ");
chomp($len=<stdin>);
@arr;
print("\nEnter the numbers in the array: \n");
for($i=0; $i<$len; $i++)
{
    chomp($arr[$i] = <stdin>);
}
print("\n---\n");
print("\nOriginal Array: @arr \n");
@sort = sort{$b <=> $a} @arr;
print("\nDescending order is: @sort \n");
print("\n---\n");
```

OUTPUT:A screenshot of a terminal window showing the execution of a Perl script. The window title is 'C:\Dwimperl\perl\bin\perl.exe'. The output of the script is as follows:

```
*DISPLAYING AN ARRAY IN DESCENDING ORDER*
Enter the length of the array: 5

Enter the numbers in the array:
9
2
5
1
6

---

Original Array: 9 2 5 1 6

Descending order is: 9 6 5 2 1

---

Press any key to continue . . .
```

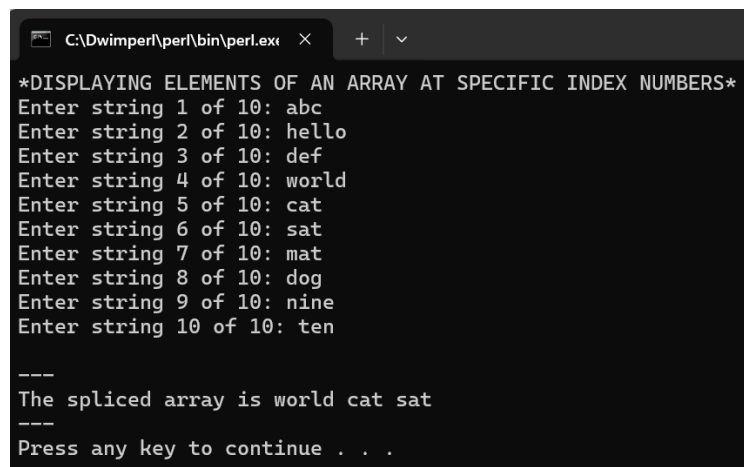
Figure 7: Displaying the user-entered array in descending order

Question 8:

Write a Perl script to stored string of an array and display index number 3,4,5 (ELEMENTS) at once.

CODE:

```
print("*DISPLAYING ELEMENTS OF AN ARRAY AT SPECIFIC INDEX
NUMBERS*\n");
@str1;
for($i = 0; $i <= 9; $i++){
    print("Enter string " . ($i + 1) . " of 10: ");
    chomp($str1[$i] = <STDIN>);
}
@spliced_str1 = (@str1[3..5]);
print("\n---\n");
print("The spliced array is @spliced_str1");
print("\n---\n");
```

OUTPUT:A screenshot of a Windows command prompt window titled "C:\Dwimper\perl\bin\perl.exe". The window displays the output of a Perl script. The first line is "*DISPLAYING ELEMENTS OF AN ARRAY AT SPECIFIC INDEX NUMBERS*". This is followed by ten prompts "Enter string 1 of 10:", "Enter string 2 of 10:", ..., "Enter string 10 of 10:". The user has entered the following strings: "abc", "hello", "def", "world", "cat", "sat", "mat", "dog", "nine", and "ten". After these inputs, there are three dashes "---" and the text "The spliced array is world cat sat". This is followed by another three dashes "---" and the text "Press any key to continue . . .".

```
C:\Dwimper\perl\bin\perl.exe
*DISPLAYING ELEMENTS OF AN ARRAY AT SPECIFIC INDEX NUMBERS*
Enter string 1 of 10: abc
Enter string 2 of 10: hello
Enter string 3 of 10: def
Enter string 4 of 10: world
Enter string 5 of 10: cat
Enter string 6 of 10: sat
Enter string 7 of 10: mat
Enter string 8 of 10: dog
Enter string 9 of 10: nine
Enter string 10 of 10: ten

---
The spliced array is world cat sat
---
Press any key to continue . . .
```

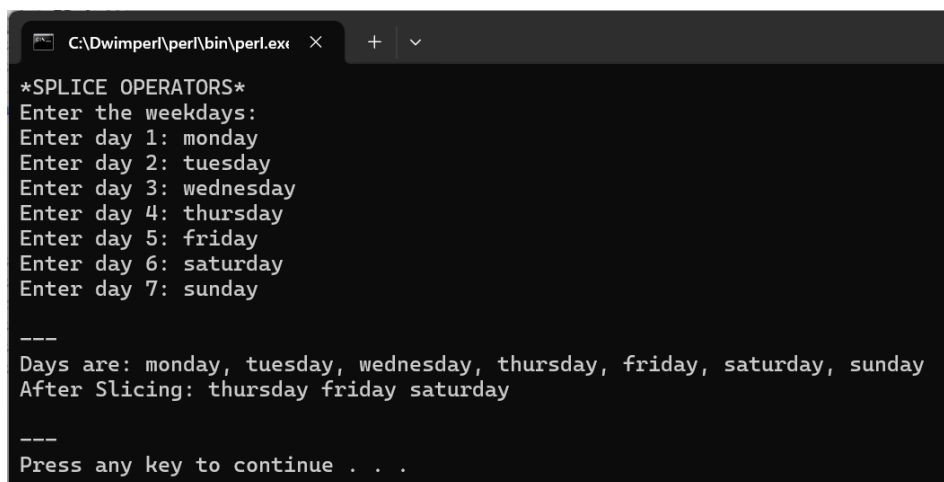
Figure 8: Displaying the elements at specific index numbers of the user-entered array

Question 9:

Write a Perl script to demonstrate splice operator.

CODE:

```
print("*SPICE OPERATORS*");
print("\nEnter the weekdays:\n");
my @input;
for(my $i = 0; $i <= 6; $i++)
{
    print("Enter day ", $i+1, ": ");
    my $day = <STDIN>;
    chomp($day);
    $input[$i] = $day;
}
print("\n---\n");
print("Days are: ", join(", ", @input), "\n");
@weekdays=@input[3,4,5];
print("After Slicing: ");
print("@weekdays\n");
print("\n---\n");
```

OUTPUT:A screenshot of a Windows command prompt window titled "C:\Dwimper\perl\bin\perl.exe". The window shows the output of a Perl script. The script prompts the user to enter weekdays, and the user has entered "monday", "tuesday", "wednesday", "thursday", "friday", "saturday", and "sunday". The script then displays the array of weekdays, followed by the result of slicing the array from index 3 to 5, which are "thursday", "friday", and "saturday". The window also shows a prompt to press any key to continue.

```
C:\Dwimper\perl\bin\perl.exe x + v
*SPLICE OPERATORS*
Enter the weekdays:
Enter day 1: monday
Enter day 2: tuesday
Enter day 3: wednesday
Enter day 4: thursday
Enter day 5: friday
Enter day 6: saturday
Enter day 7: sunday

---
Days are: monday, tuesday, wednesday, thursday, friday, saturday, sunday
After Slicing: thursday friday saturday

---
Press any key to continue . . .
```

Figure 9: Displaying the output after splicing a user-entered array

Question 10:

Write a Perl script to sort hashes using keys.

CODE:

```
print("*SORTING HASH USING
KEYS*\n");
%dna;
@names;
@values;
for($i = 0; $i <= 2; $i++)
{
    print("Enter a key: \n");
    $names[$i] = <stdin>;
    print("Enter values: \n");
    $values[$i] = <stdin>;
    $dna{$names[$i]} = $values[$i];
    print("\n");
}
print("\n---\n");
print("keys:\n");
@keys1 = keys %dna;
for($i = 0;$i <= 2;$i++)
{
    print($keys1[$i]);
}
print("\nvalues:\n");
@values1 = values %dna;
for($i = 0;$i <= 2; $i++)
{
    print($values1[$i]);
}
@hash = sort keys %dna;
print("\nAfter sorting: \n", @hash);
print("\n---\n");
```

OUTPUT:


```
C:\Dwimperl\perl\bin\perl.exe x + v
*SORTING HASH USING KEYS*
Enter a key:
3
Enter values:
hello

Enter a key:
1
Enter values:
world

Enter a key:
2
Enter values:
abc

---
keys:
1
2
3

values:
world
abc
hello

After sorting:
1
2
3

---
Press any key to continue . . .
```

Figure 10: Displaying the output after sorting a user-entered hash using keys

Question 11:

Write a Perl program to determine the frequency of nucleotide bases (i.e. A, T, G, and C) in given nucleotide sequence using nested if else.

CODE:

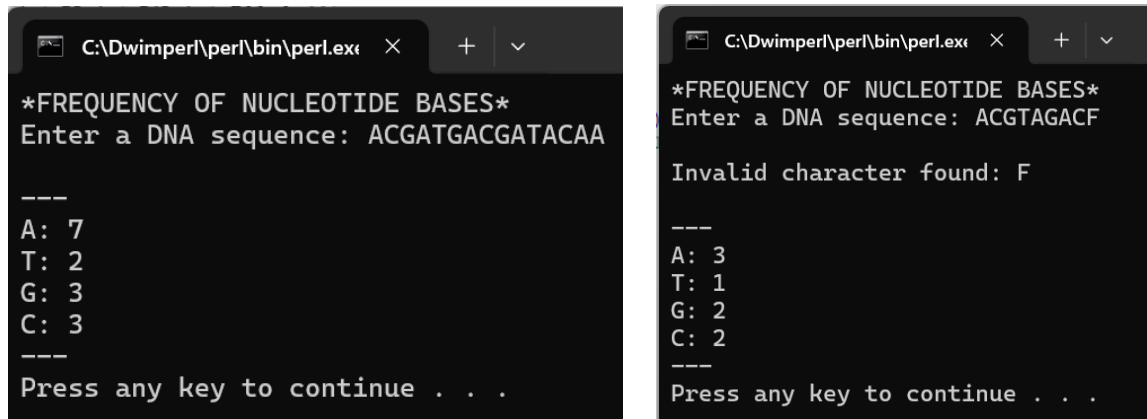
```
print("*FREQUENCY OF NUCLEOTIDE BASES*");
print("\nEnter a DNA sequence: ");
$dna_seq = <STDIN>;
chomp($dna_seq);
$A_count = 0;
$T_count = 0;
$G_count = 0;
$C_count = 0;

for ($i = 0; $i < length($dna_seq); $i++) {
    $char = substr($dna_seq, $i, 1);

    if ($char eq 'A') {
        $A_count++;
    } else {
        if ($char eq 'T') {
            $T_count++;
        } else {
            if ($char eq 'G') {
                $G_count++;
            } else {
                if ($char eq 'C') {
                    $C_count++;
                } else {
                    print("\nInvalid character found: $char\n");
                }
            }
        }
    }
}

print("\n---\n");
print("A: $A_count\n");
print("T: $T_count\n");
print("G: $G_count\n");
print("C: $C_count\n");
print("---\n");
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe x + v
*FREQUENCY OF NUCLEOTIDE BASES*
Enter a DNA sequence: ACGATGACGATACAA
---
A: 7
T: 2
G: 3
C: 3
---
Press any key to continue . . .

C:\Dwimperl\perl\bin\perl.exe x + v
*FREQUENCY OF NUCLEOTIDE BASES*
Enter a DNA sequence: ACGTAGACF
Invalid character found: F
---
A: 3
T: 1
G: 2
C: 2
---
Press any key to continue . . .
```

Figure 11: Displaying the frequency of nucleotide bases for the user-entered DNA sequence

RESULTS:

Simple programs based on the concept of operators used on scalar, array and hash variables used in Perl were coded and executed to display the output after performing operations on user-entered numbers and strings using arithmetic operators and miscellaneous operators, to display the smallest number out of the three user-entered numbers, to check and display the output when two user-entered strings are equal and not equal, to display the output after performing array operations on a user-entered array, to display the output after performing array operations based on an array consisting of DNA sequences, to display the output after performing array operations using merge, reverse and sorting, to display the user-entered array in descending order, to display the elements at specific index numbers of the user-entered array, to display the output after splicing a user-entered array, to display the output after sorting a user-entered hash using keys and to display the frequency of nucleotide bases for the user-entered DNA sequence.

CONCLUSION:

Simple programs based on the concept of operators used on scalar, array and hash variables used in Perl were coded and executed.

REFERENCES:

1. GeeksforGeeks. (2019, June 18). Perl | Hash in scalar and list context. GeeksforGeeks. <https://www.geeksforgeeks.org/perl-hash-in-scalar-and-list-context/>
 2. GeeksforGeeks. (2022, April 18). Perl | Variables and its Types. GeeksforGeeks. <https://www.geeksforgeeks.org/perl-variables-and-its-types/>
 3. Perl Scalars - javatpoint. (n.d.). www.javatpoint.com. <https://www.javatpoint.com/perl-scalars>
-

PRACTICAL 4: PERL **SUBROUTINE**

AIM:

To create simple programs based on the concept of subroutines used in Perl.

INTRODUCTION:

Larry Wall created Perl in 1987 as a high-level, interpreted programming language initially for text manipulation. Over time, it evolved into a versatile language used in web development, system administration, and network programming. Perl's syntax resembles C, making it accessible for C or C++ programmers. It supports both procedural and object-oriented programming, featuring dynamic typing and automatic memory management, enhancing its flexibility and user-friendliness. The acronym 'Practical Extraction and Reporting Language' is commonly associated with Perl.

Subroutines

Subroutines in Perl are fundamental constructs that allow programmers to encapsulate code into reusable blocks. This enhances code modularity, readability, and maintainability. They can be likened to functions in other programming languages, serving the same purpose of performing specific tasks.

Characteristics of Subroutines

1. **Scope:** By default, variables declared within a subroutine are local to that subroutine unless specified otherwise. Using the 'my' keyword allows for local variable declaration.
2. **Return Values:** A subroutine can return a value using the return statement. If no explicit return is provided, Perl automatically returns the last evaluated expression.
3. **Flexibility:** Subroutines can be defined anywhere in the code and can even be anonymous (i.e., not given a name). This allows for dynamic programming techniques such as closures.
4. **Parameter Passing:** Perl passes parameters as a flat list of scalars. This means you can pass any number of arguments without needing to declare them beforehand.
5. **Anonymous Subroutines:** You can create anonymous subroutines that are assigned to variables

Syntax

A subroutine is defined using the 'sub' keyword followed by the subroutine's name and a block of code. The basic syntax is as follows:

```
sub subroutine_name {  
    # Block of code  
}
```


Advantages of Using Subroutines

1. **Code Reusability:** By encapsulating code into subroutines, you avoid redundancy and make your program easier to manage.
2. **Improved Readability:** Breaking complex tasks into smaller subroutines makes it easier to understand and maintain code.
3. **Easier Debugging:** Isolating functionality into subroutines allows for targeted testing and debugging.

Parameters

Subroutines can accept parameters, which are passed as a list. In Perl, parameters are accessed within the subroutine using the special array `@_`. The first parameter can be accessed with `$_`, the second with `$_[1]`, and so on. For example:

```
sub greet {  
    my ($name) = @_; # Accessing the first parameter  
    print "Hello, $name!\n";  
}
```

Calling Subroutines

To invoke a subroutine, simply use its name followed by parentheses containing any arguments:

```
greet("Alice"); # Output: Hello, Alice!
```

Alternatively, you can prepend the name with an ampersand (&), although this is less common in modern Perl:

```
&greet("Bob");
```

PROGRAMS:

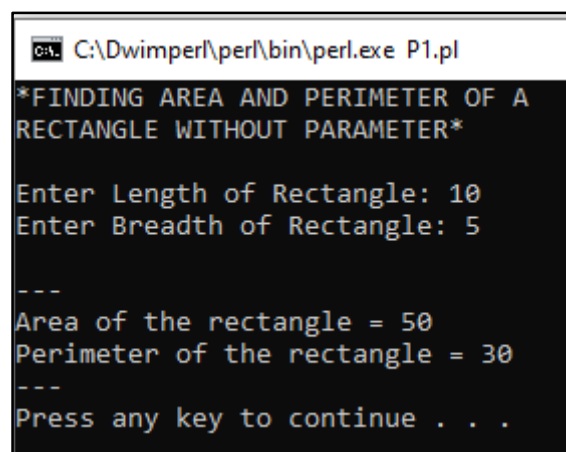
Question 1:

Write a Perl program to create a subroutine named 'Calculate' and find area and perimeter of a rectangle.

CODE:

```
print("*FINDING AREA AND PERIMETER OF A \nRECTANGLE WITHOUT  
PARAMETER*\n");  
sub Calculate {  
    print("\nEnter Length of Rectangle: ");  
    chomp($l = <stdin>);  
    print("Enter Breadth of Rectangle: ");  
    chomp($b = <stdin>);  
    $area = $l * $b;  
    $perimeter = 2 * ($l + $b);  
    print("\n---\n");  
    print("Area of the rectangle = $area\n");  
    print("Perimeter of the rectangle = $perimeter\n");  
    print("---\n");  
}  
  
Calculate();
```

OUTPUT:

A screenshot of a Windows command prompt window. The title bar shows the file path 'C:\Dwimperl\perl\bin\perl.exe P1.pl'. The command prompt displays the output of a Perl script. It starts with a header line: '*FINDING AREA AND PERIMETER OF A RECTANGLE WITHOUT PARAMETER*'. Then it prompts for 'Enter Length of Rectangle: 10' and 'Enter Breadth of Rectangle: 5'. After a separator line '---', it shows the results: 'Area of the rectangle = 50' and 'Perimeter of the rectangle = 30'. Another separator line '---' follows, and the prompt ends with 'Press any key to continue . . .'.

```
C:\Dwimperl\perl\bin\perl.exe P1.pl  
*FINDING AREA AND PERIMETER OF A  
RECTANGLE WITHOUT PARAMETER*  
  
Enter Length of Rectangle: 10  
Enter Breadth of Rectangle: 5  
  
---  
Area of the rectangle = 50  
Perimeter of the rectangle = 30  
---  
Press any key to continue . . .
```

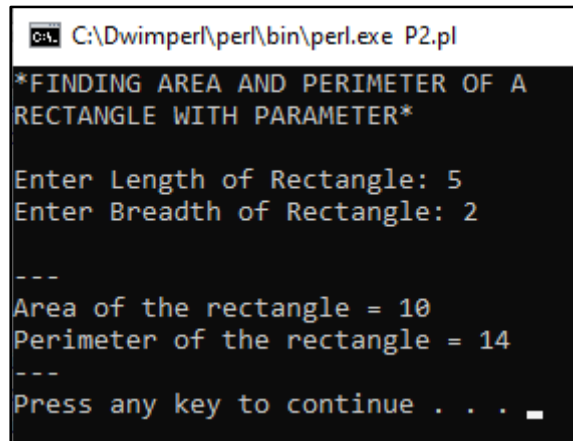
Figure 1: Displaying the area and perimeter of the rectangle without using parameters

Question 2:

Write a Perl program to create a subroutine named 'Calculate' and find area and perimeter a rectangle with parameter.

CODE:

```
print("*FINDING AREA AND PERIMETER OF A \nRECTANGLE WITH  
PARAMETER*\n");  
sub Calculate {  
    ($l, $b) = @_;  
    $area = $l * $b;  
    $perimeter = 2 * ($l + $b);  
    return ($area, $perimeter);  
}  
  
print("\nEnter Length of Rectangle: ");  
chomp($l = <stdin>);  
print("Enter Breadth of Rectangle: ");  
chomp($b = <stdin>);  
$area, $perimeter = Calculate($l, $b);  
print("\n---\n");  
print("Area of the rectangle = $area\n");  
print("Perimeter of the rectangle = $perimeter\n");  
print("---\n");
```

OUTPUT:

```
C:\Dwimperl\perl\bin\perl.exe P2.pl  
*FINDING AREA AND PERIMETER OF A  
RECTANGLE WITH PARAMETER*  
  
Enter Length of Rectangle: 5  
Enter Breadth of Rectangle: 2  
  
---  
Area of the rectangle = 10  
Perimeter of the rectangle = 14  
---  
Press any key to continue . . .
```

Figure 2: Displaying the area and perimeter of the rectangle using parameters

RESULTS:

Simple programs based on the concept of subroutines used in Perl were coded and executed to display the area and perimeter of the rectangle with and without using parameters.

CONCLUSION:

Simple programs based on the concept of subroutines used in Perl were coded and executed.

REFERENCES:

1. CS10100 Perl Subroutines & Functions. (n.d.). https://www.classes.cs.uchicago.edu/archive/2009/winter/10100-1/html/perl/perl_functions.html
 2. GeeksforGeeks. (2018, July 13). Perl | Subroutines or functions. GeeksforGeeks. <https://www.geeksforgeeks.org/perl-subroutines-or-functions/>
 3. Perl Functions and Subroutines - javatpoint. (n.d.). www.javatpoint.com. <https://www.javatpoint.com/perl-functions-and-subroutines>
 4. Perl subroutine. (2023, February 9). Perl Tutorial. <https://www.perltutorial.org/perl-subroutine/>
-

PRACTICAL 5: PERL

REFERENCES AND DEREFERENCES, AND SCOPE OF VARIABLES

AIM:

To create simple programs based on the concept of references, dereferences and scope of variables used in Perl.

INTRODUCTION:

Larry Wall created Perl in 1987 as a high-level, interpreted programming language initially for text manipulation. Over time, it evolved into a versatile language used in web development, system administration, and network programming. Perl's syntax resembles C, making it accessible for C or C++ programmers. It supports both procedural and object-oriented programming, featuring dynamic typing and automatic memory management, enhancing its flexibility and user-friendliness. The acronym 'Practical Extraction and Reporting Language' is commonly associated with Perl.

Referencing

In Perl, referencing is the process of creating a reference, which is a scalar variable that holds the memory address of another variable (such as a scalar, array, or hash). This allows you to manipulate complex data structures without copying them, enhancing efficiency. References can be created using the backslash operator (\).

Dereferencing

Dereferencing is the process of accessing the value stored at the memory address referenced by a reference variable. The type of dereferencing depends on the type of reference being accessed.

Syntax:

Variable Type	Referencing	Dereferencing	Output
Scalar Variable	<code>my \$scalar = 10;</code> <code>my \$scalar_ref = \ \$scalar;</code>	<code>print \$\$scalar_ref;</code>	10
Array Variable	<code>my @array = (1, 2, 3);</code> <code>my \$array_ref = \ @array;</code>	<code>print "@\$array_ref";</code>	1 2 3
Hash Variable	<code>my %hash = ('key1' => 'value1', 'key2'</code> <code>=> 'value2');</code> <code>my \$hash_ref = \ %hash;</code>	<code>print</code> <code>"\$\$hash_ref{'key1'}";</code>	value1

Scope of Variables

The scope of a variable in Perl defines where that variable can be accessed within the code. Variables can have different scopes based on how they are declared:

1. **Global Variables:** Declared outside any subroutine or block and can be accessed from anywhere in the program.
2. **Lexical Variables:** Declared with 'my' keyword, these variables are scoped to the block or file they are declared in. They cannot be accessed outside their defined scope.

Example Code:

```
sub function1 {  
    my $lexical_var = "I am local";  
}  
  
sub function2 {  
    print $lexical_var; # Error: Can't access lexical variable outside its scope  
}  
  
$global_var = "I am global"; // Global variable  
  
sub function3 {  
    print $global_var; # Works fine  
}
```

PROGRAMS:

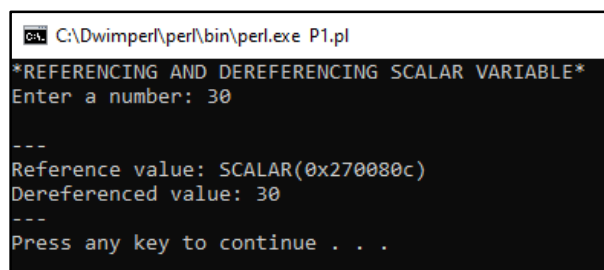
Question 1:

Write a Perl script to accept a number and create reference of scalar variable and display a value using dereferencing.

CODE:

```
print("*REFERENCING AND DEREFERENCING SCALAR VARIABLE*");
print("\nEnter a number: ");
$num = <stdin>;
$ref = \$num;
print("\n---\nReference value: $ref\n");
print("Dereferenced value: $$ref---\n");
```

OUTPUT:



```
C:\Dwimper\perl\bin\perl.exe P1.pl
*REFERENCING AND DEREFERENCING SCALAR VARIABLE*
Enter a number: 30
---
Reference value: SCALAR(0x270080c)
Dereferenced value: 30
---
Press any key to continue . . .
```

Figure 1: Displaying the output for referencing and dereferencing a scalar variable

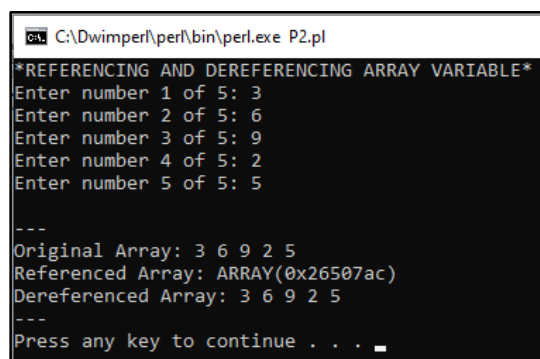
Question 2:

Write a Perl script to store an array and use reference and dereferencing.

CODE:

```
print("*REFERENCING AND DEREFERENCING ARRAY VARIABLE*\n");
@arr1;
for($i = 0; $i < 5; $i++){
    print("Enter number " . ($i+1) . " of 5: ");
    chomp($arr1[$i] = <stdin>);
}
$ref_arr1 = \@arr1;
print("\n---\nOriginal Array: @arr1\n");
print("Referenced Array: $ref_arr1\n");
print("Dereferenced Array: @$ref_arr1\n---\n");
```

OUTPUT:



```
C:\Dwimper\perl\bin\perl.exe P2.pl
*REFERENCING AND DEREFERENCING ARRAY VARIABLE*
Enter number 1 of 5: 3
Enter number 2 of 5: 6
Enter number 3 of 5: 9
Enter number 4 of 5: 2
Enter number 5 of 5: 5
---
Original Array: 3 6 9 2 5
Referenced Array: ARRAY(0x26507ac)
Dereferenced Array: 3 6 9 2 5
---
Press any key to continue . . .
```

Figure 2: Displaying the output for referencing and dereferencing an array variable

Question 3:

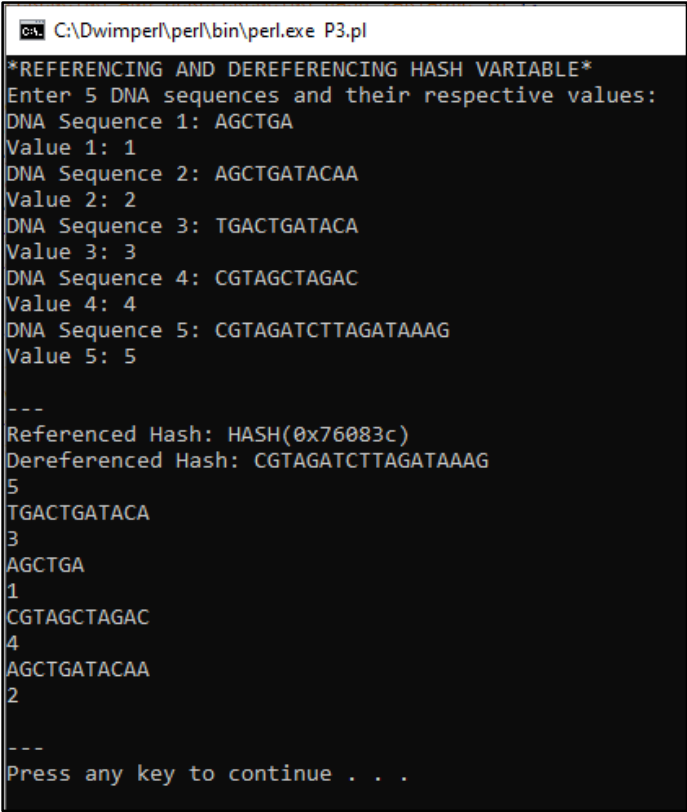
Write a Perl script to store a hash and use reference and dereferencing.

CODE:

```
print("*REFERENCING AND DEREFERENCING HASH VARIABLE*\n");
@seq;
@value;
%hash1;
print("Enter 5 DNA sequences and their respective values: \n");
for($i = 0; $i < 5; $i++){
    print("DNA Sequence " .($i+1). ": ");
    $seq[$i] = <stdin>;
    print("Value " .($i+1). ": ");
    $value[$i] = <stdin>;
    $hash1{$seq[$i]} = $value[$i];
}

$hash1_ref = \%hash1;
print("\n---\nReferenced Hash: $hash1_ref\n");
print("Dereferenced Hash: ");
print(%{$hash1_ref});
print("\n---\n");
```

OUTPUT:



```
C:\Dwimper\perl\bin\perl.exe P3.pl
*REFERENCING AND DEREFERENCING HASH VARIABLE*
Enter 5 DNA sequences and their respective values:
DNA Sequence 1: AGCTGA
Value 1: 1
DNA Sequence 2: AGCTGATACAA
Value 2: 2
DNA Sequence 3: TGACTGATACA
Value 3: 3
DNA Sequence 4: CGTAGCTAGAC
Value 4: 4
DNA Sequence 5: CGTAGATCTTAGATAAAG
Value 5: 5

---
Referenced Hash: HASH(0x76083c)
Dereferenced Hash: 
5
TGACTGATACA
3
AGCTGA
1
CGTAGCTAGAC
4
AGCTGATACAA
2

---
Press any key to continue . . .
```

Figure 3: Displaying the output for referencing and dereferencing a hash variable

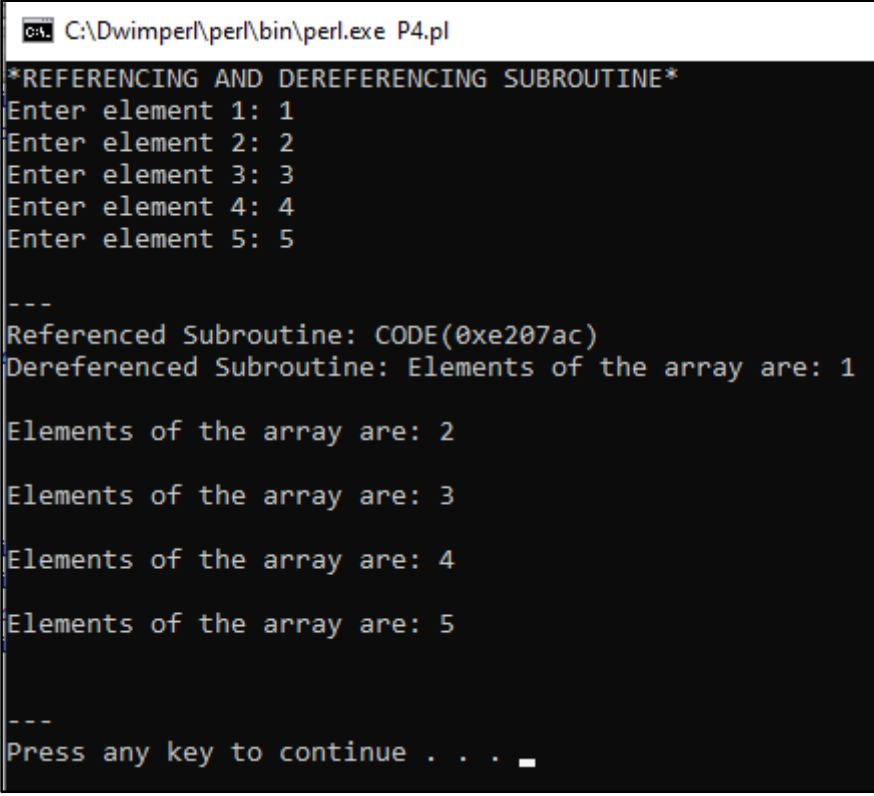
Question 4:

Write a Perl script to create a subroutine and use reference and dereferencing.

CODE:

```
print("*REFERENCING AND DEREFERENCING SUBROUTINE*\n");
sub demo{
    my @number = @_ ;
    foreach my $elements(@number){
        print("Elements of the array are: $elements\n");
    }
}
my @number;
for(my $i = 0; $i < 5; $i++){
    print("Enter element " . ($i+1) . ": ");
    $number[$i] = <stdin>;
}

my $ref = \&demo;
print("\n---\nReferenced Subroutine: $ref\n");
print("Dereferenced Subroutine: ");
&$ref(@number);
print("\n---\n");
```

OUTPUT:

```
C:\Dwimperl\perl\bin\perl.exe P4.pl
*REFERENCING AND DEREFERENCING SUBROUTINE*
Enter element 1: 1
Enter element 2: 2
Enter element 3: 3
Enter element 4: 4
Enter element 5: 5

---
Referenced Subroutine: CODE(0xe207ac)
Dereferenced Subroutine: Elements of the array are: 1

Elements of the array are: 2

Elements of the array are: 3

Elements of the array are: 4

Elements of the array are: 5

---
Press any key to continue . . .
```

Figure 4: Displaying the output for referencing and dereferencing a subroutine

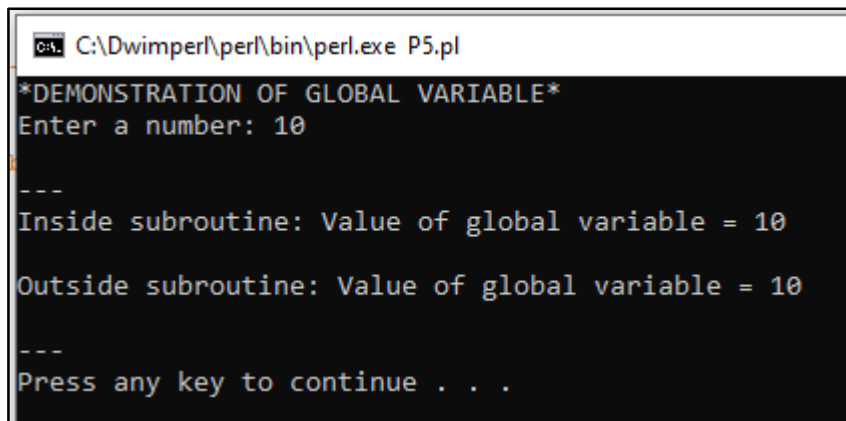
Question 5:

Write a Perl script to store a number in global variable and demonstrate the scope of it.

CODE:

```
print("*DEMONSTRATION OF GLOBAL VARIABLE*");
print("\nEnter a number: ");
$global_var = <stdin>;

sub demo{
    print("\n---\nInside subroutine: Value of global variable = $global_var\n");
}
demo();
print("Outside subroutine: Value of global variable = $global_var\n---\n");
```

OUTPUT:A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Dwimper\perl\bin\perl.exe P5.pl'. The command prompt displays the output of a Perl script. It starts with a star separator, followed by a prompt 'Enter a number: 10'. Then, after three dashes, it shows 'Inside subroutine: Value of global variable = 10'. This is followed by 'Outside subroutine: Value of global variable = 10'. After another three dashes, it prompts 'Press any key to continue . . .'.

```
C:\Dwimper\perl\bin\perl.exe P5.pl
*DEMONSTRATION OF GLOBAL VARIABLE*
Enter a number: 10

---
Inside subroutine: Value of global variable = 10

Outside subroutine: Value of global variable = 10

---
Press any key to continue . . .
```

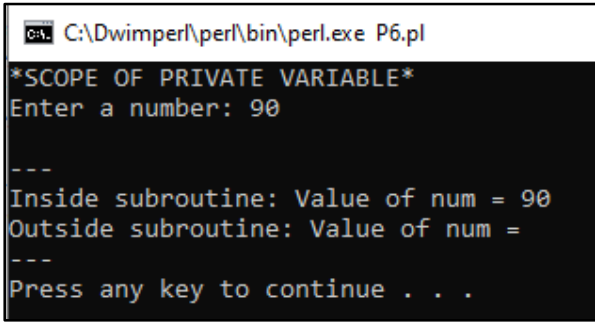
Figure 5: Output for the demonstration of scope of global variable

Question 6:

Write a Perl script to store a number in private variable and demonstrate the scope of it.

CODE:

```
print("*SCOPE OF PRIVATE VARIABLE*");
sub demonstration{
    print("\nEnter a number: ");
    my $num = <stdin>;
    print("\n---\nInside subroutine: Value of num = $num");
}
demonstration();
print("\nOutside subroutine: Value of num = $num\n---\n");
```

OUTPUT:

```
C:\Dwimper\perl\bin\perl.exe P6.pl
*SCOPE OF PRIVATE VARIABLE*
Enter a number: 90
---
Inside subroutine: Value of num = 90
Outside subroutine: Value of num =
---
Press any key to continue . . .
```

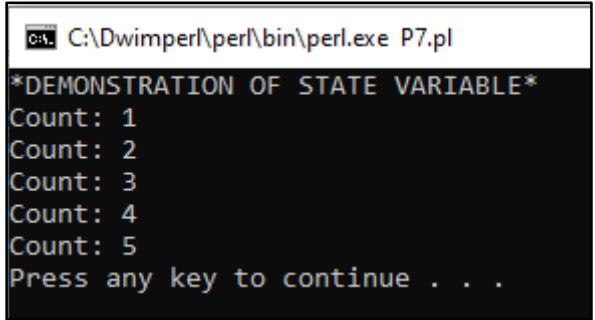
Figure 6: Output for the demonstration of scope of private variable

Question 7:

Write a Perl script to store a number in state variable and display an output.

CODE:

```
print("*DEMONSTRATION OF STATE VARIABLE*\n");
use 5.010;
sub demo{
    state $number = 0;
    $number++;
    print("Count: $number\n");
}
for(1..5){
    demo();
}
```

OUTPUT:

```
C:\Dwimper\perl\bin\perl.exe P7.pl
*DEMONSTRATION OF STATE VARIABLE*
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
Press any key to continue . . .
```

Figure 7: Output for the demonstration of scope of state variable

RESULTS:

Simple programs based on the concept of references, dereferences and scope of variables used in Perl were coded and executed to display the output for referencing and dereferencing a scalar variable, an array variable, a hash variable, and a subroutine, to display the output for the demonstration of scope of global variable, private variable and state variable.

CONCLUSION:

Simple programs based on the concept of references, dereferences and scope of variables used in Perl were coded and executed.

REFERENCES:

1. Perl - references. (n.d.). https://www.tutorialspoint.com/perl/perl_references.htm
 2. Perl references. (2023, February 9). Perl Tutorial. <https://www.perltutorial.org/perl-reference/>
 3. Perl references: create, dereference and debug with confidence. (n.d.). Perl.com. <https://www.perl.com/article/80/2014/3/27/Perl-references-create-dereference-and-debug-with-confidence/>
-

PRACTICAL 6: PERL **REGULAR EXPRESSIONS**

AIM:

To create simple programs based on the concept of regular expressions used in Perl.

INTRODUCTION:

Larry Wall created Perl in 1987 as a high-level, interpreted programming language initially for text manipulation. Over time, it evolved into a versatile language used in web development, system administration, and network programming. Perl's syntax resembles C, making it accessible for C or C++ programmers. It supports both procedural and object-oriented programming, featuring dynamic typing and automatic memory management, enhancing its flexibility and user-friendliness. The acronym 'Practical Extraction and Reporting Language' is commonly associated with Perl.

Regular Expressions

Regular expressions (regex) in Perl are powerful tools for searching and manipulating text. They define patterns that can match strings, allowing for complex text processing with relatively simple syntax. Perl's regex capabilities are among the most advanced in programming languages, making it a popular choice for tasks involving text manipulation.

Syntax:

Operator	Description	Syntax	Example
Match Operator	Used to check if a string matches a specified pattern.	m/PATTERN/	"Hello World" =~ m/World/;
Substitution Operator	Allows you to replace occurrences of a pattern with a new string.	s/PATTERN/REPLACEMENT/	\$string =~ s/cat/dog/;
Translation Operator	Replaces characters in a string based on specified mappings.	tr/SEARCHLIST/REPLACEMENTLIST/	\$string =~ tr/a/b/;

Modifiers

Types of Modifiers	Modifier	Description
Match Operator Modifiers	i	Makes the match case insensitive
	m	Specifies that if the string has newline or carriage return characters, the ^ and \$ operators will now match against a newline boundary, instead of a string boundary
	o	Evaluates the expression only once
	x	Allows you to use white space in the expression for clarity
	g	Globally finds all matches
	cg	Allows the search to continue even after a global match fails
Translation Operator Modifiers	c	Complements SEARCHLIST
	s	Squashes duplicate replaced characters

Character sets

Alternative			"cat mat" == "cat" or "mat" "python jython" == "python" or "jython"
Grouping	()	more than 1 pattern embedded in a single line	"gr (e a) y" == "grey" or "gray" "ra (mil n (ny el))" == "ramil" or "ranny" or "rael"
Quantification	?	0 or 1 of the preceding element	"rani?el" == "rael" or "rael"
	*	0 or more of the preceding element	"fo*ot" == "foot" or "fooooot" or "foooooot"
	+	1 or more of the preceding element	"too+fan" == "toofan" or (REMAINING)
	{m, n}	m to n times of the preceding element	"go{2, 3}gle" == "google" or "goooogle" "6{3}" == "666" "s{2,}" == "ss" or "sss" or "ssss"...
Anchors	^ (start / begin anchors)	Matches the starting position within the string	"^obje" == "object" or "object – oriented" "^2014" == "2014" or "2014/20/07"

	\$ (end operators)	Matches the ending position within the string	"gram\$" == "program" or "kilogram" "2014\$" == "20/07/2014" or "2013 – 2014"
Metacharacters	.	Matches any single character	"bat." == "bat" or "bats" or "bata" "87.1" == "8741" or "8751" or "8761"
	[]	Matches a single character that is contained within the brackets	"[xyz]" == "x" or "y" or "z" "[aeiou]" == any vowel "[0123456789]" == any digit
	[-]	Matches a single character that is contained within the brackets and the specified range	"[a – c]" == "a" or "b" or "c" "[a – z A – Z]" == all letters (both lowercase and uppercase) "[0 – 9]" == all digits
	[^]	Matches a single character that is not contained within the brackets	"[^aeiou]" == any non – vowel "[^0 – 9]" == any non – digit "[^xyz]" == any character but not "x" or "y" or "z"
Character class	\d	Matches a decimal digit	
	\D	Matches non – digits	
	\s	Matches any white space character	
	\S	Matches any non – white space character	
	\w	Matches alphanumeric character class	
	\W	Matches non – alphanumeric character class	
	\w+	Matches 1 or more words / characters	
	\A	Matches the beginning of the string	
	\Z	Matches the end of the string	
	\b	Returns a match where the specified characters are at the beginning or at the end of a word	
	\B	Returns a match where the specified characters are present, but not at the beginning or at the end of a word	

PROGRAMS:

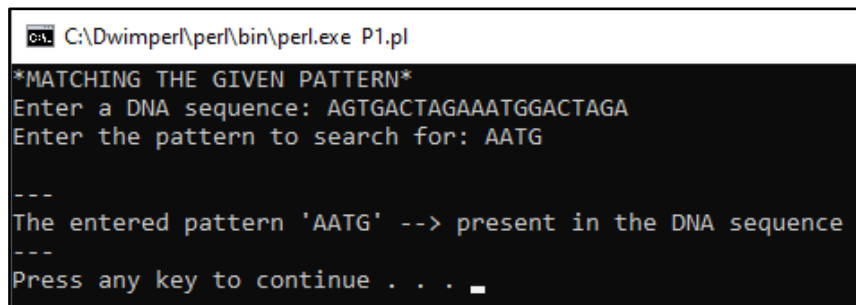
Question 1:

Write a Perl to accept a DNA sequence and match against pattern “aatg” is found on entered sequence and check whether the match case-insensitive.

CODE:

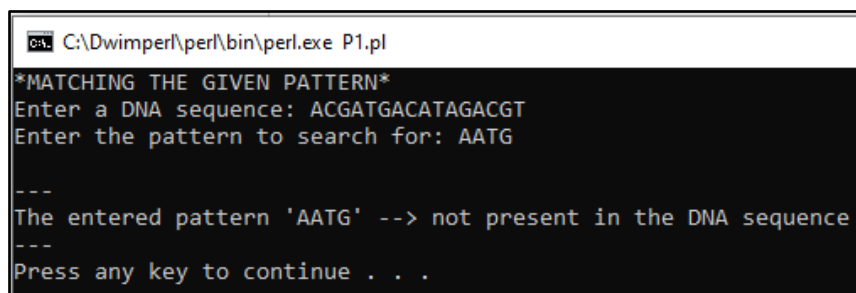
```
print("*MATCHING THE GIVEN PATTERN*\n");
print("Enter a DNA sequence: ");
chomp(my $seq = <stdin>);
print("Enter the pattern to search for: ");
chomp(my $pattern = <stdin>);
print("\n---\n");
if($seq =~ m/$pattern/i){
    print("The entered pattern '$pattern' --> present in the DNA sequence");
}
else{
    print("The entered pattern '$pattern' --> not present in the DNA sequence");
}
print("\n---\n");
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe P1.pl
*MATCHING THE GIVEN PATTERN*
Enter a DNA sequence: AGTGACTAGAAATGGACTAGA
Enter the pattern to search for: AATG
---
The entered pattern 'AATG' --> present in the DNA sequence
---
Press any key to continue . . .
```

Figure 1: Displaying the output when the user-entered pattern is present in the user-entered DNA sequence



```
C:\Dwimperl\perl\bin\perl.exe P1.pl
*MATCHING THE GIVEN PATTERN*
Enter a DNA sequence: ACGATGACATAGACGT
Enter the pattern to search for: AATG
---
The entered pattern 'AATG' --> not present in the DNA sequence
---
Press any key to continue . . .
```

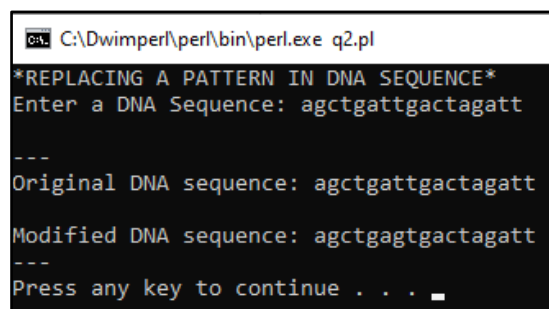
Figure 2: Displaying the output when the user-entered pattern is not present in the user-entered DNA sequence

Question 2:

Write a Perl script to accept DNA sequence from user and search pattern “att” and replace with “agt”.

CODE:

```
print("*REPLACING A PATTERN IN DNA SEQUENCE*");
print("\nEnter a DNA Sequence: ");
chomp($dna_seq = <stdin>);
print("\n---\n");
print("Original DNA sequence: $dna_seq \n");
$dna_seq =~ s/att/agt/i;
print("\nModified DNA sequence: $dna_seq");
print("\n---\n");
```

OUTPUT:

```
C:\Dwimper\perl\bin\perl.exe q2.pl
*REPLACING A PATTERN IN DNA SEQUENCE*
Enter a DNA Sequence: agctgattgactagatt
---
Original DNA sequence: agctgattgactagatt
Modified DNA sequence: agctgagtgactagatt
---
Press any key to continue . . .
```

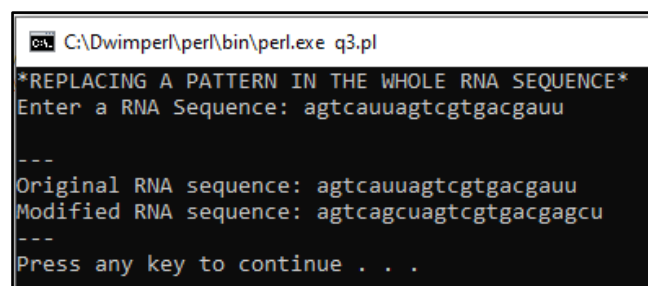
Figure 3: Displaying the modified DNA sequence after replacing the specific pattern

Question 3:

Write a Perl script to accept RNA sequence from user and search pattern “auu” and replace with “agcu” in whole sequence.

CODE:

```
print("*REPLACING A PATTERN IN THE WHOLE RNA SEQUENCE*");
print("\nEnter a RNA Sequence: ");
chomp($rna_seq = <stdin>);
print("\n---");
print("\nOriginal RNA sequence: $rna_seq \n");
$rna_seq =~ s/auu/agcu/g;
print("\nModified RNA sequence: $rna_seq \n---\n");
```

OUTPUT:

```
C:\Dwimper\perl\bin\perl.exe q3.pl
*REPLACING A PATTERN IN THE WHOLE RNA SEQUENCE*
Enter a RNA Sequence: agtcauuagtcgtgacgauu
---
Original RNA sequence: agtcauuagtcgtgacgauu
Modified RNA sequence: agtcagcuagtcgtgacgagcu
---
Press any key to continue . . .
```

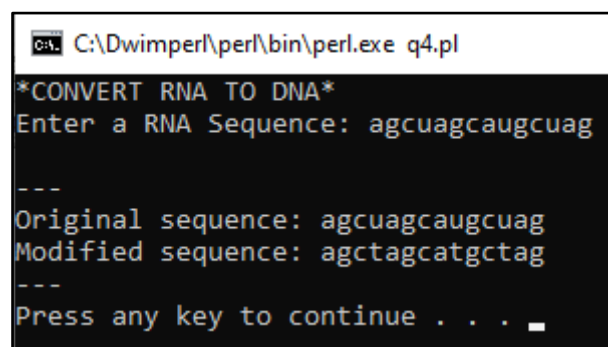
Figure 4: Displaying the modified RNA sequence after replacing the specific pattern in the entire sequence

Question 4:

Write a Perl script to accept RNA sequence and convert that into DNA sequence.

CODE:

```
print("*CONVERT RNA TO DNA*");
print("\nEnter a RNA Sequence: ");
chomp($seq = <stdin>);
print("\n---");
print("\nOriginal sequence: $seq \n");
$seq =~ s/u/t/g;
$seq =~ s/U/T/g;
print("Modified sequence: $seq \n---\n");
```

OUTPUT:

```
C:\Dwimperl\perl\bin\perl.exe q4.pl
*CONVERT RNA TO DNA*
Enter a RNA Sequence: agcuagcaugcuag
---
Original sequence: agcuagcaugcuag
Modified sequence: agctagcatgctag
---
Press any key to continue . . .
```

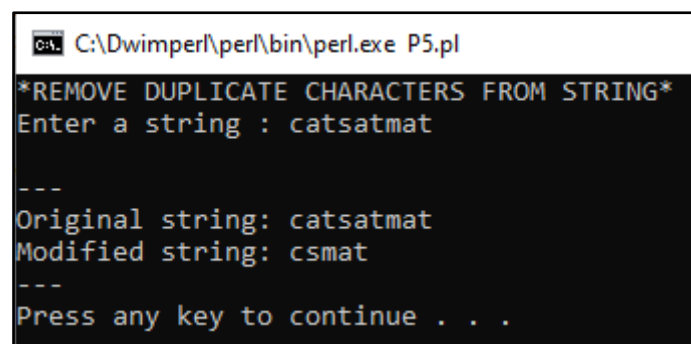
Figure 5: Displaying the modified sequence after converting it from RNA to DNA

Question 5:

Write a Perl script to accept a string and remove duplicate characters from entered string.

CODE:

```
print("*REMOVE DUPLICATE CHARACTERS FROM STRING*");
print("\nEnter a string : ");
chomp($str = <stdin>);
print("\n---\nOriginal string: $str");
$str =~ s/(.)(?=.*\1)//g;
print("\nModified string: $str\n");
print("---\n");
```

OUTPUT:

```
C:\Dwimperl\perl\bin\perl.exe P5.pl
*REMOVE DUPLICATE CHARACTERS FROM STRING*
Enter a string : catsatmat
---
Original string: catsatmat
Modified string: csmat
---
Press any key to continue . . .
```

Figure 6: Displaying the modified string after removing duplicate characters from user-entered string

RESULTS:

Simple programs based on the concept of regular expressions used in Perl were coded and executed to display the output when the user-entered pattern is present or not present in the user-entered DNA sequence, to display the modified DNA sequence after replacing the specific pattern, to display the modified RNA sequence after replacing the specific pattern in the entire sequence, to display the modified sequence after converting it from RNA to DNA and to display the modified string after removing duplicate characters from user-entered string.

CONCLUSION:

Simple programs based on the concept of regular expressions used in Perl were coded and executed.

REFERENCES:

1. Perl - regular expressions. (n.d.).
https://www.tutorialspoint.com/perl/perl_regular_expressions.htm
 2. Perl Regular expressions - javatpoint. (n.d.). www.javatpoint.com.
<https://www.javatpoint.com/perl-regular-expression>
 3. Perl tutorial: Regular Expressions (REgEX), File IO and Text Processing. (n.d.).
https://www3.ntu.edu.sg/home/ehchua/programming/webprogramming/Perl2_Regexe.html
-

PRACTICAL 7: PERL **METACHARACTERS, QUANTIFIERS AND SUBSTRING** **AND FILE HANDLING**

AIM:

To create simple programs based on the concept of Metacharacters, Quantifiers and Substring and File handling used in Perl.

INTRODUCTION:

Larry Wall created Perl in 1987 as a high-level, interpreted programming language initially for text manipulation. Over time, it evolved into a versatile language used in web development, system administration, and network programming. Perl's syntax resembles C, making it accessible for C or C++ programmers. It supports both procedural and object-oriented programming, featuring dynamic typing and automatic memory management, enhancing its flexibility and user-friendliness. The acronym 'Practical Extraction and Reporting Language' is commonly associated with Perl.

Metacharacters

Metacharacters in Perl are special characters used in regular expressions to match specific patterns. They are crucial for defining the structure of a pattern. Here are some of the most common metacharacters and their uses:

Metacharacter	Description	Example
<code>.(dot)</code>	Matches any single character except a newline.	'a.b' would match "aXb" where X can be any character except a newline.
<code>^</code>	Matches the beginning of a string or line.	<code>^hello</code> would match "hello" only if it is at the start of the string.
<code>\$</code>	Matches the end of a string or line.	<code>world\$</code> would match "world" only if it is at the end of the string.
<code> </code>	Used for alternation (OR).	<code>cat dog</code> would match either "cat" or "dog"
<code>()</code>	Used for grouping.	<code>(cat dog)</code> would group "cat" and "dog" together, allowing you to apply quantifiers or other operations to the group as a whole.
<code>[]</code>	Used to define a character class.	<code>[a-zA-Z]</code> would match any letter (both uppercase and lowercase).
<code>*,+, and ?</code>	Quantifiers that specify how many times the preceding element should be matched.	
<code>{ }</code>	Used for specifying the number of occurrences.	<code>a{3}</code> would match "aaa".

<code>\</code> (backslash)	Used to escape metacharacters or to form special sequences like <code>\n</code> , <code>\t</code> , etc.	<code>\.</code> matches a literal dot
----------------------------	--	---------------------------------------

Quantifiers

Quantifiers are used to specify how many times a pattern should be matched. They are essential for defining the frequency of a pattern in a string. Here are some common quantifiers:

1. *****: Matches the preceding element zero or more times. For example, `a*` would match `""`, `"a"`, `"aa"`, etc.
2. **+**: Matches the preceding element one or more times. For example, `a+` would match `"a"`, `"aa"`, etc., but not an empty string.
3. **?**: Matches the preceding element zero or one time. For example, `a?` would match `""` or `"a"`.
4. **{n}**: Matches the preceding element exactly n times. For example, `a{3}` would match `"aaa"`.
5. **{n,}**: Matches the preceding element at least n times. For example, `a{3,}` would match `"aaa"`, `"aaaa"`, etc.
6. **{n,m}**: Matches the preceding element at least n times but no more than m times. For example, `a{3,5}` would match `"aaa"`, `"aaaa"`, or `"aaaaa"`.

Substring Handling

Substring handling in Perl involves extracting or manipulating parts of a string. Here are some common ways to handle substrings:

Matching and Capturing

Use parentheses `()` to capture parts of a match. These captured groups can be accessed using `$1`, `$2`, etc.

```
# Extract the year, month, and day from a date string
if ($string =~ /(\d{4})-(\d{2})-(\d{2})/) {
    print "Year: $1, Month: $2, Day: $3\n";
}
```

Substitution

Use the `s///` operator to replace substrings.

```
# Replace all occurrences of "old" with "new"
$string =~ s/old/new/g;
print $string;
```

Extraction

Use regular expressions or the `substr` function to extract substrings.

```
# Extract the first 10 characters of a string
my $substring = substr($string, 0, 10);
print $substring;
```

Example of Substring Handling

```
# Extract and print the first word from each line in a string
foreach my $line (split /\n/, $string) {
    if ($line =~ /\^(w+)/) {
        print "$1\n";
    }
}
```

File Handling

File handling in Perl involves reading from or writing to files. Here are some basic operations:

Opening a File

Use the open function to open a file for reading or writing. It's important to check if the file was opened successfully.

```
# Open a file for reading
open(my $fh, '<', 'input.txt') or die "Could not open file 'input.txt' $!";
```

Reading from a File

Use the <> operator or readline function to read lines from a file.

```
# Read and print each line from the file
while (my $line = <$fh>) {
    print $line;
}
```

Writing to a File

Use the print function with a filehandle to write to a file.

```
# Open a file for writing and write a line
open(my $fh, '>', 'output.txt') or die "Could not open file 'output.txt' $!";
print $fh "Hello, World!\n";
```

Closing a File

Use the close function to close a filehandle. However, in modern Perl, it's often not necessary to explicitly close files as they are automatically closed when they go out of scope.

```
# Close the filehandle
close($fh);
```

Example of File Handling

```
# Read a file line by line, convert to uppercase, and write to another file
open(my $in_fh, '<', 'input.txt') or die "Could not open file 'input.txt' $!";
open(my $out_fh, '>', 'output.txt') or die "Could not open file 'output.txt' $!";

while (my $line = <$in_fh>) {
    print $out_fh uc($line);
}
close($in_fh);
close($out_fh);
```

PROGRAMS:

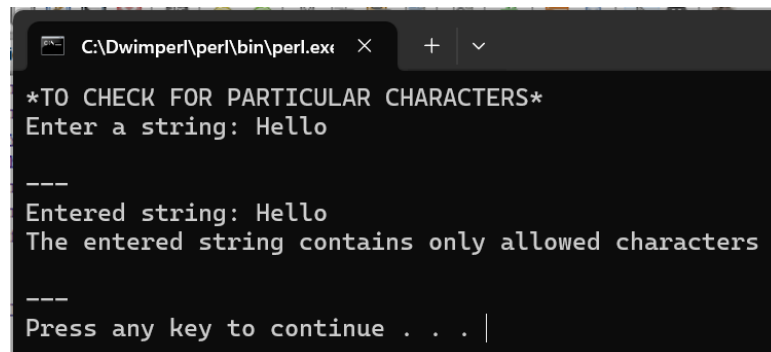
Question 1:

Write a Perl program to check that a string contains only a certain set of characters (in this case a-z, A-Z and 0-9).

CODE:

```
print("*TO CHECK FOR PARTICULAR CHARACTERS*\n");
print("Enter a string: ");
my $string1 = <STDIN>;
chomp($string1);
print("\n---\n");
print("Entered string: $string1\n");
if ($string1 =~ /^[a-zA-Z0-9]+$/) {
    print("The entered string contains only allowed characters\n");
}
else {
    print("The string does not contain allowed characters\n");
}
print("\n---\n");
```

OUTPUT:

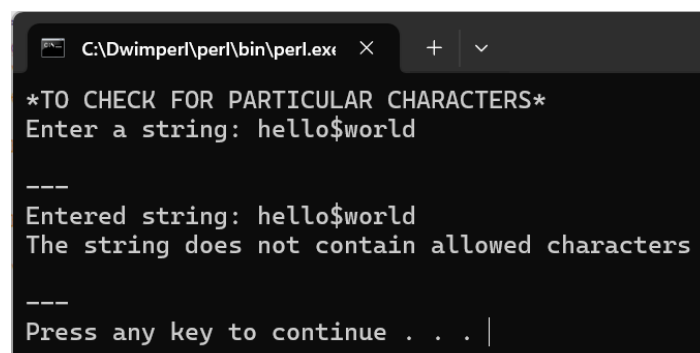
A screenshot of a Windows command prompt window titled 'C:\Dwimperl\perl\bin\perl.exe'. The output of the Perl program is displayed in a monospaced font. It starts with the prompt '*TO CHECK FOR PARTICULAR CHARACTERS*', followed by 'Enter a string: Hello'. After a separator '---', it shows 'Entered string: Hello' and 'The entered string contains only allowed characters'. Another separator '---' follows, and the prompt 'Press any key to continue . . . |' is shown at the bottom.

```
C:\Dwimperl\perl\bin\perl.exe  X  +  v
*TO CHECK FOR PARTICULAR CHARACTERS*
Enter a string: Hello

---
Entered string: Hello
The entered string contains only allowed characters

---
Press any key to continue . . . |
```

Figure 1: Displaying the output when the user-entered string contains allowed characters

A screenshot of a Windows command prompt window titled 'C:\Dwimperl\perl\bin\perl.exe'. The output of the Perl program is displayed in a monospaced font. It starts with the prompt '*TO CHECK FOR PARTICULAR CHARACTERS*', followed by 'Enter a string: hello\$world'. After a separator '---', it shows 'Entered string: hello\$world' and 'The string does not contain allowed characters'. Another separator '---' follows, and the prompt 'Press any key to continue . . . |' is shown at the bottom.

```
C:\Dwimperl\perl\bin\perl.exe  X  +  v
*TO CHECK FOR PARTICULAR CHARACTERS*
Enter a string: hello$world

---
Entered string: hello$world
The string does not contain allowed characters

---
Press any key to continue . . . |
```

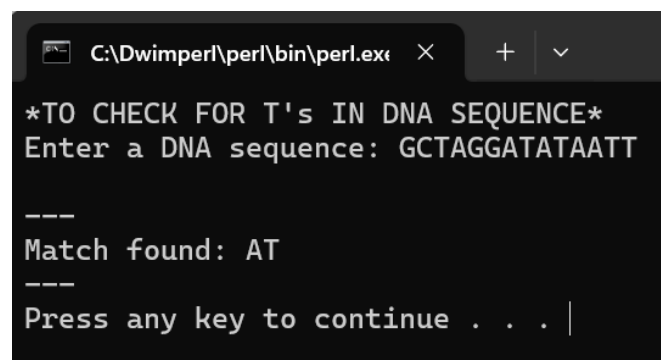
Figure 2: Displaying the output when the user-entered string does not contain allowed characters

Question 2:

Write a Perl program that matches a string that has an a followed by zero or more t's in a DNA sequence.

CODE:

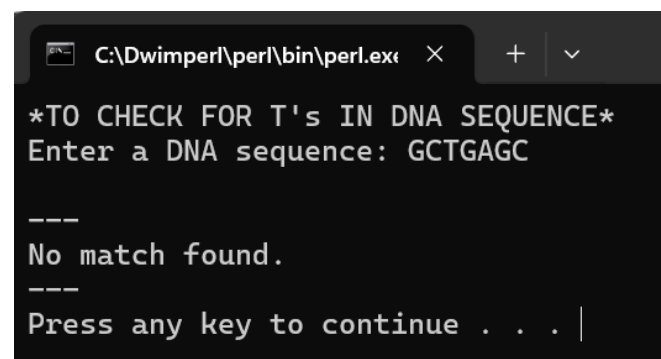
```
print("*TO CHECK FOR T's IN DNA SEQUENCE*\n");
print("Enter a DNA sequence: ");
my $dna = <STDIN>;
chomp($dna);
print("\n---\n");
if ($dna =~ m/a+t+/i) {
    print("Match found: $&\n");
}
else {
    print("No match found.\n");
}
print("---\n");
```

OUTPUT:A screenshot of a Windows command prompt window titled "C:\Dwimperl\perl\bin\perl.exe". The output of the Perl script is displayed in white text on a black background. It shows the prompt "*TO CHECK FOR T's IN DNA SEQUENCE*", followed by the input "Enter a DNA sequence: GCTAGGATATAATT". After a separator line "---", it displays "Match found: AT". Another separator line "---" follows, and then the prompt "Press any key to continue . . . |".

```
C:\Dwimperl\perl\bin\perl.exe
*TO CHECK FOR T's IN DNA SEQUENCE*
Enter a DNA sequence: GCTAGGATATAATT

---
Match found: AT
---
Press any key to continue . . . |
```

Figure 3: Displaying the output when the DNA sequence has zero or more T's

A screenshot of a Windows command prompt window titled "C:\Dwimperl\perl\bin\perl.exe". The output of the Perl script is displayed in white text on a black background. It shows the prompt "*TO CHECK FOR T's IN DNA SEQUENCE*", followed by the input "Enter a DNA sequence: GCTGAGC". After a separator line "---", it displays "No match found.". Another separator line "---" follows, and then the prompt "Press any key to continue . . . |".

```
C:\Dwimperl\perl\bin\perl.exe
*TO CHECK FOR T's IN DNA SEQUENCE*
Enter a DNA sequence: GCTGAGC

---
No match found.
---
Press any key to continue . . . |
```

Figure 4: Displaying the output when the DNA sequence has no match found

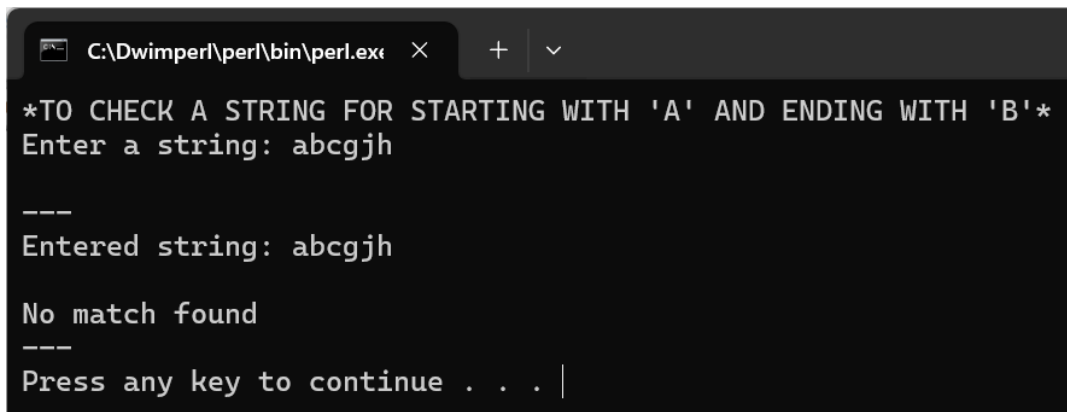
Question 3:

Write a Perl program that matches a string that has an 'a' followed by anything, ending in 'b'.

CODE:

```
print("*TO CHECK A STRING FOR STARTING WITH 'A' AND ENDING WITH 'B'*\n");
print("Enter a string: ");
my $string1 = <STDIN>;
print("\n---\nEnter string: $string1\n");
chomp($string1);
if ($string1 =~ /a.*b$/) {
    print("Match found: $string1\n");
}
else {
    print("No match found\n");
}
print("---\n");
```

OUTPUT:

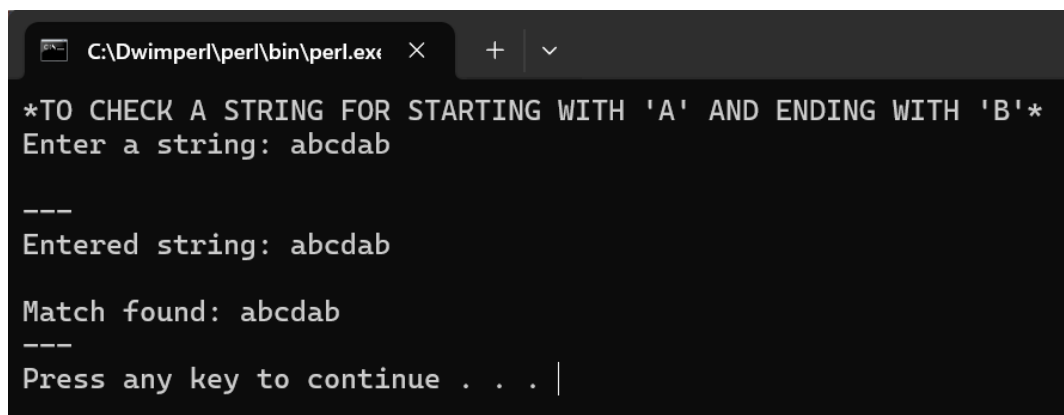
A screenshot of a Windows command prompt window titled 'C:\Dwimperl\perl\bin\perl.exe'. The output of the Perl program is displayed in green text on a black background. It shows the program's title, a prompt to enter a string, the entered string 'abcgjh', and the result 'No match found' because the string does not start with 'a' and end with 'b'. The prompt 'Press any key to continue . . . |' is visible at the bottom.

```
C:\Dwimperl\perl\bin\perl.exe  X  +  v
*TO CHECK A STRING FOR STARTING WITH 'A' AND ENDING WITH 'B'*
Enter a string: abcgjh

---
Entered string: abcgjh

No match found
---
Press any key to continue . . . |
```

Figure 5: Displaying the output when the entered string does not start with 'a' and end with 'b'

A screenshot of a Windows command prompt window titled 'C:\Dwimperl\perl\bin\perl.exe'. The output of the Perl program is displayed in green text on a black background. It shows the program's title, a prompt to enter a string, the entered string 'abcdab', and the result 'Match found: abcdab' because the string starts with 'a' and ends with 'b'. The prompt 'Press any key to continue . . . |' is visible at the bottom.

```
C:\Dwimperl\perl\bin\perl.exe  X  +  v
*TO CHECK A STRING FOR STARTING WITH 'A' AND ENDING WITH 'B'*
Enter a string: abcdab

---
Entered string: abcdab

Match found: abcdab
---
Press any key to continue . . . |
```

Figure 6: Displaying the output when the entered string starts with 'a' and ends with 'b'

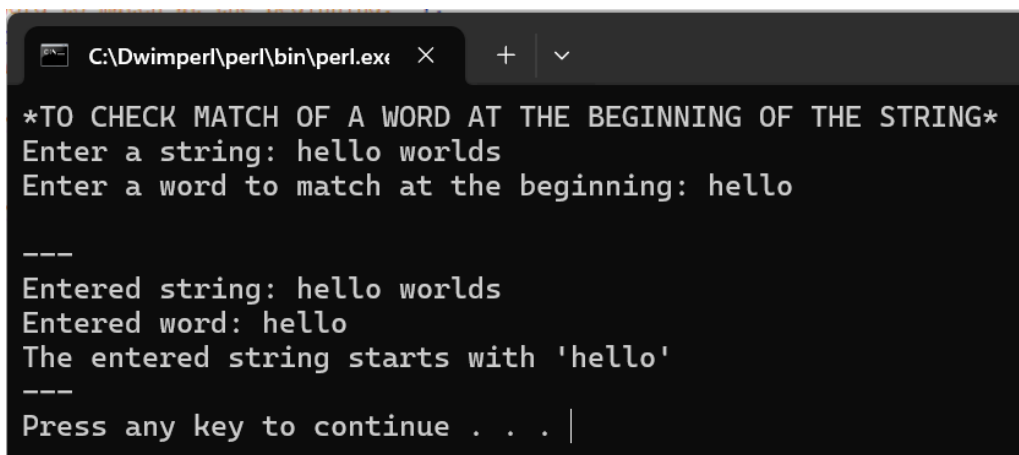
Question 4:

Write a Perl program that matches a word at the beginning of a string.

CODE:

```
print("*TO CHECK MATCH OF A WORD AT THE BEGINNING OF THE STRING*\n");
print("Enter a string: ");
chomp($string1 = <STDIN>);
print("Enter a word to match at the beginning: ");
chomp($word1 = <STDIN>);
print("\n---\nEnter string: $string1\nEntered word: $word1\n");
if ($string1 =~ /^$word1/) {
    print("The entered string starts with '$word1'\n");
}
else {
    print("The string does not start with '$word1'\n");
}
print("---\n");
```

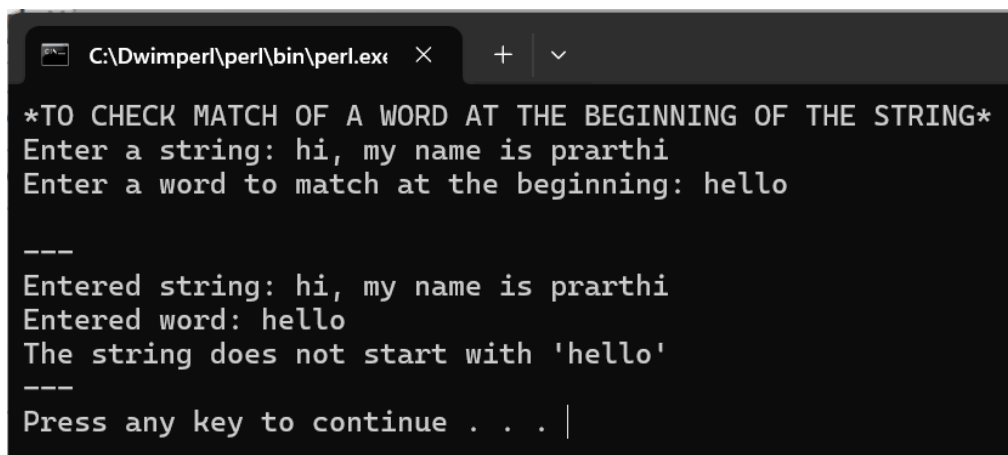
OUTPUT:



```
C:\Dwimper\perl\bin\perl.exe x + v
*TO CHECK MATCH OF A WORD AT THE BEGINNING OF THE STRING*
Enter a string: hello worlds
Enter a word to match at the beginning: hello

---
Entered string: hello worlds
Entered word: hello
The entered string starts with 'hello'
---
Press any key to continue . . . |
```

Figure 7: Displaying the output when the entered string starts with the entered word



```
C:\Dwimper\perl\bin\perl.exe x + v
*TO CHECK MATCH OF A WORD AT THE BEGINNING OF THE STRING*
Enter a string: hi, my name is prarthi
Enter a word to match at the beginning: hello

---
Entered string: hi, my name is prarthi
Entered word: hello
The string does not start with 'hello'
---
Press any key to continue . . . |
```

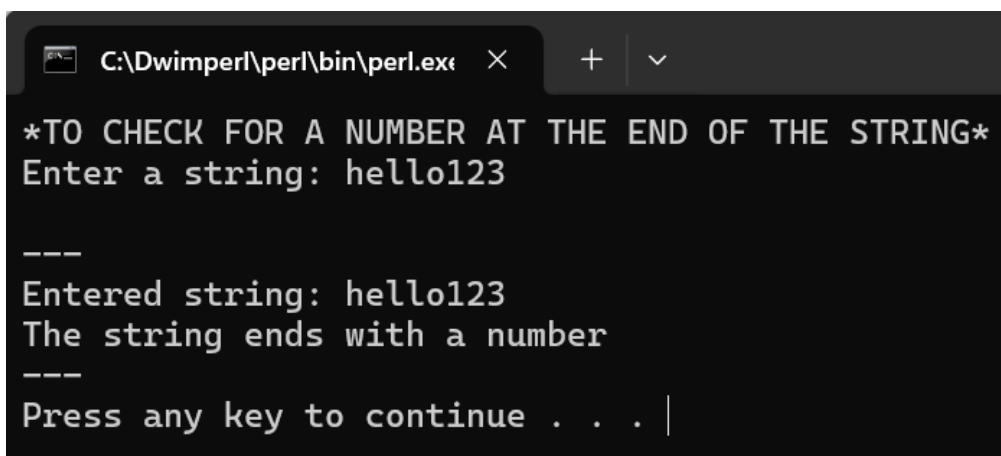
Figure 8: Displaying the output when the entered string does not start with the entered word

Question 5:

Write a Perl program to check for a number at the end of a string.

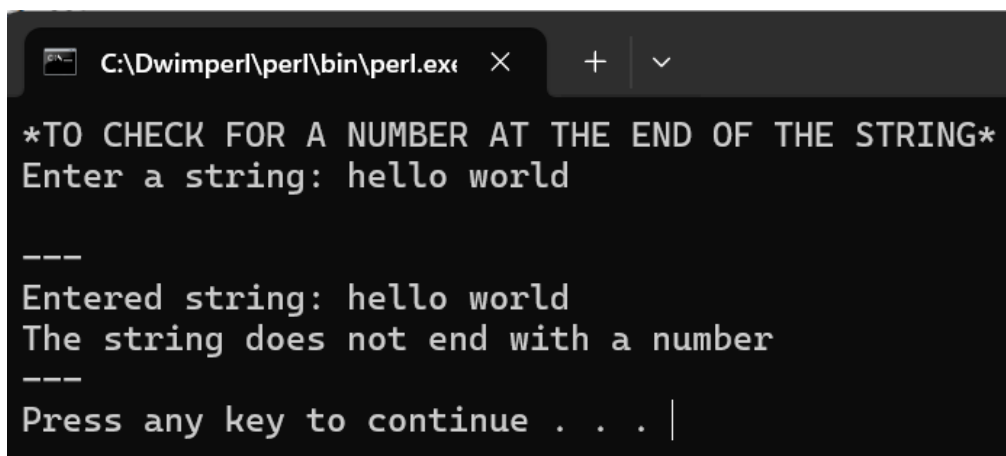
CODE:

```
print("*TO CHECK FOR A NUMBER AT THE END OF THE STRING*\n");
print("Enter a string: ");
chomp($input = <STDIN>);
print("\n---\nEnter string: $input\n");
if ($input =~ /\d+$/) {
    print("The string ends with a number\n");
}
else {
    print("The string does not end with a number\n");
}
print("---\n");
```

OUTPUT:A screenshot of a Windows command prompt window titled "C:\Dwimperl\perl\bin\perl.exe". The window shows the output of a Perl script. The first line is "*TO CHECK FOR A NUMBER AT THE END OF THE STRING*". The second line is "Enter a string: hello123". The third line is "----". The fourth line is "Entered string: hello123". The fifth line is "The string ends with a number". The sixth line is "----". The seventh line is "Press any key to continue . . . |".

```
C:\Dwimperl\perl\bin\perl.exe × + v
*TO CHECK FOR A NUMBER AT THE END OF THE STRING*
Enter a string: hello123
----
Entered string: hello123
The string ends with a number
----
Press any key to continue . . . |
```

Figure 9: Displaying the output when the entered string ends with a number

A screenshot of a Windows command prompt window titled "C:\Dwimperl\perl\bin\perl.exe". The window shows the output of a Perl script. The first line is "*TO CHECK FOR A NUMBER AT THE END OF THE STRING*". The second line is "Enter a string: hello world". The third line is "----". The fourth line is "Entered string: hello world". The fifth line is "The string does not end with a number". The sixth line is "----". The seventh line is "Press any key to continue . . . |".

```
C:\Dwimperl\perl\bin\perl.exe × + v
*TO CHECK FOR A NUMBER AT THE END OF THE STRING*
Enter a string: hello world
----
Entered string: hello world
The string does not end with a number
----
Press any key to continue . . . |
```

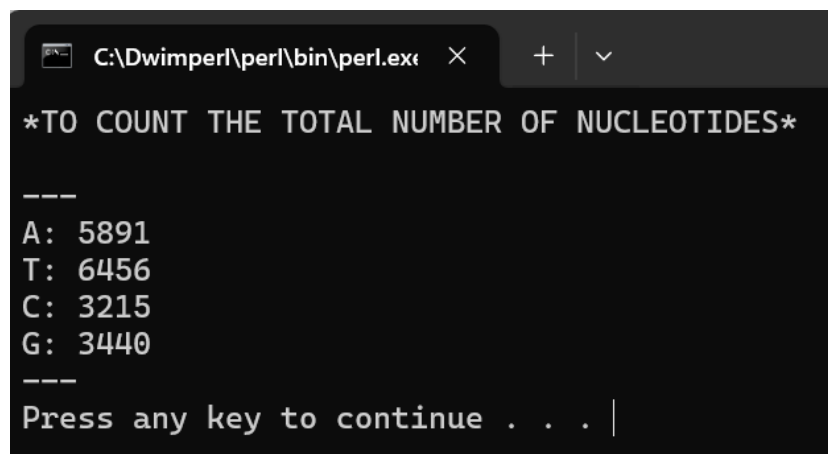
Figure 10: Displaying the output when the entered string does not end with a number

Question 6:

Write a Perl program to count number of nucleotides in a sequence using regex.

CODE:

```
print("*TO COUNT THE TOTAL NUMBER OF NUCLEOTIDES*\n");
my $fasta_file = "sequence.fasta";
open my $fh, '<', $fasta_file or die "Cannot open file '$fasta_file': $!\n";
my $sequence = "";
while (my $line = <$fh>) {
    chomp $line;
    next if $line =~ /^>/;
    $sequence .= $line;
}
close $fh;
my $count_A = () = $sequence =~ /A/gi;
my $count_T = () = $sequence =~ /T/gi;
my $count_C = () = $sequence =~ /C/gi;
my $count_G = () = $sequence =~ /G/gi;
print("\n---\n");
print "A: $count_A\n";
print "T: $count_T\n";
print "C: $count_C\n";
print "G: $count_G\n";
print("---\n");
```

OUTPUT:

```
*TO COUNT THE TOTAL NUMBER OF NUCLEOTIDES*
---
A: 5891
T: 6456
C: 3215
G: 3440
---
Press any key to continue . . . |
```

Figure 11: Displaying the total count of nucleotides in the given fasta file

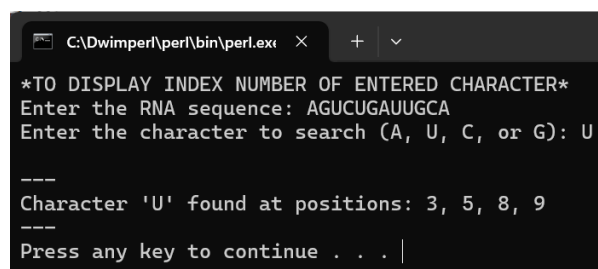
Question 7:

Write a Perl program accept an RNA sequence and display the index number of an entered character.

CODE:

```
print("*TO DISPLAY INDEX NUMBER OF ENTERED CHARACTER*\n");
print "Enter the RNA sequence: ";
my $rna_sequence = <STDIN>;
chomp($rna_sequence);
print "Enter the character to search (A, U, C, or G): ";
my $char_to_search = <STDIN>;
chomp($char_to_search);
$char_to_search = uc($char_to_search);
$rna_sequence = uc($rna_sequence);
my @indices;
for (my $i = 0; $i < length($rna_sequence); $i++) {
    if (substr($rna_sequence, $i, 1) eq $char_to_search) {
        push @indices, $i + 1;
    }
}
print("\n---\n");
if (@indices) {
    print "Character '$char_to_search' found at positions: ", join(", ", @indices),
"\n";
} else {
    print "Character '$char_to_search' not found in the sequence.\n";
}
print("---\n");
```

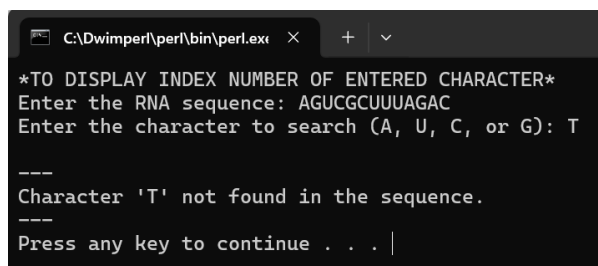
OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe x + v
*TO DISPLAY INDEX NUMBER OF ENTERED CHARACTER*
Enter the RNA sequence: AGUCUGAUUGCA
Enter the character to search (A, U, C, or G): U

---
Character 'U' found at positions: 3, 5, 8, 9
---
Press any key to continue . . . |
```

Figure 12: Displaying the index number of the entered character in an RNA sequence



```
C:\Dwimperl\perl\bin\perl.exe x + v
*TO DISPLAY INDEX NUMBER OF ENTERED CHARACTER*
Enter the RNA sequence: AGUCGCUUAGAC
Enter the character to search (A, U, C, or G): T

---
Character 'T' not found in the sequence.
---
Press any key to continue . . . |
```

Figure 13: Displaying the output when the entered character does not exist in an RNA sequence

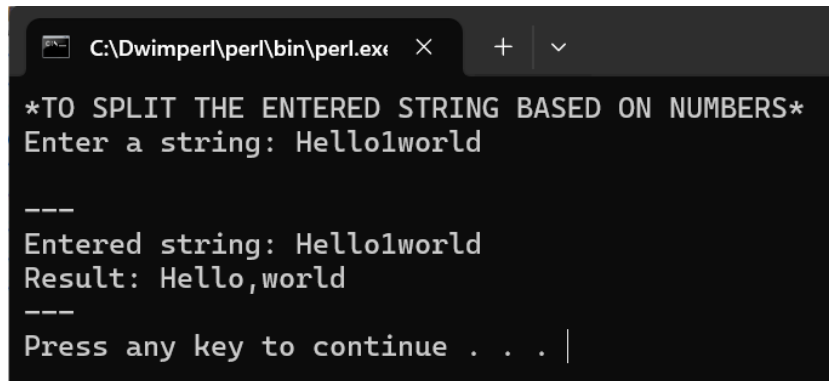
Question 8:

Write a Perl script to split on digit and join using single comma.

CODE:

```
print("*TO SPLIT THE ENTERED STRING BASED ON NUMBERS*\n");
print "Enter a string: ";
my $string = <STDIN>;
chomp($string);
my @value = split(/\d/, $string);
my $result = join(",", @value);
print("\n---\nEnter string: $string\n");
print "Result: $result\n";
print("---\n");
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe x + v
*TO SPLIT THE ENTERED STRING BASED ON NUMBERS*
Enter a string: Hello1world

---
Entered string: Hello1world
Result: Hello,world
---
Press any key to continue . . . |
```

Figure 14: Displaying the string after splitting it based on numbers

Question 9:

Write a Perl script to remove stop codon from file and replace it with NNN.

CODE:

```
print("*TO SPLIT THE ENTERED STRING BASED ON NUMBERS*\n");
my $file = "D:\\Prarthi\\Khalsa - MSc Bioinformatics\\Semester 4\\Mandatory Paper - 2\\Journal\\sequence.fasta";
open(my $fh, "<", $file) or die "Unable to open file: $!\n";
my $sequence = "";
my $id = "";
while (my $line = <$fh>) {
    chomp($line);
    if ($line =~ /^>/) {
        if ($sequence ne "") {
            $sequence =~ s/(TAG|TGA|TAA)/NNN/g;
        }
        $id = $line;
    } else {
        $sequence.= $line;
    }
}
if ($sequence) {
    $sequence =~ s/(TAG|TGA|TAA)/NNN/g;
    print "$id\n$sequence\n";
}
close($fh) or die "Unable to close file: $!\n";
print("---\n");
```

OUTPUT:

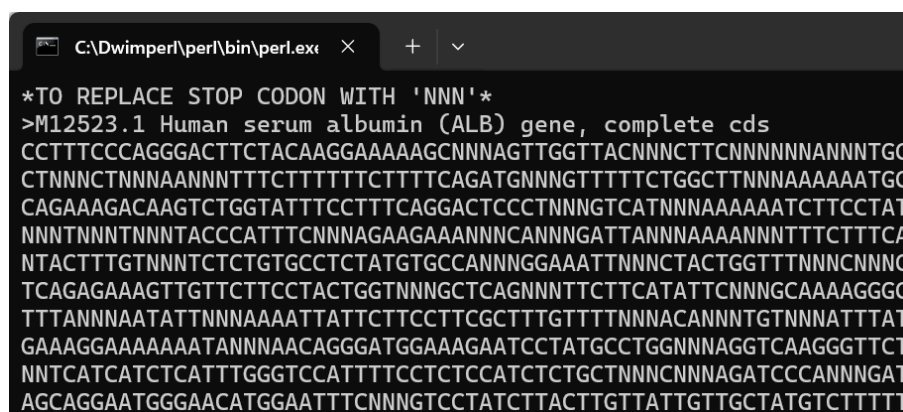


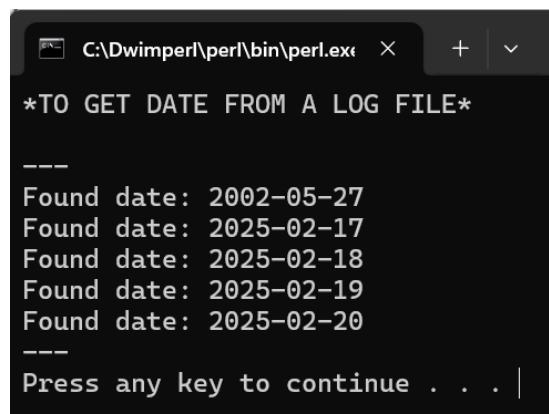
Figure 15: Displaying the output after replacing the stop codons with 'NNN'

Question 10:

Write a Perl script to get date from log file.

CODE:

```
print("*TO GET DATE FROM A LOG FILE*\n");
$log_file = "D:\\Prarthi\\Khalsa - MSc Bioinformatics\\Semester 4\\Mandatory Paper - 2\\Journal\\demo.txt";
open(fh, '<', $log_file) or die $!;
print("\n---\n");
while ($line = <fh>) {
    if ($line =~ /(\\d{4}-\\d{2}-\\d{2})/) {
        print "Found date: $&\n";
    }
}
close fh;
print("---\n");
```

OUTPUT:A screenshot of a Windows command prompt window titled "C:\\Dwimperl\\perl\\bin\\perl.exe". The window displays the output of a Perl script. The output starts with a header line "*TO GET DATE FROM A LOG FILE*", followed by a separator line "----". Then, five lines of output are shown, each starting with "Found date:" followed by a date in YYYY-MM-DD format: "2002-05-27", "2025-02-17", "2025-02-18", "2025-02-19", and "2025-02-20". The output ends with another separator line "----" and a prompt "Press any key to continue . . . |".

```
C:\Dwimperl\perl\bin\perl.exe
*TO GET DATE FROM A LOG FILE*
----
Found date: 2002-05-27
Found date: 2025-02-17
Found date: 2025-02-18
Found date: 2025-02-19
Found date: 2025-02-20
----
Press any key to continue . . . |
```

Figure 16: Displaying the output after retrieving the dates from a log file

RESULTS:

Simple programs based on the concept of Metacharacters, Quantifiers and Substring and File handling used in Perl were coded and executed to display the output when the user-entered string contains allowed characters and does not contain allowed characters, to display the output when the DNA sequence has zero or more T's and when the DNA sequence has no match found, to display the output when the entered string starts with and does not start with 'a' and end with 'b', to display the output when the entered string starts with the entered word and does not start with the entered word, to display the output when the entered string ends with a number and does not end with a number, to display the total count of nucleotides in the given fasta file, to display the index number of the entered character in an RNA sequence, to display the output when the entered character does not exist in an RNA sequence, to display the string after splitting it based on numbers and to display the output after replacing the stop codons with 'NNN'.

CONCLUSION:

Simple programs based on the concept of Metacharacters, Quantifiers and Substring and File handling used in Perl were coded and executed.

REFERENCES:

1. O'Reilly & Associates, Inc. (n.d.). Metacharacters and Metasymbols (Programming perl). https://docstore.mik.ua/orelly/perl/prog3/ch05_03.htm
 2. Perl File handling - Tpoint Tech. (n.d.). www.tpointtech.com. <https://www.tpointtech.com/perl-file-handling>
 3. perlre - Perl regular expressions - Perldoc Browser. (n.d.). <https://perldoc.perl.org/perlre>
 4. perlrecharclass - Perl Regular Expression Character Classes - Perldoc Browser. (n.d.). <https://perldoc.perl.org/perlrecharclass>
-

PRACTICAL 8: PERL **PERL FORMATTING**

AIM:

To create simple programs based on the concept of perl formatting used in Perl.

INTRODUCTION:

Larry Wall created Perl in 1987 as a high-level, interpreted programming language initially for text manipulation. Over time, it evolved into a versatile language used in web development, system administration, and network programming. Perl's syntax resembles C, making it accessible for C or C++ programmers. It supports both procedural and object-oriented programming, featuring dynamic typing and automatic memory management, enhancing its flexibility and user-friendliness. The acronym 'Practical Extraction and Reporting Language' is commonly associated with Perl.

Perl Formatting

Perl formatting is a powerful feature in the Perl programming language that allows developers to create structured and visually appealing output, particularly for reports. This is achieved through the use of formats, which act as templates defining how data should be displayed. Perl's formatting capabilities provide a robust way to create structured output for reports and other applications. By defining formats with various fieldholders and invoking them correctly, developers can ensure their output is both readable and professionally presented. The flexibility of formats allows for easy adjustments and enhancements based on specific reporting needs.

Defining a Format

To use formatting in Perl, you first need to define a format.

Syntax for defining a format:

```
format FormatName =  
  fieldline  
  value_one, value_two, value_three  
  fieldline  
  value_one, value_two
```

.

FormatName: This is the identifier for the format.

fieldline: This line specifies how the data should be formatted. It can include fixed text and fieldholders, which are placeholders for variable data.

The definition ends with a **single period (.)** on its own line.

Fieldholders

Fieldholders determine how data will be aligned in the output. Common types include:

1. @<<<<: Left-justified with a specified width.
2. @>>>>: Right-justified.
3. @|||: Centered.
4. @####.##: Numeric field holder for decimal values.
5. @*: Multiline field holder that allows for longer text entries.

Example of a Format Definition:

```
format EMPLOYEE =
=====
@<<<<<<<<<<<<<<<<<< @<< $name $age
@#####.## $salary
=====
```

In this example, \$name will be left-aligned within 22 character spaces, followed by age in two spaces and salary formatted as a number with two decimal places.

Using the Format

Once a format is defined, it can be invoked using the `write` function. Before calling `write`, you must set the current format using the special variable `$~`:

```
select(STDOUT); # Direct output to standard output
$~ = "EMPLOYEE"; # Set the format to use
```

Example Program

Here's a simple Perl script demonstrating how to use formats:

```
#!/usr/bin/perl

format EMPLOYEE =
=====
@<<<<<<<<<<<<<<<<<<< @<< $name $age
@#####.## $salary
=====
```

```
select(STDOUT);
$~ = EMPLOYEE;
```

```
@n = ("Ali", "Raza", "Jaffer");
@a = (20, 30, 40);
@s = (2000.00, 2500.00, 4000.00);
```

```

$i = 0;
foreach (@n) {
    $name = $_;
    $age = $a[$i];
    $salary = $s[$i++];
    write; # Output formatted data
}

```

When executed, this program produces neatly formatted output for each employee's name, age, and salary.

Advanced Formatting Features

1. Multiline Fields

The multiline fieldholder (@*) allows for inserting values that span multiple lines without truncation. For example:

```

format MYMULTILINE =
Text Before.
@*
$long_string
Text After.
.

$long_string = "Line 1\nLine 2\nLine 3\n";
write;

```

Output:

```

Text Before.
Line 1
Line 2
Line 3
Text After.

```

2. Special Characters and Value Fields

Perl formats also allow special characters and value fields to be included directly in the format definition. For instance:

```

format SPECIAL =
This line contains the special character @.
"@ "
.

```

This would produce an output showing that special character.

PROGRAMS:

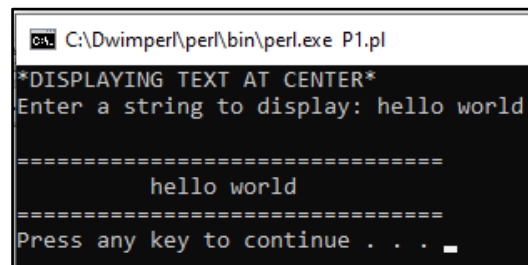
Question 1:

Write a Perl script to display text at center.

CODE:

```
print("*DISPLAYING TEXT AT CENTER*\n");
print("Enter a string to display: ");
chomp($str = <stdin>);
print("\n");
format STDOUT =
=====
@|||||
$str;
=====
.
write;
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe P1.pl
*DISPLAYING TEXT AT CENTER*
Enter a string to display: hello world

=====
                hello world
=====
Press any key to continue . . .
```

Figure 1: Displaying the user-entered text at the center

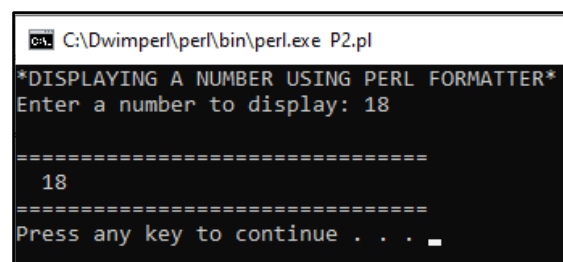
Question 2:

Write a Perl script to accept a number and display using Perl formatter.

CODE:

```
print("*DISPLAYING A NUMBER USING PERL FORMATTER*\n");
print("Enter a number to display: ");
chomp($num = <stdin>);
print("\n");
format STDOUT =
=====
@###
$num;
=====
.
write;
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe P2.pl
*DISPLAYING A NUMBER USING PERL FORMATTER*
Enter a number to display: 18

=====
                18
=====
Press any key to continue . . .
```

Figure 2: Displaying the user-entered number using perl formatter

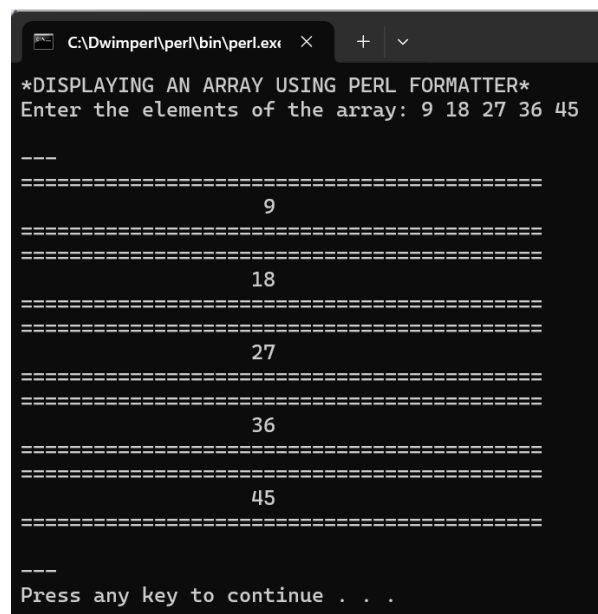
Question 3:

Write a Perl script to accept an array and display the values using Perl formatter.

CODE:

```
print("*DISPLAYING AN ARRAY USING PERL FORMATTER*\n");
print("Enter the elements of the array: ");
@array = split(' ', <STDIN>);
print("\n---\n");
foreach $value (@array) {
    format STDOUT =
=====
@#####
$value
=====
.
    write;
}
print("\n---\n");
```

OUTPUT:

A screenshot of a Windows command prompt window titled "C:\Dwimperl\perl\bin\perl.exe". The window shows the output of a Perl script. It starts with the title "*DISPLAYING AN ARRAY USING PERL FORMATTER*", followed by the prompt "Enter the elements of the array: 9 18 27 36 45". Below this, there is a separator line "----". Then, each element of the array is displayed in a formatted block: "9", "18", "27", "36", and "45". Each block is preceded by a line of equals signs "=====" and followed by a line of equals signs "=====" and a line of hashes "#####". After the last element, there is another separator line "----" and the prompt "Press any key to continue . . .".

```
C:\Dwimperl\perl\bin\perl.exe x + v
*DISPLAYING AN ARRAY USING PERL FORMATTER*
Enter the elements of the array: 9 18 27 36 45

----
=====
@#####
9
=====
=====
18
=====
=====
27
=====
=====
36
=====
=====
45
=====
----
Press any key to continue . . .
```

Figure 3: Displaying the user-entered array using perl formatter

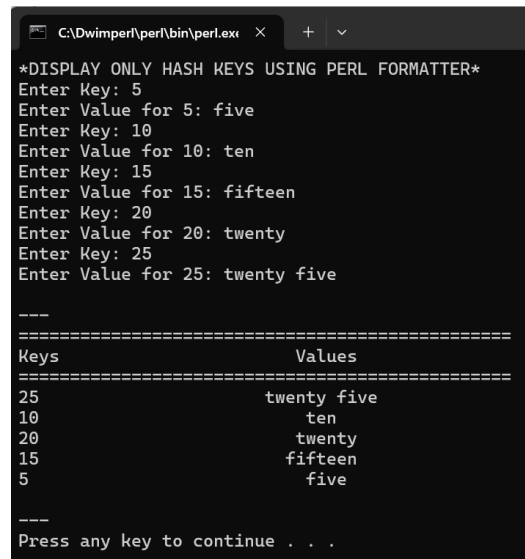
Question 4:

Write a Perl script to accept a hash and display only keys using right/center formatting.

CODE:

[illegible]

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe X + v
*DISPLAY ONLY HASH KEYS USING PERL FORMATTER*
Enter Key: 5
Enter Value for 5: five
Enter Key: 10
Enter Value for 10: ten
Enter Key: 15
Enter Value for 15: fifteen
Enter Key: 20
Enter Value for 20: twenty
Enter Key: 25
Enter Value for 25: twenty five

---
=====
Keys                               Values
=====
25                                twenty five
10                                ten
20                                twenty
15                                fifteen
5                                 five
---
Press any key to continue . . .
```

Figure 4: Displaying the user-entered hash number using perl formatter

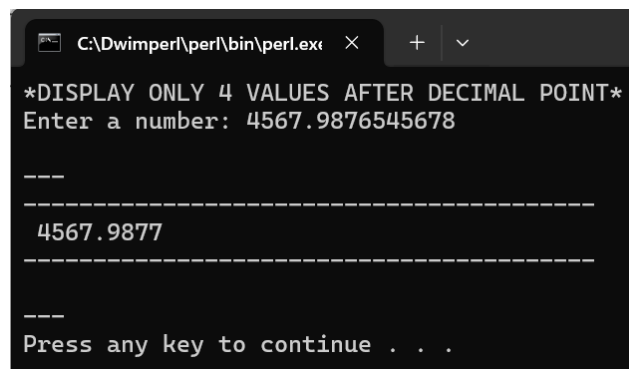
Question 5:

Write a Perl script to accept a decimal number and display only 4 values after decimal using formatting.

CODE:

```
print("*DISPLAY ONLY 4 VALUES AFTER DECIMAL POINT*\n");
print("Enter a number: ");
$num = <STDIN>;
print("\n---\n");
format STDOUT =
-----
@#####.####
$num
-----
.
write;
print("\n---\n");
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe X + v
*DISPLAY ONLY 4 VALUES AFTER DECIMAL POINT*
Enter a number: 4567.9876545678

---
-----
4567.9877
-----
---
Press any key to continue . . .
```

Figure 5: Displaying the user-entered hash number using perl formatter

Write a Perl script to display patient_name, patient_age, health_issue and cost_of_treatment of a Patient_details using formatting.

```
print("*DISPLAYING PATIENT DETAILS USING PERL FORMATTER*\n");
format PATIENT =
=====
=====
@<<<<<<<<<<<<<<<<< @<<<<<<<<<<<<<<<<< @<<<<<<<<<<< @#####.##
$patient_name, $patient_age, $health_issue, $cost_of_treatment

=====
=====
.

format PATIENT_TOP=
=====
=====
Patient Name      Patient Age    Health Issue   Treatment Cost
=====
=====
.

select(STDOUT);
$~ = PATIENT;
$^= PATIENT_TOP;

my @patients;
print("Enter the number of patients: ");
my $num_patients = <>;
chomp($num_patients);

for (my $i = 0; $i < $num_patients; $i++) {
    print("\nEnter details for Patient " . ($i + 1) . ": \n");

    print("Patient Name: ");
    my $patient_name = <>;
    chomp($patient_name);

    print("Patient Age: ");
    my $patient_age = <>;
    chomp($patient_age);

    print("Health Issue: ");
    my $health_issue = <>;
    chomp($health_issue);

    print("Cost of Treatment: ");
```

```

my $cost_of_treatment = <>;
chomp($cost_of_treatment);

push @patients, {
    name => $patient_name,
    age => $patient_age,
    issue => $health_issue,
    cost => $cost_of_treatment,
};
}

foreach my $patient (@patients) {
    $patient_name = $patient->{name};
    $patient_age = $patient->{age};
    $health_issue = $patient->{issue};
    $cost_of_treatment = $patient->{cost};
    print("\n---\n");
    write;
    print("\n---\n");
}

```

OUTPUT:

```

C:\Dwimperl\perl\bin\perl.exe
*DISPLAYING PATIENT DETAILS USING PERL FORMATTER*
Enter the number of patients: 2

Enter details for Patient 1:
Patient Name: ABC
Patient Age: 22
Health Issue: Diabetes
Cost of Treatment: 10000

Enter details for Patient 2:
Patient Name: DEF
Patient Age: 90
Health Issue: Cancer
Cost of Treatment: 500000

---
=====
Patient Name      Patient Age      Health Issue      Treatment Cost
=====
ABC                22                Diabetes          10000.00
=====
---
---
=====
DEF                90                Cancer            500000.00
=====
---
Press any key to continue . . . |

```

Figure 6: Displaying the patient details using perl formatter

Question 7:

Write a Perl script to display `sequence_id`, `Sequence` and `alphabet_name` using `REPORT` formatting.

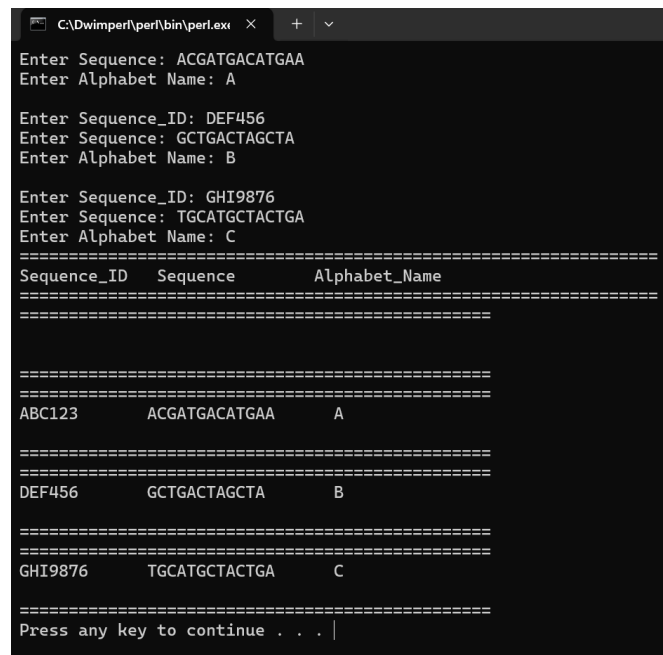
CODE:

```
print("DISPLAYING SEQUENCE INFORMATION USING PERL FORMATTER*\n");  
my @info;  
for (my $i = 0; $i < 3; $i++) {  
    print("\nEnter Sequence_ID: ");  
    chomp(my $sequence_id = <STDIN>);  
  
    print("Enter Sequence: ");  
    chomp(my $sequence = <STDIN>);  
  
    print("Enter Alphabet Name: ");  
    chomp(my $alphabet_name = <STDIN>);  
  
    push @info, {  
        sequence_id => $sequence_id,  
        Sequence     => $sequence,  
        alphabet_name => $alphabet_name,  
    };  
}  
  
format SEQ =  
=====
```

```
foreach my $entry (@info) {
    $sequence_id = $entry->{sequence_id};
    $sequence     = $entry->{Sequence};
    $alphabet_name = $entry->{alphabet_name};

    write;
}
```

OUTPUT:



```
C:\Dwimperl\perl\bin\perl.exe x + v
Enter Sequence: ACGATGACATGAA
Enter Alphabet Name: A

Enter Sequence_ID: DEF456
Enter Sequence: GCTGACTAGCTA
Enter Alphabet Name: B

Enter Sequence_ID: GHI9876
Enter Sequence: TGCATGCTACTGA
Enter Alphabet Name: C

=====
Sequence_ID  Sequence      Alphabet_Name
=====
=====
ABC123       ACGATGACATGAA  A
=====
=====
DEF456       GCTGACTAGCTA   B
=====
=====
GHI9876      TGCATGCTACTGA  C
=====
Press any key to continue . . . |
```

Figure 7: Displaying the sequence details using perl formatter

RESULTS:

Simple programs based on the concept of perl formatting used in Perl were coded and executed to display the user-entered text at the center, to display the user-entered number using perl formatter, to display the user-entered array using perl formatter, to display the user-entered hash number using perl formatter, to display the user-entered hash number using perl formatter, to display the patient details using perl formatter and to display the sequence details using perl formatter.

CONCLUSION:

Simple programs based on the concept of perl formatting used in Perl were coded and executed.

REFERENCES:

1. Perl - Formats. (n.d.). https://www.tutorialspoint.com/perl/perl_formats.htm
2. perlform - Perl formats - Perldoc Browser. (n.d.). <https://perldoc.perl.org/perlform>

PRACTICAL 9: PERL **OOPS IN PERL AND DBI IN PERL**

AIM:

To create simple programs based on the concept of OOPs and DBI used in Perl.

INTRODUCTION:

Larry Wall created Perl in 1987 as a high-level, interpreted programming language initially for text manipulation. Over time, it evolved into a versatile language used in web development, system administration, and network programming. Perl's syntax resembles C, making it accessible for C or C++ programmers. It supports both procedural and object-oriented programming, featuring dynamic typing and automatic memory management, enhancing its flexibility and user-friendliness. The acronym 'Practical Extraction and Reporting Language' is commonly associated with Perl.

OOPs in Perl Programming Language

Object-Oriented Programming (OOPS) in Perl is based on references and anonymous arrays and hashes. Perl's OOPS model is flexible and allows for various programming styles.

Key Concepts

1. **Objects:** In Perl, an object is a reference to a data type that knows its class. It is stored as a reference in a scalar variable.
2. **Classes:** A class is a package that contains methods for creating and manipulating objects.
3. **Methods:** A method is a subroutine defined within a package. The first argument to a method is typically an object reference or a package name.
4. **Inheritance:** Inheritance allows a child class to inherit properties and methods from a parent class.
5. **Encapsulation:** Encapsulation involves hiding the implementation details of an object from the outside world and only exposing necessary information through methods.

Defining a Class

To define a class in Perl, you create a package. A package is a self-contained unit of user-defined variables and subroutines that can be reused.

Example: package Person;

Constructors

A constructor is a method that creates an instance of a class. It is typically named new.

```
sub new {  
    my $class = shift;  
    my $self = {  
        _firstName => shift,  
        _lastName => shift,  
        _ssn      => shift,  
    };  
    bless $self, $class;  
    return $self;  
}
```

bless Function: The bless function associates the object with its class. It is used to mark the reference as an object of the specified class.

Inheritance

Inheritance allows a child class to inherit properties and methods from a parent class. Perl uses the @ISA array to manage inheritance.

```
package Employee;  
our @ISA = qw(Person);
```

Example

Here's a simple example of a Person class and an Employee class that inherits from Person.

```
package Person;  
  
sub new {  
    my $class = shift;  
    my $self = {  
        _firstName => shift,  
        _lastName => shift,  
        _ssn      => shift,  
    };  
    bless $self, $class;  
    return $self;  
}  
  
sub getFirstName {  
    my $self = shift;  
    return $self->{_firstName};  
}
```

```

package Employee;
our @ISA = qw(Person);

sub new {
    my $class = shift;
    my $self = $class->SUPER::new(shift, shift, shift);
    $self->{_department} = shift;
    return $self;
}

sub getDepartment {
    my $self = shift;
    return $self->{_department};
}

# Usage
my $employee = Employee->new("John", "Doe", "123456789", "Sales");
print $employee->getFirstName() . "\n";
print $employee->getDepartment() . "\n";

```

DBI in Perl

The Database Interface (DBI) is a Perl module that provides a consistent interface to various databases. It acts as a layer between the Perl application and the database driver modules.

Key Features

1. **Database Independence:** DBI allows you to write database-independent code.
2. **Prepared Statements:** DBI supports prepared statements for efficient query execution.
3. **Error Handling:** DBI provides robust error handling mechanisms.

Basic Usage

1. **Loading DBI:** Start by loading the DBI module.

```
use DBI;
```

2. **Connecting to a Database:** Connect to a database using the connect method. The Data Source Name (DSN) specifies the database type and connection details.

```
my $dbh = DBI->connect($dsn, $user, $password, \%attr);
```

- **DSN Format:** For example, for MySQL, the DSN might look like "DBI:mysql:database=test;host=localhost".
- **Attributes:** Common attributes include RaiseError => 1 to automatically die on errors and AutoCommit => 0 for transaction support.

3. **Preparing and Executing Queries:** Prepare a query using the prepare method and execute it using the execute method.

```
my $sth = $dbh->prepare("SELECT * FROM table");
$sth->execute();
```

4. **Fetching Data:** Fetch data from the query results using methods like fetchrow_arrayref.

```
while (my $row = $sth->fetchrow_arrayref()) {
    print "@$row\n";
}
```

5. **Disconnecting:** Finally, disconnect from the database.

```
$dbh -> disconnect;
```

Example

Here's a complete example of connecting to a MySQL database, executing a query, and fetching results.

```
use DBI;

# Connect to the database
my $dsn = "DBI:mysql:database=test;host=localhost";
my $user = "user";
my $password = "password";
my $dbh = DBI->connect($dsn, $user, $password,
    { RaiseError => 1, AutoCommit => 0 });

# Prepare and execute a query
my $sth = $dbh->prepare("SELECT * FROM users");
$sth->execute();

# Fetch and print results
while (my $row = $sth->fetchrow_arrayref()) {
    print "@$row\n";
}

# Clean up
$sth->finish();
$dbh->commit(); # If AutoCommit is off
$dbh->disconnect();
```


Advanced Features

- **Transactions:** Use transactions for atomic operations. Start with `AutoCommit => 0`, then manually call `commit` or `rollback`.
- **Prepared Statements with Parameters:** Use placeholders (?) in queries and pass parameters to execute for safer and more efficient queries.

Binding Values

Using **binding values** in Perl's DBI is a powerful feature that allows you to safely and efficiently execute SQL queries by separating the SQL code from the data. This approach helps prevent SQL injection attacks and improves performance by allowing the database to cache query plans.

Why Use Binding Values?

1. **Security:** Binding values prevent SQL injection by ensuring that user input is treated as literal data, not as part of the SQL command.
2. **Performance:** By using placeholders, the database can cache the query plan, which can significantly improve performance when executing the same query multiple times with different data.
3. **Convenience:** Binding values simplify the process of handling different data types and quoting requirements.

How to Use Binding Values

1. **Prepare the Query:** Use placeholders (?) in your SQL query where you want to insert data.

```
my $sth = $dbh->prepare("SELECT * FROM users WHERE name = ?");
```

2. **Execute with Bind Values:** Pass the actual values to the execute method.

```
my $name = "John";  
$sth->execute($name) or die $DBI::errstr;
```

3. **Binding Multiple Values:** When you need to bind multiple values, you simply pass them as separate arguments to the execute method.

```
my $sth = $dbh->prepare("SELECT * FROM users WHERE name = ? AND age = ?");  
my $name = "John";  
my $age = 30;  
$sth->execute($name, $age) or die $DBI::errstr;
```

Binding Lists for IN Queries

Binding a list of values for an IN query is a bit more complex because DBI does not directly support binding arrays to a single placeholder. Instead, you need to create a placeholder for each value in the list.

```
my @values = ('value1', 'value2', 'value3');
my $placeholders = join ' ', ' (?)' x @values;
my $sth = $dbh->prepare("SELECT * FROM table WHERE column IN
($placeholders)");
$sth->execute(@values) or die $DBI::errstr;
```

Example

Here's a complete example of using binding values to execute a query:

```
use DBI;

# Connect to the database
my $dsn = "DBI:mysql:database=test;host=localhost";
my $user = "user";
my $password = "password";
my $dbh = DBI->connect($dsn, $user, $password,
    { RaiseError => 1, AutoCommit => 0 });

# Prepare the query with placeholders
my $name = "John";
my $age = 30;
my $sth = $dbh->prepare("SELECT * FROM users WHERE name = ? AND age = ?");

# Execute the query with bind values
$sth->execute($name, $age) or die $DBI::errstr;

# Fetch and print results
while (my $row = $sth->fetchrow_arrayref()) {
    print "@$row\n";
}

# Clean up
$sth->finish();
$dbh->disconnect();
```

PROGRAMS:

Question 1:

Write a Perl script to create class Square and find area of square.

CODE:

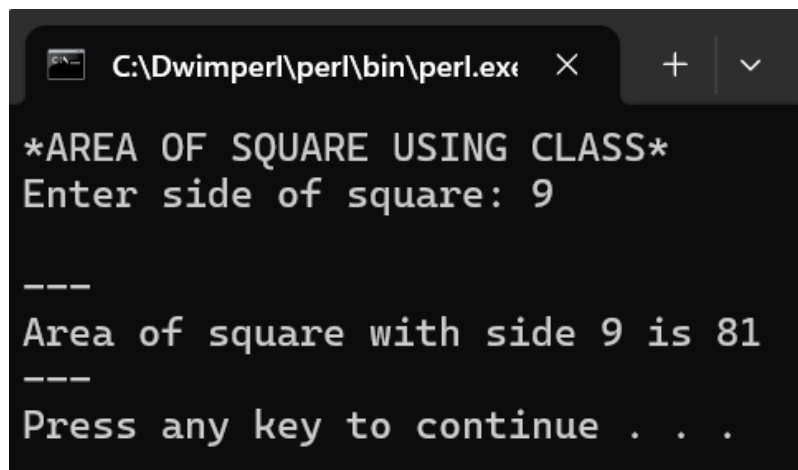
Square.pm

```
package Square;
sub new {
    my $class = shift;
    my $self = {};
    bless $self, $class;
    return $self;
}
sub area {
    my ($self, $side) = @_;
    return $side * $side;
}
1;
```

Square.pl

```
use Square;
$obj = Square->new();
print("*AREA OF SQUARE USING CLASS*\n");
print "Enter side of square: ";
chomp($side = <STDIN>);
$area = $obj->area($side);
print("\n---\n");
print "Area of square with side $side is $area\n";
print("---\n");
```

OUTPUT:

A screenshot of a Windows command prompt window titled "C:\Dwimperl\perl\bin\perl.exe". The window shows the output of a Perl script. The first line is "*AREA OF SQUARE USING CLASS*". The second line is "Enter side of square: 9". The third line is "---". The fourth line is "Area of square with side 9 is 81". The fifth line is "---". The sixth line is "Press any key to continue . . .".

```
C:\Dwimperl\perl\bin\perl.exe
*AREA OF SQUARE USING CLASS*
Enter side of square: 9
---
Area of square with side 9 is 81
---
Press any key to continue . . .
```

Figure 1: Calculating and displaying the area of the square using class

Question 2:

Write a Perl script to Create a student class and initialize it with name and roll number by using hash reference. Make methods to Display - It should display all information's of the student.

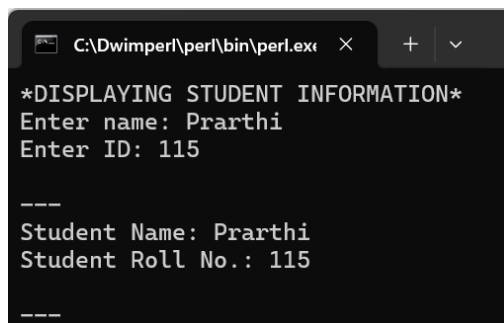
CODE:**Student.pm**

```
package Student;
sub new {
    my $class = shift;
    my %args = @_;
    my $self = {
        name => $args{name},
        roll_no => $args{roll_no},
    };
    bless $self, $class;
    return $self;
}
sub display {
    my $self = shift;
    print("\n---\n");
    print "Student Name: $self->{name}";
    print "Student Roll No.: $self->{roll_no}\n---\n";
}
1;
```

Student.pl

```
use Student;
print("*DISPLAYING STUDENT INFORMATION*\n");
print("Enter name: ");
$name=<stdin>;
print("Enter ID: ");
$student_id=<stdin>;

my $obj = Student->new(
    name => $name,
    roll_no => $student_id
);
$obj->display();
```

OUTPUT:

```
C:\Dwimperl\perl\bin\perl.exe
*DISPLAYING STUDENT INFORMATION*
Enter name: Prarthi
Enter ID: 115

---
Student Name: Prarthi
Student Roll No.: 115
---
```

Figure 2: Displaying student information using class and hash reference

Question 3:

Write a Perl script to create class Perimeter to calculate perimeter of a square and triangle using inheritance.

CODE:**Perimeter.pm**

```
package Perimeter;
sub new {
    my ($class, $side, $side1, $side2, $side3) = @_;
    my $self = {
        'side' => $side,
        'side1' => $side1,
        'side2' => $side2,
        'side3' => $side3,
    };
    bless $self, $class;
    return $self;
}
sub peri_square {
    my $self = shift;
    print "Enter the side of the square: ";
    my $side = <STDIN>;
    chomp($side);
    $self->{side} = $side;
    return 4 * $self->{side}; }
sub peri_triangle {
    my $self = shift;
    print "\nEnter the 1st side of the triangle: ";
    my $side1 = <STDIN>;
    chomp($side1);
    $self->{side1} = $side1;
    print "Enter the 2nd side of the triangle: ";
    my $side2 = <STDIN>;
    chomp($side2);
    $self->{side2} = $side2;
    print "Enter the 3rd side of the triangle: ";
    my $side3 = <STDIN>;
    chomp($side3);
    $self->{side3} = $side3;
    return $self->{side1} + $self->{side2} + $self->{side3};
}
1;
```

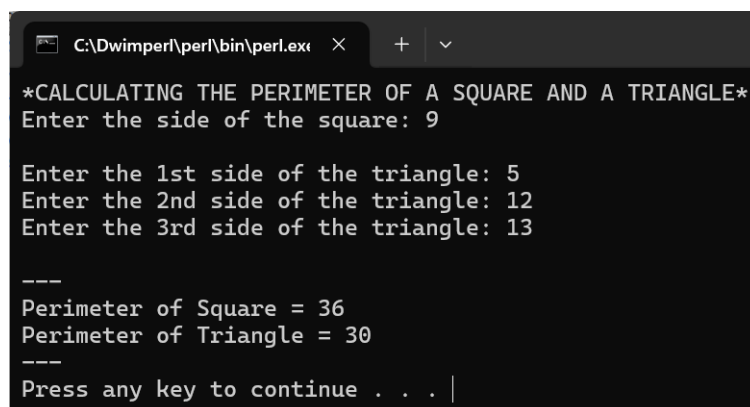
Calculation.pm

```
package Calculation;
use Perimeter;
our @ISA = ('Perimeter'); # Inherit from Perimeter class
sub new {
    my ($class) = shift;
    my $self = {
        sq_perimeter => undef, # To store square's perimeter
        tri_perimeter => undef, # To store triangle's perimeter
    };
    bless $self, $class;
    return $self;
}
sub calculate {
    my $self = shift;
    $self->{sq_perimeter} = $self->{side} * 4;
    print("\n---\n");
    print "Perimeter of Square = $self->{sq_perimeter}\n";
    $self->{tri_perimeter} = $self->{side1} + $self->{side2} + $self->{side3};
    print "Perimeter of Triangle = $self->{tri_perimeter}";
    print("\n---\n");
}
1;
```

Practicals.pl

```
use Calculation;
print("*CALCULATING THE PERIMETER OF A SQUARE AND A TRIANGLE*\n");
$p= new Calculation ($side, $side1, $side2, $side3);
$p->peri_square();
$p->peri_triangle();
$p->calculate();
```

OUTPUT:



```
C:\Dwimper\perl\bin\perl.exe  X  +  v
*CALCULATING THE PERIMETER OF A SQUARE AND A TRIANGLE*
Enter the side of the square: 9

Enter the 1st side of the triangle: 5
Enter the 2nd side of the triangle: 12
Enter the 3rd side of the triangle: 13

---
Perimeter of Square = 36
Perimeter of Triangle = 30
---
Press any key to continue . . . |
```

Figure 3: Calculating and displaying the perimeter of a square and a triangle

Question 4:

Write a Perl script to make a connection with MySQL for any database.

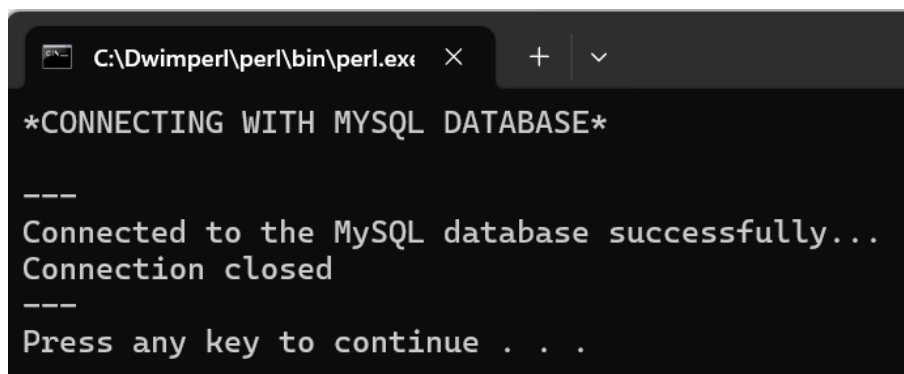
CODE:

```
print("*CONNECTING WITH MYSQL DATABASE*\n");
use DBI;
my $dsn = "DBI:mysql:college","root";
my $username = "root";
my $password = "root";

print("\n---\n");
my $dbh = DBI->connect($dsn, $username, $password)
    or die "Cannot connect to the database: $DBI::errstr";

print "Connected to the MySQL database successfully...\n";

$dbh->disconnect();
print "Connection closed";
print("\n---\n");
```

OUTPUT:A screenshot of a Windows command prompt window titled "C:\Dwimperl\perl\bin\perl.exe". The window shows the output of a Perl script. The output is as follows: *CONNECTING WITH MYSQL DATABASE*, followed by a separator line ---, then "Connected to the MySQL database successfully...", then "Connection closed", followed by another separator line ---, and finally "Press any key to continue . . .".

```
C:\Dwimperl\perl\bin\perl.exe
*CONNECTING WITH MYSQL DATABASE*
---
Connected to the MySQL database successfully...
Connection closed
---
Press any key to continue . . .
```

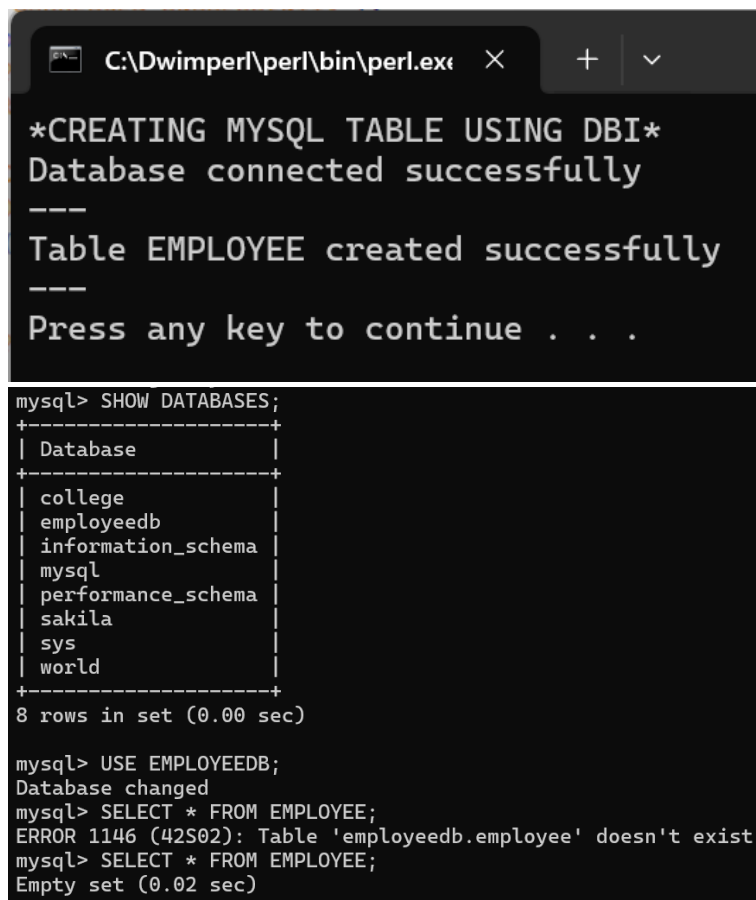
Figure 4: Connection with MySQL successfully established

Question 5:

Write a Perl Script to create table Employee under database Company.

CODE:

```
print("*CREATING MYSQL TABLE USING DBI*\n");
use DBI;
$dbh=DBI->connect("DBI:mysql:employeeedb","root", "root ") or die $DBI::errstr;
print("Database connected successfully");
$sth = $dbh -> prepare("CREATE TABLE EMPLOYEE(
    ID INT PRIMARY KEY AUTO_INCREMENT,
    NAME VARCHAR(20) NOT NULL,
    AGE INT NOT NULL,
    DEPARTMENT VARCHAR(20),
    SALARY DECIMAL(10,2));");
$sth -> execute();
$sth -> finish();
print("\n---\n");
print("Table EMPLOYEE created successfully\n");
print("---\n");
```

OUTPUT:

The screenshot shows a Windows command prompt window titled "C:\Dwimperl\perl\bin\perl.exe". The output of the Perl script is displayed in green text on a black background. It shows the script connecting to a MySQL database named 'employeeedb' and creating a table named 'EMPLOYEE'. The output includes a separator line '---' and a prompt 'Press any key to continue . . .'. Below this, a MySQL command prompt shows the command 'SHOW DATABASES;' and its output, which lists several databases including 'employeeedb'. Then, the command 'USE EMPLOYEEEDB;' is executed, followed by 'SELECT * FROM EMPLOYEE;', which results in an error: 'ERROR 1146 (42S02): Table 'employeeedb.employee' doesn't exist'. Finally, another 'SELECT * FROM EMPLOYEE;' command is executed, resulting in an 'Empty set'.

```
C:\Dwimperl\perl\bin\perl.exe × + v
*CREATING MYSQL TABLE USING DBI*
Database connected successfully
---
Table EMPLOYEE created successfully
---
Press any key to continue . . .

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| college |
| employeeedb |
| information_schema |
| mysql |
| performance_schema |
| sakila |
| sys |
| world |
+-----+
8 rows in set (0.00 sec)

mysql> USE EMPLOYEEEDB;
Database changed
mysql> SELECT * FROM EMPLOYEE;
ERROR 1146 (42S02): Table 'employeeedb.employee' doesn't exist
mysql> SELECT * FROM EMPLOYEE;
Empty set (0.02 sec)
```

Figure 5: Creating a table named 'employee' using DBI

Question 6:

Write a Perl Script to insert two values (one without binding Values and one with binding Values respectively).

CODE:

```
print("INSERTING 2 VALUES WITH AND WITHOUT USING BINDING VALUES*\n");
use DBI;
$dbh=DBI->connect("DBI:mysql:college","root","root") or die $DBI::errstr;
print("\nDatabase connected successfully\n");

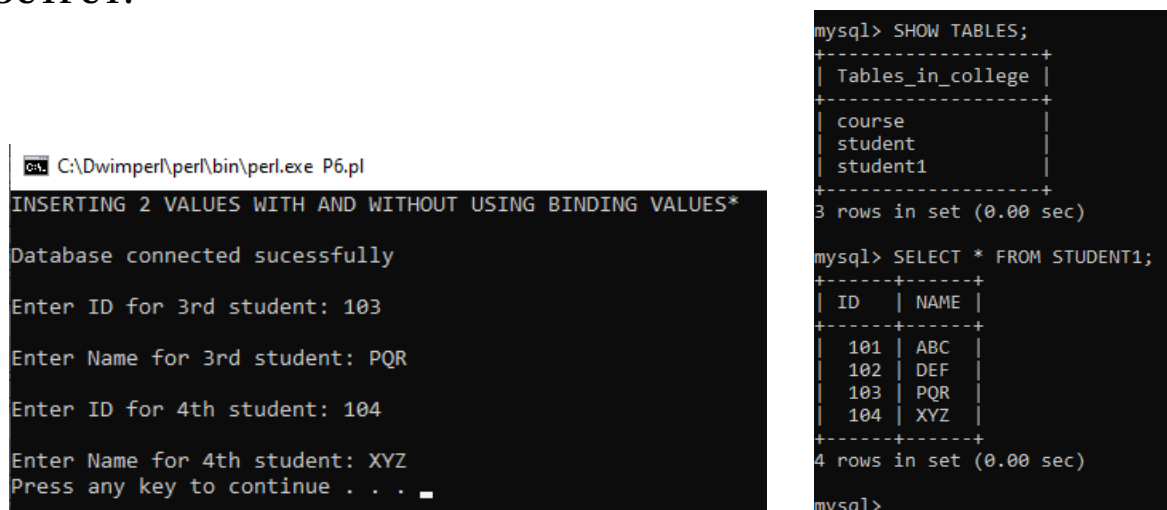
my $sth = $dbh->prepare("CREATE TABLE STUDENT1(ID INT, NAME
VARCHAR(20));");
$sth->execute() or die $DBI::errstr;
$sth->finish();

my $sth = $dbh->prepare("INSERT INTO STUDENT1 (ID, NAME) VALUES (101, 'ABC'),
(102,'DEF');");
$sth->execute() or die $DBI::errstr;
$sth->finish();

print("\nEnter ID for 3rd student: ");
chomp($id3 = <STDIN>);
print("\nEnter Name for 3rd student: ");
chomp($name3 = <STDIN>);
print("\nEnter ID for 4th student: ");
chomp($id4 = <STDIN>);
print("\nEnter Name for 4th student: ");
chomp($name4 = <STDIN>);

my $sth = $dbh->prepare("INSERT INTO STUDENT1 (ID, NAME) VALUES (?,?);");
$sth->execute($id3, $name3) or die $DBI::errstr;
$sth->execute($id4, $name4) or die $DBI::errstr;
$sth->finish();
```

OUTPUT:



The screenshot shows a terminal window with the following output:

```
C:\Dwimper\perl\bin\perl.exe P6.pl
INSERTING 2 VALUES WITH AND WITHOUT USING BINDING VALUES*
Database connected successfully
Enter ID for 3rd student: 103
Enter Name for 3rd student: PQR
Enter ID for 4th student: 104
Enter Name for 4th student: XYZ
Press any key to continue . . .
```

On the right, a MySQL command prompt shows the following output:

```
mysql> SHOW TABLES;
+-----+
| Tables_in_college |
+-----+
| course             |
| student            |
| student1           |
+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM STUDENT1;
+----+-----+
| ID  | NAME |
+----+-----+
| 101 | ABC  |
| 102 | DEF  |
| 103 | PQR  |
| 104 | XYZ  |
+----+-----+
4 rows in set (0.00 sec)

mysql>
```

Figure 6: Inserting 2 values in a table with and without using binding values using DBI

Question 7:

Write a Perl Script to display all data from the DSDDemployee table.

CODE:

```
print("*DISPLAY MYSQL TABLE DATA USING DBI*\n");
use DBI;
$dbh = DBI->connect("DBI:mysql:employee", "root", "root") or die $DBI::errstr;
print("Database connected successfully\n");
$stmt = $dbh->prepare("SELECT * FROM EMPLOYEE");
$stmt->execute() or die $DBI::errstr;
print("\n---Employee Table---\n");
print("\nID  NAME  DEPARTMENT\n");
while (my @row = $stmt->fetchrow_array) {
    print join(" | ", @row), "\n";
}
print("\n---\n");
$stmt->finish();
```

OUTPUT:

The screenshot shows a command prompt window titled 'C:\Dwimperl\perl\bin\perl.exe'. The output of the Perl script is displayed as follows:

```
*DISPLAY MYSQL TABLE DATA USING DBI*
Database connected successfully

---Employee Table---

ID  NAME  DEPARTMENT
101 | JAKE | IT
102 | ALICE | OPERATIONS
103 | TIM | MARKETING

---
Press any key to continue . . .
```

Figure 7: Displaying all data from the table ‘employee’

RESULTS:

Simple programs based on the concept of DBI used in Perl were coded and executed to calculate and display the area of the square using class, to display student information using class and hash reference, to calculate and display the perimeter of a square and a triangle, to establish a connection with MySQL successfully, to create a table named ‘employee’ using DBI, to insert 2 values in a table with and without using binding values using DBI and to display all data from the table ‘employee’.

CONCLUSION:

Simple programs based on the concept of perl formatting used in Perl were coded and executed.

REFERENCES:

1. Brand, H. (n.d.). DBI. MetaCPAN. <https://metacpan.org/pod/DBI>
 2. Hay, S. (n.d.). perloutut. MetaCPAN. <https://metacpan.org/dist/perl/view/pod/perloutut.pod>
 3. Object Oriented Programming in PERL. (n.d.). https://www.tutorialspoint.com/perl/perl_object_oriented.htm
 4. Old school object oriented perl. (n.d.). Perl.com. <https://www.perl.com/article/25/2013/5/20/Old-School-Object-Oriented-Perl/>
 5. Perl - Database Access. (n.d.). https://www.tutorialspoint.com/perl/perl_database_access.htm
 6. Perl DBI - www.perl.org. (n.d.). <https://www.perl.org/dbi/>
 7. perloutut - Object-Oriented Programming in Perl Tutorial - Perldoc Browser. (n.d.). <https://perldoc.perl.org/perloutut>
-

R Practical 1: **INTRODUCTION TO R**

AIM:

To explore R programming language by creating simple programs using R programming language.

INTRODUCTION:

R is a domain-specific, open-source programming language designed primarily for statistical computing, data visualization, and data analysis. It was created by Ross Ihaka and Robert Gentleman in the early 1990s as an alternative to the S programming language developed by John Chambers and colleagues at Bell Laboratories.

Key Features of R

1. **Statistical Computing:** R is exceptionally well-suited for statistical analysis, offering a wide range of procedures including linear and nonlinear modeling, time series analysis, hypothesis testing, and more.
2. **Data Visualization:** It provides extensive capabilities for creating high-quality, publication-ready graphics and visualizations, including scatter plots, bar charts, histograms, and box plots.
3. **Open-Source and Free:** R is available under the GNU General Public License, making it accessible to anyone for free.
4. **Cross-Platform Compatibility:** R can run on various operating systems such as Windows, macOS, and Linux.
5. **Extensive Community Support:** R has a large and active community that contributes packages, documentation, and support, enhancing its functionality and usability.

Uses of R

R is widely used in:

1. **Data Science:** For tasks like data mining, machine learning, and predictive analytics.
2. **Academic Research:** In fields such as bioinformatics, economics, and social sciences for statistical analysis and data visualization.
3. **Business and Industry:** Companies use R for data analysis, statistical modeling, and decision-making processes.

Basic Operations

R can be used as a simple calculator for performing basic arithmetic operations. Here are some examples:

1. **Addition:** $5 + 4$
2. **Subtraction:** $5 - 4$
3. **Multiplication:** $3 * 4$
4. **Division:** $(5 + 5) / 2$
5. **Exponentiation:** 5^2

You can also perform more complex operations like calculating the square root or absolute value:

1. Square root
`sqrt(16)`
2. Absolute value
`abs(-5)`

Data Types and Structures

R supports several data types and structures:

1. Data Types

1. Numeric: Numbers, e.g., 5, 3.14.
2. Integer: Whole numbers, e.g., 5L.
3. Logical: Boolean values, e.g., TRUE, FALSE.
4. Character: Strings, e.g., "Hello".
5. Factor: Categorical data.

2. Data Structures

1. Vectors: A sequence of values of the same type. You can create a vector using the `c()` function:
`myVector <- c(1, 2, 3, 4, 5)`
2. Matrices: A two-dimensional array of values. You can create a matrix using the `matrix()` function:
`myMatrix <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)`
3. Arrays: Multi-dimensional arrays. You can create an array using the `array()` function:
`myArray <- array(c(1, 2, 3, 4, 5, 6), dim = c(2, 3))`
4. Lists: A collection of objects of different types. You can create a list using the `list()` function:
`myList <- list(name = "John", age = 25)`
5. Data Frames: A table of data with rows and columns. You can create a data frame using the `data.frame()` function:
`myDataFrame <- data.frame(
 Name = c("John", "Mary", "David"),
 Age = c(25, 31, 42)
)`

3. Statistical Analysis

R provides extensive capabilities for statistical analysis:

1. Descriptive Statistics
You can calculate mean, median, mode, variance, and standard deviation using built-in functions:
 - i. Calculate mean
`mean(myVector)`
 - ii. Calculate median
`median(myVector)`
 - iii. Calculate variance

- ```
var(myVector)
```
- iv. Calculate standard deviation

```
sd(myVector)
```
  2. Inferential Statistics

R supports various hypothesis tests and confidence intervals:
  3. T-test

```
t.test(myVector)
```
  4. Confidence interval

```
t.test(myVector)$conf.int
```

#### **4. Regression Analysis**

You can perform linear and nonlinear regression using the `lm()` function for linear models:

1. Simple Linear Regression

```
x <- 1:6
y <- x^2
model <- lm(y ~ x)
summary(model)
```

#### **5. Time Series Analysis**

R provides tools for analyzing data over time using packages like `forecast`:

1. # Example time series data

```
ts_data <- ts(c(1, 2, 3, 4, 5, 6), start = c(2023, 1), frequency = 12)
```
2. # Plot time series

```
plot(ts_data)
```

## **PROGRAMS:**

### **Question 1:**

Write a R program to take input from the user (name and age) and display the values.

### **CODE:**

```
name <- readline(prompt = "Enter your name: ")
age <- readline(prompt = "Enter your age: ")
print("---")
print(paste("Name:", name, sep=" "))
print(paste("Age:", age, sep=" "))
print("---")
```

### **OUTPUT:**

```
[1] "Enter name:"
Enter your name: John Doe
[1] "Enter age:"
Enter your age: 23
[1] "Your Name is John Doe"
[1] "Your age is 23"
```

**Figure 1:** R program to take input from the user (name and age) and display the values

### **Question 2:**

Write a R program to read the .csv file and display the content.

### **CODE:**

```
content <- read.csv("../Downloads/Disease.csv")
print(content)
```

### **OUTPUT:**

```
> source("C:/Users/BIOINFO/Desktop/R Practicals/P2.R")
 DiseaseID DiseaseName X
1 1 CAD NA
2 2 LAD stenosis NA
3 3 LCX stenosis NA
4 4 RCA stenosis NA
> |
```

**Figure 2:** R program to read the .csv file and display the content

**Question 3:**

Write a R program to create an array of two 3x3 matrices each with 3 rows and 3 columns from given two vectors.

**CODE:**

```
var1 <- readline(prompt = "Enter vector1: ")
var2 <- readline(prompt = "Enter vector2: ")
a <- array(c(var1, var2), dim = c(3,3,2))
print(a)
```

**OUTPUT:**

```
> source("C:/Users/BIOINFO/Desktop/R Practicals/P3.R")
Enter vector1: cat
Enter vector2: dog
, , 1
 [,1] [,2] [,3]
[1,] "cat" "dog" "cat"
[2,] "dog" "cat" "dog"
[3,] "cat" "dog" "cat"
, , 2
 [,1] [,2] [,3]
[1,] "dog" "cat" "dog"
[2,] "cat" "dog" "cat"
[3,] "dog" "cat" "dog"
```

**Figure 3:** R program to create an array of two 3x3 matrices each with 3 rows and 3 columns from given two vectors



**Question 4:**

Write a R program to create a data frame from four given vectors.

1. To get the structure of a given data frame
2. To get the statistical summary and nature of the data of a given data frame
3. To extract specific column from a data frame using column name
4. To add new row(s) to an existing data frame
5. To add new column(s) to an existing data frame
6. To replace NA values with 3 in each data frame

**CODE:**

```
name <- c("Alice", "Bob", "Charlie", "David", "Eva")
age <- c(25, 30, 35, 40, 40)
height <- c(5.5, 6.0, 5.7, 5.8, 5.4)
score <- c(90, 85, 88, 92, 78)
df <- data.frame(Name = name, Age = age, Height = height, Score = score)
Print the structure of the data frame
print("Structure of the data frame:")
str(df)
Print the statistical summary of the data frame
print("Statistical summary of the data frame:")
summary(df)
Extract the Age column
age_column <- df$Age
Print the extracted Age column
print("Extracted Age column:")
print(age_column)
Add a new row to the data frame
new_row <- data.frame(Name = "Frank", Age = 50, Height = 5.9, Score = 85)
df <- rbind(df, new_row)
Print the data frame after adding the new row
print("Data frame after adding a new row:")
print(df)
Add a new column 'Country' to the data frame
df$Country <- c("USA", "USA", "UK", "Canada", "Australia", "Germany")
Print the data frame after adding the new column
print("Data frame after adding a new column:")
print(df)
Replace NA values with 3
df[is.na(df)] <- 3
Print the data frame after replacing NA values with 3
print("Data frame after replacing NA values with 3:")
print(df)
```

## OUTPUT:

```
[1] "Structure of the data frame:"
'data.frame': 5 obs. of 4 variables:
 $ Name : chr "Alice" "Bob" "Charlie" "David" ...
 $ Age : num 25 30 35 40 40
 $ Height: num 5.5 6 5.7 5.8 5.4
 $ Score : num 90 85 88 92 78
[1] "Statistical summary of the data frame:"
[1] "Extracted Age column:"
[1] 25 30 35 40 40
[1] "Data frame after adding a new row:"
 Name Age Height Score
1 Alice 25 5.5 90
2 Bob 30 6.0 85
3 Charlie 35 5.7 88
4 David 40 5.8 92
5 Eva 40 5.4 78
6 Frank 50 5.9 85
[1] "Data frame after adding a new column:"
 Name Age Height Score Country
1 Alice 25 5.5 90 USA
2 Bob 30 6.0 85 USA
3 Charlie 35 5.7 88 UK
4 David 40 5.8 92 Canada
5 Eva 40 5.4 78 Australia
6 Frank 50 5.9 85 Germany
[1] "Data frame after replacing NA values with 3:"
 Name Age Height Score Country
1 Alice 25 5.5 90 USA
2 Bob 30 6.0 85 USA
3 Charlie 35 5.7 88 UK
4 David 40 5.8 92 Canada
5 Eva 40 5.4 78 Australia
6 Frank 50 5.9 85 Germany
```

**Figure 4:** R program to create a data frame from four given vectors and perform various functions.

**Question 5:**

Write a R program to create a sequence of numbers from 20 to 50 and find the mean of numbers from 20 to 60 and sum of numbers from 51 to 91.

**CODE:**

```
print(paste("Sequence of 20 to 50: ", toString(seq(20,50))), sep=" ")
print(paste("Mean of 20 to 60: ", mean(seq(20,60))), sep=" ")
print(paste("Sum of 20 to 60: ", sum(seq(51,91))), sep=" ")
```

**OUTPUT:**

```
> source("C:/Users/BIOINFO/Desktop/R Practicals/P5.R")
[1] "Sequence of 20 to 50: 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35,
36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50"
[1] "Mean of 20 to 60: 40"
[1] "Sum of 20 to 60: 2911"
```

**Figure 5:** R program to create a sequence of numbers from 20 to 50 and find the mean of numbers from 20 to 60 and sum of numbers from 51 to 91

**Question 6:**

Write R program to read data of CardioGoodFitness.csv and perform the following:

1. To find average age
2. To find median fitness
3. To find mode miles
4. To find variance and standard deviation income

**CODE:**

```
data <- read.csv("C:\\Users\\BIOINFO\\Downloads\\CardioGoodFitness.csv")
average_age <- mean(data$Age)
cat("Average Age:", average_age, "\n")
median_fitness <- median(data$Fitness)
cat("Median Fitness:", median_fitness, "\n")
get_mode <- function(x) {
 uniq_x <- unique(x)
 uniq_x[which.max(tabulate(match(x, uniq_x)))]
}
mode_miles <- get_mode(data$Miles)
cat("Mode Miles:", mode_miles, "\n")
variance_income <- var(data$Income)
sd_income <- sd(data$Income)
cat("Variance of Income:", variance_income, "\n")
cat("Standard Deviation of Income:", sd_income, "\n")
```

## OUTPUT:

```
Average Age: 28.78889
Median Fitness: 3
Mode Miles: 85
Variance of Income: 272470624
Standard Deviation of Income: 16506.68
```

**Figure 6:** R program to read data of CardioGoodFitness.csv and perform various functions on it

## **RESULTS:**

R programs were coded and executed to demonstrate the following: to take input from the user (name and age) and display the values, to read the .csv file and display the content, to create an array of two 3x3 matrices each with 3 rows and 3 columns from given two vectors, to create a data frame from four given vectors and perform various functions, to create a sequence of numbers from 20 to 50 and find the mean of numbers from 20 to 60 and sum of numbers from 51 to 91, to read data of CardioGoodFitness.csv and perform various functions on it

## **CONCLUSION:**

R programs were coded and executed to demonstrate various functionalities of R programming language.

## **REFERENCES:**

1. Neth, H. (2025, February 10). Chapter 1 Basic R concepts and commands | Data Science for Psychologists. <https://bookdown.org/hneth/ds4psy/1-basics.html>
  2. R Language Definition. (n.d.). <https://cran.r-project.org/doc/manuals/r-release/R-lang.html>
  3. R: What is R? (n.d.). <https://www.r-project.org/about.html>
  4. Robinson, S., & Burns, E. (2024, May 31). R programming language. Search Business Analytics. <https://www.techtarget.com/searchbusinessanalytics/definition/R-programming-language>
  5. W3Schools.com. (n.d.). [https://www.w3schools.com/r/r\\_intro.asp](https://www.w3schools.com/r/r_intro.asp)
-

**DATE: 06/03/2025**

## **PRACTICAL 2:** **R - DATA VISUALIZATION**

### **AIM:**

To create simple programs based on the concept of data visualization used in R programming language.

### **INTRODUCTION:**

#### **Data Visualization in R Programming Language**

Data visualization is a crucial aspect of data analysis, allowing users to represent data graphically to gain insights, identify patterns, and communicate findings effectively. R, as a powerful statistical programming language, provides a wide array of tools and functions for data visualization.

#### **Importance of Data Visualization**

Data visualization serves as an indispensable tool in data exploration, inference making, and results presentation. It transforms complex data sets into intuitive graphical representations, facilitating a deeper understanding of the data and enabling the communication of insights in a universally comprehensible manner.

#### **Types of Data Visualizations in R**

R supports various types of data visualizations, including:

1. Scatter Plots: Used to visualize the relationship between two continuous variables. They can be created using functions like `plot()` or `ggplot()`.
2. Line Plots: Display information as a series of data points connected by straight-line segments, primarily used for visualizing data trends over intervals or time.
3. Bar Charts: Used to compare categorical data or display the distribution of a single categorical variable. They can be created with `barplot()` or `geom_bar()` from `ggplot2`.
4. Histograms: Display the distribution of a continuous variable by grouping data into bins. They can be created using `hist()` or `geom_histogram()` from `ggplot2`.
5. Box Plots: Used to compare distributions across groups. They can be created using `boxplot()` or `geom_boxplot()` from `ggplot2`.
6. Heatmaps: Useful for displaying the relationship between two categorical variables or for visualizing large datasets.
7. Pie Charts: Used to show how different categories contribute to a whole.

#### **Data Visualization Packages in R**

R offers several packages for data visualization:

1. `ggplot2`: One of the most versatile and easy-to-use tools for creating elegant graphics. It is part of the tidyverse collection of packages.

2. **plotly**: Provides online interactive and quality graphs, extending upon the JavaScript library `plotly.js`.
3. **lattice**: Offers a more traditional approach to data visualization, similar to `ggplot2` but with different syntax and features.
4. **base graphics**: R's built-in graphics system allows users to create a variety of static plots without the need for external packages.

### **Customizing Visualizations**

R allows users to customize visualizations by adjusting colors, line styles, markers, and text annotations. Users can also modify axis labels, titles, legends, and other graphical elements to enhance the interpretability of the visualizations.

### **Advanced Visualization Techniques**

- **Faceting**: Used to split data into subsets based on categorical variables and display each subset in a separate subplot. This is particularly useful with `ggplot2` using `facet_wrap()` or `facet_grid()`.
- **Interactive Visualizations**: Packages like `plotly` enable the creation of interactive plots that can be shared online or embedded in web pages.

### **Example of Data Visualization in R**

Here's an example using `ggplot2` to create a scatter plot:

```
Install and load ggplot2
install.packages("ggplot2")
library(ggplot2)
Create a scatter plot
ggplot(mtcars, aes(x = wt, y = mpg, col = cyl)) +
 geom_point() +
 labs(x = "Weight", y = "Miles per Gallon", color = "Cylinders")
```

This code creates a scatter plot showing the relationship between car weight and miles per gallon, colored by the number of cylinders.

### **Conclusion**

Data visualization in R is a powerful tool for exploring, analyzing, and presenting data insights. With its extensive range of packages and customization options, R provides users with the flexibility to create high-quality, publication-ready visualizations that effectively communicate complex data insights.

## **PROGRAMS:**

### **Question 1:**

Load titanic.csv and save to an object named titanic.

- Explore the data. What is the structure of the data? Try str(). What are the column names?
- Display column names. How can you get help if you do not know how to use these functions?
- Make a simple scatter plot. Is there a relationship between the age of the passenger and the passenger fare?
- Make a multiple line plot. Is there a relationship between the age of the passenger and the passenger fare?
- Create box plot by creating label on x and y axis and analyze age and survival as any relation? If yes, which age group as survived mostly and how many outliers?

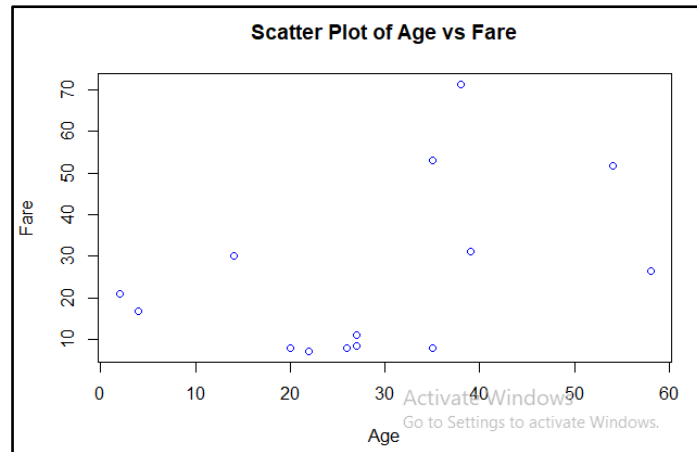
### **CODE:**

```
titanic <- read.csv("titanic.csv")
str(titanic)
colnames(titanic)
plot(titanic$Age, titanic$Fare, main = "Scatter Plot of Age vs Fare", xlab = "Age", ylab = "Fare", col = "blue")
plot(titanic$Age, type = 'l', col = 'red')
lines(titanic$Fare, type = 'l', col = 'blue')
boxplot(titanic$Age ~ titanic$Survived, data = titanic, main = "Box Plot of Age vs Survived", xlab = "Survived", ylab = "Age")
```

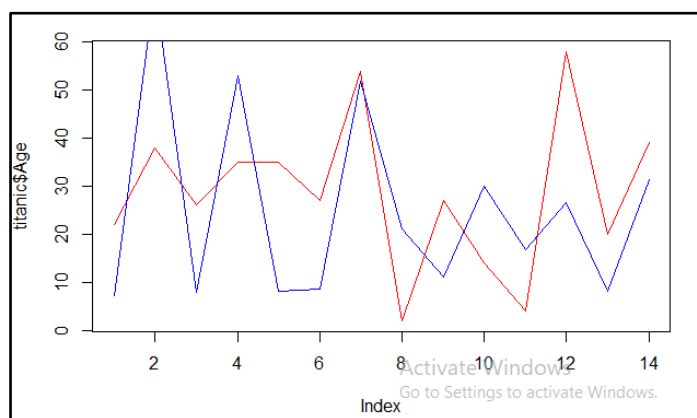
### **OUTPUT:**

```
> source("C:/Users/BIOINFO/Desktop/R Practicals/Practs2/P1.R")
'data.frame': 14 obs. of 8 variables:
 $ Survived : int 0 1 1 1 0 0 0 0 1 1 ...
 $ Pclass : int 3 1 3 1 3 3 1 3 3 2 ...
 $ Name : chr "Mr. Owen Harris Braund" "Mrs. John Bradley (Florence Briggs Thayer) Cumings" "Miss. Laina Heikkinen" "Mrs. Jacques Heath (Lily May Peel) Futrelle" ...
 $ Sex : chr "male" "female" "female" "female" ...
 $ Age : int 22 38 26 35 35 27 54 2 27 14 ...
 $ Siblings.Spouses.Aboard: int 1 1 0 1 0 0 0 3 0 1 ...
 $ Parents.Children.Aboard: int 0 0 0 0 0 0 0 1 2 0 ...
 $ Fare : num 7.25 71.28 7.92 53.1 8.05 ...
```

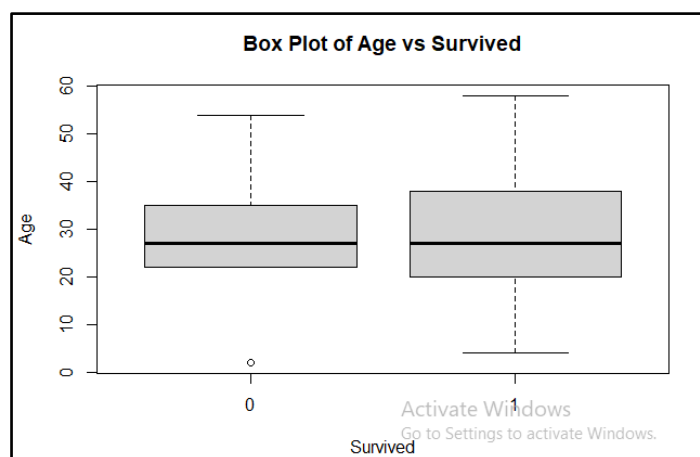
**Figure 1:** Displaying the structure of the data and the column names



**Figure 2:** Displaying a simple scatter plot between the age of the passenger and the passenger fare



**Figure 3:** Displaying a multiple line plot between the age of the passenger and the passenger fare



**Figure 4:** Displaying the box plot to analyse age and survival

### INTERPRETATION:

The graph suggests that age and fare are not directly proportional but do show some correlations at certain points. Some points show parallel movement of the two lines, suggesting that passengers of similar ages may have paid similar fares. Other points show divergence, indicating that fare prices varied significantly for passengers of similar ages. This could be due to differences in class, cabin type, or other ticket-related factors.



### Question 2:

Load Heart Disease Dataset in R and read the contents and display and draw bar chart and boxplot to comparing Age with fasting blood sugar and interpret the result.

### CODE:

```
heart_disease=read.csv("C:\\Users\\BIOINFO\\Downloads\\heart.csv")
print(heart_disease)

boxplot(heart_disease$age~heart_disease$fbs,data=heart_disease,main = "Age by
Classification Level")
```

### OUTPUT:

|    | age | sex | cp | trestbps | chol | fbs | restecg | thalach | exang | oldpeak | slope | ca | thal | target |
|----|-----|-----|----|----------|------|-----|---------|---------|-------|---------|-------|----|------|--------|
| 1  | 52  | 1   | 0  | 125      | 212  | 0   | 1       | 168     | 0     | 1.0     | 2     | 2  | 3    | 0      |
| 2  | 53  | 1   | 0  | 140      | 203  | 1   | 0       | 155     | 1     | 3.1     | 0     | 0  | 3    | 0      |
| 3  | 70  | 1   | 0  | 145      | 174  | 0   | 1       | 125     | 1     | 2.6     | 0     | 0  | 3    | 0      |
| 4  | 61  | 1   | 0  | 148      | 203  | 0   | 1       | 161     | 0     | 0.0     | 2     | 1  | 3    | 0      |
| 5  | 62  | 0   | 0  | 138      | 294  | 1   | 1       | 106     | 0     | 1.9     | 1     | 3  | 2    | 0      |
| 6  | 58  | 0   | 0  | 100      | 248  | 0   | 0       | 122     | 0     | 1.0     | 1     | 0  | 2    | 1      |
| 7  | 58  | 1   | 0  | 114      | 318  | 0   | 2       | 140     | 0     | 4.4     | 0     | 3  | 1    | 0      |
| 8  | 55  | 1   | 0  | 160      | 289  | 0   | 0       | 145     | 1     | 0.8     | 1     | 1  | 3    | 0      |
| 9  | 46  | 1   | 0  | 120      | 249  | 0   | 0       | 144     | 0     | 0.8     | 2     | 0  | 3    | 0      |
| 10 | 54  | 1   | 0  | 122      | 286  | 0   | 0       | 116     | 1     | 3.2     | 1     | 2  | 2    | 0      |
| 11 | 71  | 0   | 0  | 112      | 149  | 0   | 1       | 125     | 0     | 1.6     | 1     | 0  | 2    | 1      |
| 12 | 43  | 0   | 0  | 132      | 341  | 1   | 0       | 136     | 1     | 3.0     | 1     | 0  | 3    | 0      |
| 13 | 34  | 0   | 1  | 118      | 210  | 0   | 1       | 192     | 0     | 0.7     | 2     | 0  | 2    | 1      |
| 14 | 51  | 1   | 0  | 140      | 298  | 0   | 1       | 122     | 1     | 4.2     | 1     | 3  | 3    | 0      |
| 15 | 52  | 1   | 0  | 128      | 204  | 1   | 1       | 156     | 1     | 1.0     | 1     | 0  | 0    | 0      |
| 16 | 34  | 0   | 1  | 118      | 210  | 0   | 1       | 192     | 0     | 0.7     | 2     | 0  | 2    | 1      |
| 17 | 51  | 0   | 2  | 140      | 308  | 0   | 0       | 142     | 0     | 1.5     | 2     | 1  | 2    | 1      |
| 18 | 54  | 1   | 0  | 124      | 266  | 0   | 0       | 109     | 1     | 2.2     | 1     | 1  | 3    | 0      |
| 19 | 50  | 0   | 1  | 120      | 244  | 0   | 1       | 162     | 0     | 1.1     | 2     | 0  | 2    | 1      |

Figure 5: Output of the contents of CSV file

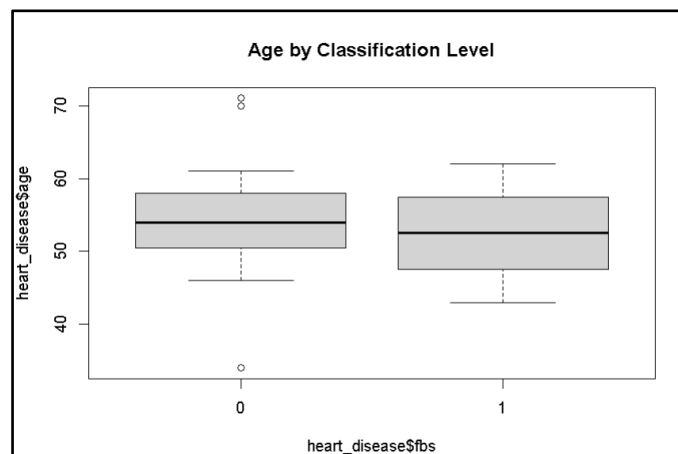


Figure 6: BoxPlot of Age v/s FBS

### INTERPRETATION:

Both groups have similar age distributions. The median age (thick black line inside the boxes) is slightly above 50 for both groups. The interquartile range (IQR, the width of the box) is similar, meaning that the middle 50% of ages are distributed similarly for both fbs groups. A few outliers are visible in the group where fbs = 0, suggesting that some individuals with normal fasting blood sugar levels are significantly younger or older than the general distribution.

### Question 3:

Create data frame for following below data:

Age: 6, 12, 24, 56, 32,34,6,12,24,24, 56, 32, 6, 12, 2, 6, 7, 34, 23, 23, 34

BMI: 17, 18, 27, 31, 29, 29, 27, 34, 12, 27, 31, 29, 17, 18, 27,23,25,27,27,49,45

Classification Labels:

Underweight, Underweight, Overweight, Obese, Overweight, Normal Weight, Obese, Underweight, Overweight, Overweight, Obese, Overweight, Underweight, Underweight, Overweight, Normal Weight, Normal Weight, Normal Weight, Underweight, Underweight, Underweight

Use boxplots to interpret the result between age and Classification Labels.

### CODE:

```
data = data.frame(
 Age = c(6, 12, 24, 56, 32,34,6,12,24,24, 56, 32,6, 12, 2,6,7,34,23,23,34),
 BMI = c(17, 18, 27, 31, 29,29,27,34,12,27, 31, 29,17, 18, 27,23,25,27,27,49,45),
 ClassificationLabels = c('underweight', 'Underweight', 'Overweight', 'Obese',
 'Overweight', 'Normal Weight', 'Obese', 'Underweight', 'Overweight', 'Overweight',
 'Obese', 'Overweight', 'underweight', 'Underweight', 'Overweight', 'Normal Weight',
 'Normal Weight', 'Normal Weight', 'Underweight', 'Underweight', 'Underweight')
)

data$ClassificationLabels = tolower(data$ClassificationLabels)
data$ClassificationLabels = gsub(" ", "_", data$ClassificationLabels)

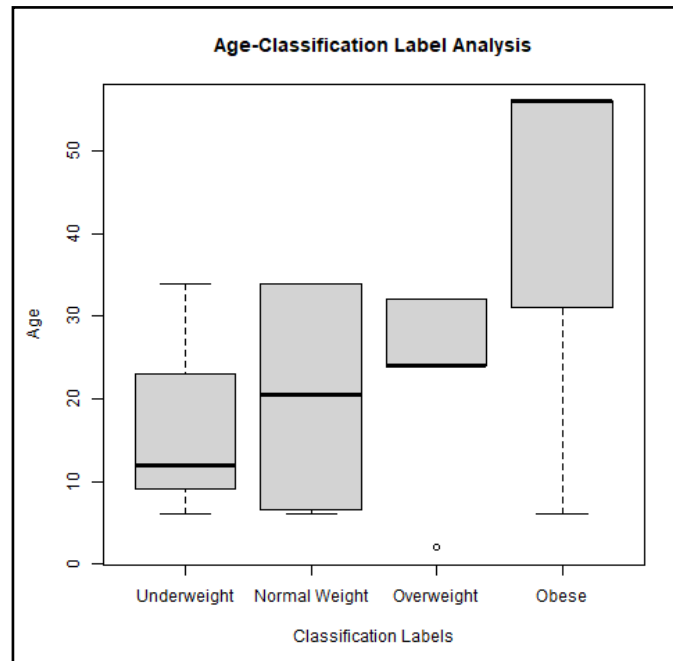
label_order = c('underweight', 'normal_weight', 'overweight', 'obese')
print(data$ClassificationLabels)
orderedClassificationLabels <- ordered(data$ClassificationLabels, levels =
 label_order)
data$ClassificationLabels = orderedClassificationLabels

png("age-classification.png")
boxplot(data$Age ~ data$ClassificationLabels, main = "Age-Classification Label
Analysis", xlab = "Classification Labels", ylab = "Age")
dev.off()
```

### OUTPUT:

```
> source("C:/Users/BIOINFO/Desktop/R Practicals/Practs2/P3.R")
[1] "underweight" "Underweight" "Overweight" "obese" "overweight"
[6] "Normal weight" "obese" "Underweight" "Overweight" "overweight"
[11] "Obese" "Overweight" "Underweight" "Underweight" "overweight"
[16] "Normal weight" "Normal weight" "Normal weight" "Underweight" "underweight"
[21] "Underweight"
```

**Figure 7:** Classification labels from the Dataset



**Figure 8:** BoxPlot of Classification Labels against Age

### **INTERPRETATION:**

Younger individuals tend to be in the Underweight or Normal Weight categories. Older individuals are more commonly found in the Overweight and Obese categories. The Obese category has the widest age distribution, suggesting obesity affects all age groups. The presence of outliers (e.g., a young person classified as Overweight) indicates exceptions to general trends.

### **RESULTS:**

Simple programs based on the concept of data visualization used in R programming language were coded and executed to display the structure of the data and the column names, to display a simple scatter plot between the age of the passenger and the passenger fare, to display a multiple line plot between the age of the passenger and the passenger fare, to display the box plot to analyse age and survival, to display the output of the contents of CSV file, to display the box plot of Age v/s fasting blood sugar, to display the classification labels from the dataset, to display the box plot of classification labels against age and to interpret all the observations and graph plots.

### **CONCLUSION:**

Data visualization was demonstrated using simple programs that were coded and executed in R programming language.

## **REFERENCES:**

1. Omar, A. (2023, October 27). Data visualization in R - Scaler topics. Scaler Topics.  
<https://www.scaler.com/topics/data-visualization-in-r>
  2. R Data Visualization - TPoint Tech. (n.d.). www.tpointtech.com.  
<https://www.tpointtech.com/r-data-visualization>
  3. Research Guides: Data Visualization in R: Introduction. (n.d.).  
<https://libguides.princeton.edu/R-Visualization>
  4. Silveira, O. (2025, March 5). Data Visualization with R. Dataquest.  
<https://www.dataquest.io/path/data-visualization-with-r>
-

## **PRACTICAL 3: R** **BIOCONDUCTOR**

### **AIM:**

To create simple programs based on the concept of Bioconductor module used in R Programming.

### **INTRODUCTION:**

Bioconductor is an open-source and open-development software project primarily based on the R programming language. It is designed for the analysis and comprehension of genomic data generated by wet lab experiments in molecular biology. The project was initiated in the fall of 2001 and is overseen by the Bioconductor core team, with significant contributions from international institutions.

### **Key Features of Bioconductor**

- 1. Open Source and Open Development:** Bioconductor is fully open source, allowing users to access and modify the code. It also encourages open development, where anyone can contribute to the project by creating or improving packages and documentation.
- 2. R-Based Packages:** Most Bioconductor components are distributed as R packages, which are add-on modules for R. These packages provide a wide range of functionalities for analyzing genomic data, including microarray analysis, sequence data analysis, and SNP data analysis.
- 3. Statistical and Graphical Methods:** Bioconductor offers access to powerful statistical and graphical methods for genomic data analysis. This includes tools for data preprocessing, identifying differentially expressed genes, graph theoretical analyses, and plotting genomic data.
- 4. Genome Annotation:** The project provides software for associating genomic data with biological metadata from databases like GenBank, LocusLink, and PubMed. This facilitates the integration of annotation data into analyses.
- 5. Documentation and Reproducible Research:** Each Bioconductor package includes vignettes, which are documents that provide step-by-step instructions on using the package. These vignettes support reproducible research by ensuring that analyses can be easily replicated.

### Major Bioconductor Packages

|                             |                                                                            |
|-----------------------------|----------------------------------------------------------------------------|
| <b>GenomicRanges</b>        | Provides classes for describing genomic ranges and their annotations.      |
| <b>Biostrings</b>           | Handles DNA and other sequence data.                                       |
| <b>GenomicAlignments</b>    | Manages aligned reads from sequencing experiments.                         |
| <b>GenomicFeatures</b>      | Offers tools for working with genomic features like transcripts and genes. |
| <b>AnnotationDbi</b>        | Facilitates access to annotation databases.                                |
| <b>SummarizedExperiment</b> | Coordinates experimental data across different assays.                     |
| <b>rtracklayer</b>          | Imports and exports genome annotations in formats like BED, WIG, and GTF.  |

### Goals of Bioconductor

1. **Provide Access to Statistical Methods:** Offer a broad range of statistical and graphical methods for genomic data analysis.
2. **Integrate Biological Metadata:** Facilitate the inclusion of biological metadata in analyses.
3. **Promote Reproducible Research:** Ensure that analyses are reproducible through high-quality documentation and open tools.
4. **Foster Collaboration:** Encourage collaboration among researchers by providing a common software platform.

### Advantages of Bioconductor

1. **Comprehensive Suite of Tools:** Bioconductor offers a wide range of packages for analyzing genomic data, including tools for microarray analysis, sequence data analysis, and SNP data analysis.
2. **Interoperability:** Packages in Bioconductor are designed to work seamlessly with each other, allowing users to create complex workflows without worrying about data format conversions.
3. **Open Source and Open Development:** This allows for community-driven improvements, bug fixes, and the development of new tools.
4. **Statistical and Graphical Methods:** Provides access to powerful statistical and graphical methods, facilitating advanced data analysis and visualization.
5. **Reproducibility:** Each package includes vignettes that support reproducible research by providing step-by-step instructions on using the software.
6. **Ease of Installation and Use:** Bioconductor packages are easy to install and use, with a short learning curve due to their modular nature.

### Disadvantages of Bioconductor

1. **Steep Learning Curve for Beginners:** While individual packages are relatively easy to use, mastering the entire suite of tools can be overwhelming for new users.
2. **Dependence on R:** Users must be familiar with the R programming language to effectively utilize Bioconductor.
3. **Version Compatibility Issues:** Users may encounter compatibility issues between different versions of R and Bioconductor packages.
4. **Resource Intensive:** Some analyses, especially those involving large-scale sequencing data, can be computationally intensive and require significant resources.

### **Applications of Bioconductor**

1. **Genomic Data Analysis:** Bioconductor is widely used for analyzing high-throughput sequencing data, including RNA-seq, DNA-seq, and ChIP-seq.
2. **Microarray Analysis:** Provides mature facilities for analyzing microarray data, including differential expression analysis and quality control.
3. **Clinical Genomics and Transcriptomics:** Used in clinical settings for analyzing genomic and transcriptomic data, supporting personalized medicine approaches.
4. **Proteomics and Metabolomics:** Offers tools for analyzing proteomic and metabolomic data, facilitating multi-omics research.
5. **Cancer Genomics:** Packages like GNOSIS support cancer genomics by integrating clinical and genomic data for survival analysis and visualization.
6. **Flow Cytometry and Imaging:** Supports the analysis of flow cytometry and quantitative imaging data, expanding its utility beyond genomic data.

## **PROGRAMS:**

### **1. Bioconductor Basics**

#### **Question 1:**

How do you install and load Bioconductor packages in R? Write a script demonstrating the installation of Biostrings and GenomicRanges.

#### **CODE:**

```
Install Bioconductor packages
install.packages("BiocManager")

Install required packages
BiocManager::install(c("Biostrings", "GenomicRanges", "ggplot2", "vcfR", "dplyr"))

Load packages
library(Biostrings)
library(GenomicRanges)
library(ggplot2)
library(vcfR)
library(dplyr)
```

#### **OUTPUT:**

```
package 'ape' successfully unpacked and MD5 sums checked
package 'bitops' successfully unpacked and MD5 sums checked
package 'cli' successfully unpacked and MD5 sums checked
package 'colorspace' successfully unpacked and MD5 sums checked
package 'digest' successfully unpacked and MD5 sums checked
package 'farver' successfully unpacked and MD5 sums checked
package 'glue' successfully unpacked and MD5 sums checked
package 'Rcpp' successfully unpacked and MD5 sums checked
package 'RCurl' successfully unpacked and MD5 sums checked
package 'rlang' successfully unpacked and MD5 sums checked
package 'stringi' successfully unpacked and MD5 sums checked
package 'vegan' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
C:\Users\BIOINFO\AppData\Local\Temp\Rtmpgry4fM\downloaded_packages
```

**Figure 1:** Installation and loading of Bioconductor packages in R



## 2. Working with DNA Sequences using Biostrings

### Question 2.1:

Write a script to create a DNAString object in R and print its contents.

#### CODE:

```
Create a DNA sequence
dna_seq <- DNAString("ATGCGTACGTAGCTAGCTA")
print(dna_seq)
```

#### OUTPUT:

```
The following objects are masked from 'package:stats':
 filter, lag
The following objects are masked from 'package:base':
 intersect, setdiff, setequal, union

19-letter DNAString object
seq: ATGCGTACGTAGCTAGCTA
Warning messages:
1: package(s) not installed when version(s) same as or greater than current; use `force = TRUE` to re-install: 'Biostrings' 'GenomicRanges' 'ggplot2' 'vcfR' 'dplyr'
2: package 'ggplot2' was built under R version 4.2.3
3: package 'vcfR' was built under R version 4.2.3
4: package 'dplyr' was built under R version 4.2.3
```

**Figure 2.1:** Creating and displaying the contents of a DNAString object

### Question 2.2:

Write an R script to compute the GC content of a given DNA sequence using Biostrings.

#### CODE:

```
Function to compute GC content
gc_content <- function(sequence) {
 (sum(letterFrequency(sequence, c("G", "C")))) / nchar(sequence)) * 100
}

Example usage
gc_content(dna_seq)
```

#### OUTPUT:

```
> # Create a DNA sequence
> dna_seq <- DNAString("ATGCGTACGTAGCTAGCTA")
> print(dna_seq)
19-letter DNAString object
seq: ATGCGTACGTAGCTAGCTA
> # Function to compute GC content
> gc_content <- function(sequence) {
+ (sum(letterFrequency(sequence, c("G", "C")))) / nchar(sequence)) * 100
+ }
> # Example usage
> gc_content(dna_seq)
[1] 47.36842
```

**Figure 2.2:** Creating and displaying the contents of a DNAString object

**Question 2.3:**

Write an R script to find the reverse complement of a DNA sequence using Biostrings.

**CODE:**

```
Reverse complement of DNA sequence
rev_comp <- reverseComplement(dna_seq)
print(rev_comp)
```

**OUTPUT:**

```
> # Reverse complement of DNA sequence
> rev_comp <- reverseComplement(dna_seq)
> print(rev_comp)
19-letter DNAString object
seq: TAGCTAGCTACGTACGCAT
```

**Figure 2.3:** Displaying the reverse complement of a DNA sequence using Biostrings

**Question 2.4:**

Write a script to count the frequency of each nucleotide (A, T, G, C) in a given DNA sequence.

**CODE:**

```
Count the frequency of nucleotides
nt_freq <- alphabetFrequency(dna_seq, baseOnly = TRUE)
print(nt_freq)
```

**OUTPUT:**

```
> source("C:/Users/BIOINFO/Desktop/Prarthi Perl Programs/Bioconductor/P1.R")
19-letter DNAString object
seq: ATGCGTACGTAGCTAGCTA
19-letter DNAString object
seq: TAGCTAGCTACGTACGCAT
 A C G T other
 5 4 5 5 0
> print(nt_freq)
 A C G T other
 5 4 5 5 0
```

**Figure 2.4:** Counting the frequency of each nucleotide in a given DNA sequence

**Question 2.5:**

How would you read a DNA sequence from a FASTA file using Biostrings? Write a script to display the sequence and compute its length.

**CODE:**

```
Read a FASTA file
fasta_file <- readDNAStringSet("C:\\Users\\BIOINFO\\Desktop\\Prarthi Perl
Programs\\Bioconductor\\example.fasta")
print(fasta_file)

Get lengths of sequences
seq_lengths <- nchar(fasta_file)
print(seq_lengths)
```

### OUTPUT:

```
> # Read a FASTA file
> fasta_file <- readDNASTringSet("C:\\Users\\BIOINFO\\Desktop\\Prarthi Perl Program
s\\Bioconductor\\example.fasta")
> print(fasta_file)
DNASTringSet object of length 1:
 width seq names
[1] 358 AAGAACCCAGAGAGAACTTGAAGC...GGAGATGGAAATTGCAGAATACA AB049616.1 Paral...
```

**Figure 2.5:** Reading and displaying a FASTA file along with its sequence and length

### Question 2.6:

Write a script to extract and print only the first 20 bases of a DNA sequence stored in a DNASTringSet object.

### CODE:

```
sub_seq <- subseq(fasta_file, start=1, end=20)
print(sub_seq)
```

### OUTPUT:

```
> sub_seq <- subseq(fasta_file, start=1, end=20)
> print(sub_seq)
DNASTringSet object of length 1:
 width seq names
[1] 20 AAGAACCCAGAGAGAACTTGA AB049616.1 Paral...
```

**Figure 2.6:** Displaying only first 20 bases of a DNA sequence

### Question 2.7a:

Write an R script to convert DNA to RNA.

### CODE:

```
Define a DNA sequence
dna_seq <-
DNASTring("ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG")

Convert to RNA (Replace 'T' with 'U')
rna_seq <- RNAString(dna_seq)

Print RNA sequence
print(rna_seq)
```

### OUTPUT:

```
> # Define a DNA sequence
> dna_seq <- DNASTring("ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG")
> # Convert to RNA (Replace 'T' with 'U')
> rna_seq <- RNAString(dna_seq)
> # Print RNA sequence
> print(rna_seq)
39-letter RNAString object
seq: AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG
```

**Figure 2.7a:** Converting DNA to RNA

**Question 2.7b:**

Write an R script to convert DNA sequence into a protein sequence using Biostrings.

**CODE:**

```
Translate DNA sequence to protein
protein_seq <- translate(dna_seq)
print(protein_seq)
```

**OUTPUT:**

```
> # Convert to RNA (Replace 'T' with 'U')
> rna_seq <- RNAString(dna_seq)
> # Print RNA sequence
> print(rna_seq)
39-letter RNAString object
seq: AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG
> # Translate DNA sequence to protein
> protein_seq <- translate(dna_seq)
> print(protein_seq)
13-letter AAString object
seq: MAIVMGR*KGAR*
```

**Figure 2.7b:** Converting DNA to protein

**Question 2.8:**

Demonstrate how to search for a specific motif (e.g., "ATG") in a given DNA sequence using matchPattern().

**CODE:**

```
Find occurrences of "ATG" motif
motif_match <- matchPattern("ATG", dna_seq)
print(motif_match)
```

**OUTPUT:**

```
> # Find occurrences of "ATG" motif
> motif_match <- matchPattern("ATG", dna_seq)
> print(motif_match)
Views on a 39-letter DNAString subject
subject: ATGGCCATTGTAATGGGCCGCTGAAAGGGTCCCGATAG
views:
 start end width
[1] 1 3 3 [ATG]
[2] 13 15 3 [ATG]
```

**Figure 2.8:** Displaying the occurrences of the motif searched in a given DNA sequence

### 3. Handling FASTA and VCF files

#### Question 3.1:

Write an R script to read a FASTA file and display the length of each sequence it contains.

#### CODE:

```
Load Biostrings package
library(Biostrings)

Read the FASTA file
fasta_file <- readDNAStringSet("example.fasta")

Display length of each sequence
sequence_lengths <- width(fasta_file)
names(sequence_lengths) <- names(fasta_file)

Print sequence lengths
print(sequence_lengths)
```

#### OUTPUT:

```
> # Load Biostrings package
> library(Biostrings)
> # Read the FASTA file
> fasta_file <- readDNAStringSet("C:\\Users\\BIOINFO\\Desktop\\Prarthi Perl Program
s\\Bioconductor\\example.fasta")
> # Display length of each sequence
> sequence_lengths <- width(fasta_file)
> names(sequence_lengths) <- names(fasta_file)
> # Print sequence lengths
> print(sequence_lengths)
AB049616.1 Paralichthys olivaceus mRNA for keratin, partial cds
358
```

**Figure 3.1:** Reading and displaying the FASTA file sequence

#### Question 3.2:

Write an R script to extract sequences longer than 500 bases from a FASTA file.

#### CODE:

```
Filter sequences longer than 500 bases
long_sequences <- fasta_file[width(fasta_file) > 500]

Save filtered sequences to a new FASTA file
writeXStringSet(long_sequences, "filtered_sequences.fasta")

Print names of extracted sequences
print(names(long_sequences))
```

#### OUTPUT:

```
> # Filter sequences longer than 500 bases
> long_sequences <- fasta_file[width(fasta_file) > 500]
> # Save filtered sequences to a new FASTA file
> writeXStringSet(long_sequences, "filtered_sequences.fasta")
> # Print names of extracted sequences
> print(names(long_sequences))
character(0)
```

**Figure 3.2:** Extracting and displaying the sequences longer than 500 bases

**Question 3.3:**

How do you read and manipulate VCF files using the vcfR package? Write a script to load a VCF file and extract variant positions.

**CODE:**

```
Load vcfR package
library(vcfR)

Read a VCF file
vcf_data <- read.vcfR("example.vcf")

Extract variant positions
variant_positions <- extract.gt(vcf_data, element="POS")

Print first few positions
head(variant_positions)
```

**OUTPUT:**

```
> # Read a VCF file
> vcf_data <- read.vcfR("C:\\Users\\BIOINFO\\Desktop\\Prarthi Perl Programs\\Bioconductor\\example.vcf")
Scanning file to determine attributes.
File attributes:
 meta lines: 159
 header_line: 160
 variant count: 902
 column count: 10
Meta line 159 read in.
All meta lines processed.
gt matrix initialized.
Character matrix gt created.
 Character matrix gt rows: 902
 Character matrix gt cols: 10
 skip: 0
 nrows: 902
 row_num: 0
Processed variant: 902
All variants processed
> # Extract variant positions
> variant_positions <- extract.gt(vcf_data, element="POS")
> # Print first few positions
> head(variant_positions)
 NA12878
chrM_1_1 NA
chrM_2_2 NA
chrM_73_3 NA
chrM_74_4 NA
chrM_150_5 NA
chrM_151_6 NA
```

**Figure 3.3:** Reading and manipulating VCF files

**Question 3.4:**

Write an R script to count the number of SNPs in a VCF file using vcfR.

**CODE:**

```
Filter SNPs (Single Nucleotide Polymorphisms)
snp_variants <- vcf_data[vcf_data@fix[, "REF"] %in% c("A", "T", "C", "G")
& vcf_data@fix[, "ALT"] %in% c("A", "T", "C", "G"),]

Count the number of SNPs
num_snps <- nrow(snp_variants@fix)

Print SNP count
print(num_snps)
```

**OUTPUT:**

```
> # Filter SNPs (Single Nucleotide Polymorphisms)
> snp_variants <- vcf_data[vcf_data@fix[, "REF"] %in% c("A", "T", "C", "G") &
+ vcf_data@fix[, "ALT"] %in% c("A", "T", "C", "G"),]
> # Count the number of SNPs
> num_snps <- nrow(snp_variants@fix)
> # Print SNP count
> print(num_snps)
[1] 109
```

**Figure 3.4:** Counting the number of SNPs in a VCF file

**4. Data Manipulation with dplyr****Question 4.1:**

What is the dplyr package used for in R? Explain with an example.

**CODE:**

```
Load dplyr
library(dplyr)

Example dataset (genomic positions)
genomic_data <- data.frame(
 Chromosome = c("chr1", "chr1", "chr2", "chr2", "chr3"),
 Position = c(12345, 67890, 34567, 89012, 45678),
 GC_Content = c(45.6, 47.8, 43.2, 50.1, 48.0)
)

Filter for GC content > 45
filtered_data <- genomic_data %>% filter(GC_Content > 45)

Print result
print(filtered_data)
```

**OUTPUT:**

```
> # Load dplyr
> library(dplyr)
> # Example dataset (genomic positions)
> genomic_data <- data.frame(
+ Chromosome = c("chr1", "chr1", "chr2", "chr2", "chr3"),
+ Position = c(12345, 67890, 34567, 89012, 45678),
+ GC_Content = c(45.6, 47.8, 43.2, 50.1, 48.0)
+)
> # Filter for GC content > 45
> filtered_data <- genomic_data %>% filter(GC_Content > 45)
> # Print result
> print(filtered_data)
 Chromosome Position GC_Content
1 chr1 12345 45.6
2 chr1 67890 47.8
3 chr2 89012 50.1
4 chr3 45678 48.0
```

**Figure 4.1:** Demonstration of dplyr package with example

**Question 4.2:**

Write an R script using dplyr to filter and select specific columns from a genomic dataset.

**CODE:**

```
Select only Chromosome and Position columns where GC_Content > 45
selected_data <- genomic_data %>% filter(GC_Content > 45) %>%
 select(Chromosome, Position)

Print result
print(selected_data)
```

**OUTPUT:**

```
> # Select only Chromosome and Position columns where GC_Content > 45
> selected_data <- genomic_data %>% filter(GC_Content > 45) %>% select(Chromosome,
 Position)
> # Print result
> print(selected_data)
 Chromosome Position
1 chr1 12345
2 chr1 67890
3 chr2 89012
4 chr3 45678
```

**Figure 4.2:** Filtering and selecting specific columns from a genomic dataset

**Question 4.3:**

How would you summarize the GC content of multiple DNA sequences stored in a tibble using dplyr?

**CODE:**

```
install.packages("stringr")
library(stringr)

Example DNA sequences in a tibble
sequences <- tibble(
 ID = c("Seq1", "Seq2", "Seq3"),
 DNA = c("ATGCGCGGATTA", "GGGTTTCCCAAAGGG",
 "ATATGCATGCAT"))

Compute GC content
sequences <- sequences %>%
 mutate(GC_Content = (str_count(DNA, "G") + str_count(DNA, "C")) /
 nchar(DNA) * 100)

Print summary
print(sequences)
```



## OUTPUT:

```
package 'stringr' successfully unpacked and MD5 sums checked
The downloaded binary packages are in
 C:\Users\BIOINFO\AppData\Local\Temp\Rtmpgry4fM\downloaded_packages
> library(stringr)
Warning message:
package 'stringr' was built under R version 4.2.3
> # Example DNA sequences in a tibble
> sequences <- tibble(
+ ID = c("Seq1", "Seq2", "Seq3"),
+ DNA = c("ATGCGCGGATTA", "GGGTTTCCCAAAGGG", "ATATGCATGCAT")
+)
> # Compute GC content
> sequences <- sequences %>%
+ mutate(GC_Content = (str_count(DNA, "G") + str_count(DNA, "C")) / nchar(DNA) *
+ 100)
> # Print summary
> print(sequences)
A tibble: 3 x 3
 ID DNA GC_Content
<chr> <chr> <dbl>
1 Seq1 ATGCGCGGATTA 50
2 Seq2 GGGTTTCCCAAAGGG 60
3 Seq3 ATATGCATGCAT 33.3
```

**Figure 4.3:** Summarizing and displaying GC content of multiple DNA sequences

## Question 4.4:

Demonstrate how to use `group_by()` and `summarise()` in `dplyr` with a biological dataset.

## CODE:

```
Group data by Chromosome and summarize mean GC content
gc_summary <- genomic_data %>%
 group_by(Chromosome) %>%
 summarise(Mean_GC = mean(GC_Content))

Print summary
print(gc_summary)
```

## OUTPUT:

```
> # Group data by Chromosome and summarize mean GC content
> gc_summary <- genomic_data %>%
+ group_by(Chromosome) %>%
+ summarise(Mean_GC = mean(GC_Content))
> # Print summary
> print(gc_summary)
A tibble: 3 x 2
 Chromosome Mean_GC
<chr> <dbl>
1 chr1 46.7
2 chr2 46.7
3 chr3 48
```

**Figure 4.4:** Demonstrating use of `group_by()` and `sumamrise()`

## 5. Data Visualization with ggplot2

### Question 5.1:

Explain the role of ggplot2 in bioinformatics. Give necessary examples.

#### CODE:

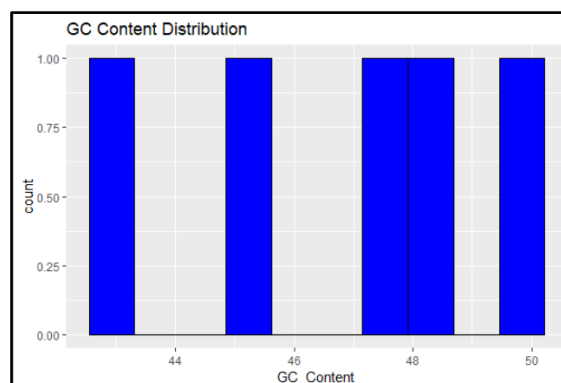
```
Load ggplot2
library(ggplot2)

Plot GC content distribution
ggplot(genomic_data, aes(x=GC_Content)) +
 geom_histogram(bins=10, fill="blue", color="black") +
 ggtitle("GC Content Distribution")
```

#### OUTPUT:

```
> # Load ggplot2
> library(ggplot2)
> # Plot GC content distribution
> ggplot(genomic_data, aes(x=GC_Content)) +
+ geom_histogram(bins=10, fill="blue", color="black") +
+ ggtitle("GC Content Distribution")
> |
```

**Figure 5.1:** Demonstrating the role of ggplot2



**Figure 5.1b:** Graph plot of GC content

### Question 5.2:

Write a script to plot a bar chart showing nucleotide frequencies using ggplot2.

#### CODE:

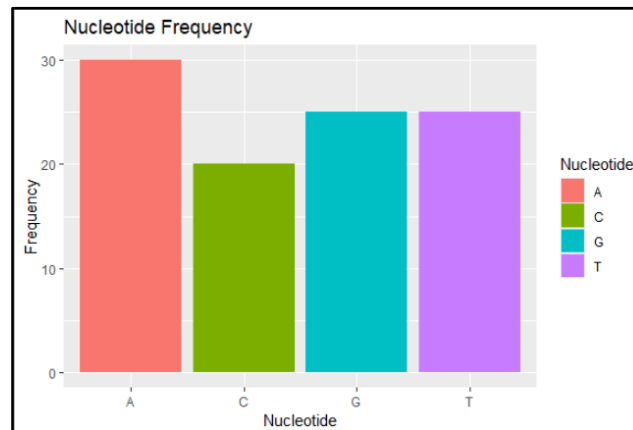
```
Example nucleotide frequency data
nucleotide_data <- data.frame(
 Nucleotide = c("A", "T", "C", "G"),
 Frequency = c(30, 25, 20, 25)
)

Plot bar chart
ggplot(nucleotide_data, aes(x=Nucleotide, y=Frequency, fill=Nucleotide)) +
 geom_bar(stat="identity") +
 ggtitle("Nucleotide Frequency")
```

## OUTPUT:

```
> # Example nucleotide frequency data
> nucleotide_data <- data.frame(
+ Nucleotide = c("A", "T", "C", "G"),
+ Frequency = c(30, 25, 20, 25)
+)
> # Plot bar chart
> ggplot(nucleotide_data, aes(x=Nucleotide, y=Frequency, fill=Nucleotide)) +
+ geom_bar(stat="identity") +
+ ggtitle("Nucleotide Frequency")
> |
```

**Figure 5.2a:** Demonstrating bar chart using ggplot2



**Figure 5.2b:** Bar chart of nucleotide frequencies

## Question 5.3:

Write an R script to visualize GC content distribution in multiple sequences using ggplot2.

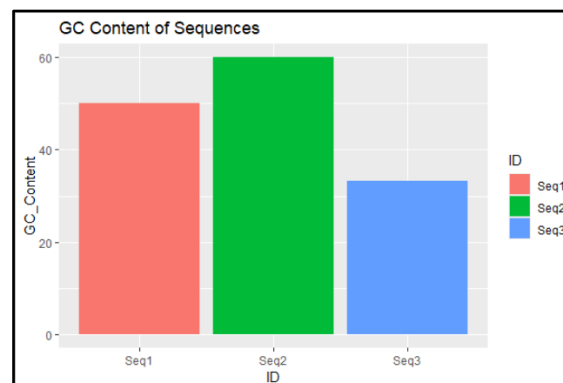
## CODE:

```
ggplot(sequences, aes(x=ID, y=GC_Content, fill=ID)) +
 geom_bar(stat="identity") +
 ggtitle("GC Content of Sequences")
```

## OUTPUT:

```
> ggplot(sequences, aes(x=ID, y=GC_Content, fill=ID)) +
+ geom_bar(stat="identity") +
+ ggtitle("GC Content of Sequences")
> |
```

**Figure 5.3a:** Demonstrating visualization of GC content distribution in multiple sequences



**Figure 5.3b:** Bar chart of GC content distribution in multiple sequences

## 6. Advanced Genomic Analysis with GenomicRanges

### Question 6.1:

Write an R script to create a GRanges object for storing genomic intervals.

#### CODE:

```
Load GenomicRanges
library(GenomicRanges)

Create GRanges object
gr <- GRanges(seqnames="chr1", ranges=IRanges(start=100, end=500))

Print GRanges object
print(gr)
```

#### OUTPUT:

```
> # Load GenomicRanges
> library(GenomicRanges)
> # Create GRanges object
> gr <- GRanges(seqnames="chr1", ranges=IRanges(start=100, end=500))
> # Print GRanges object
> print(gr)
GRanges object with 1 range and 0 metadata columns:
 seqnames ranges strand
 <Rle> <IRanges> <Rle>
[1] chr1 100-500 *

seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

**Figure 6.1:** Creating and displaying GRanges object for storing genomic intervals

### Question 6.2:

How can you find overlapping genomic regions using GenomicRanges? Provide an example.

#### CODE:

```
Define two sets of genomic ranges
gr1 <- GRanges(seqnames="chr1", ranges=IRanges(start=c(100, 300),
end=c(200, 400)))
gr2 <- GRanges(seqnames="chr1", ranges=IRanges(start=c(150, 350),
end=c(250, 450)))

Find overlaps
overlaps <- findOverlaps(gr1, gr2)

Print results
print(overlaps)
```

#### OUTPUT:

```
> # Define two sets of genomic ranges
> gr1 <- GRanges(seqnames="chr1", ranges=IRanges(start=c(100, 300), end=c(200, 400)))
> gr2 <- GRanges(seqnames="chr1", ranges=IRanges(start=c(150, 350), end=c(250, 450)))
> # Find overlaps
> overlaps <- findOverlaps(gr1, gr2)
> # Print results
> print(overlaps)
Hits object with 2 hits and 0 metadata columns:
 queryHits subjectHits
 <integer> <integer>
[1] 1 1
[2] 2 2

queryLength: 2 / subjectLength: 2
```

**Figure 6.2:** Finding and displaying overlapping genomic regions

**Question 6.3:**

Demonstrate how to count the number of genes falling within specific genomic intervals using GenomicRanges.

**CODE:**

```
Example gene positions
gene_ranges <- GRanges(seqnames="chr1", ranges=IRanges(start=c(150, 350, 600), end=c(250, 450, 700)))

Count genes within a specific range
gene_count <- countOverlaps(gr, gene_ranges)

Print count
print(gene_count)
```

**OUTPUT:**

```
> # Example gene positions
> gene_ranges <- GRanges(seqnames="chr1", ranges=IRanges(start=c(150, 350, 600), end=c(250, 450, 700)))
> # Count genes within a specific range
> gene_count <- countOverlaps(gr, gene_ranges)
> # Print count
> print(gene_count)
[1] 2
```

**Figure 6.3:** Counting the number of genes falling within specific genomic intervals

**RESULTS:**

Simple programs based on the concept of Bioconductor module used in R Programming were coded and executed to demonstrate the use of various Bioconductor packages in R such as Biostrings, reading and displaying FASTA files, converting sequences from DNA and RNA to proteins, handling FASTA and VCF files manipulating data with dplyr, visualizing data in the form of graph plots (example: bar charts) using ggplot2 and demonstrating advanced genomic analysis using GenomicRanges..

**CONCLUSION:**

Simple programs based on the concept of Bioconductor module used in R Programming.

**REFERENCES:**

1. Bioconductor - home. (n.d.). <https://www.bioconductor.org/>
2. Chapter 2 Learning R and Bioconductor | Orchestrating Single-Cell Analysis with Bioconductor. (n.d.). <https://bioconductor.org/books/3.12/OSCA/learning-r-and-more.html>
3. Introduction to Bioconductor and report generation with R - R Introductory Series. (n.d.). [https://bioinformatics.ccr.cancer.gov/docs/rintro/Lesson\\_8/](https://bioinformatics.ccr.cancer.gov/docs/rintro/Lesson_8/)